HAW HAMBURG

BACHELORTHESIS
Omar Kandil

Comparative Analysis of Deep Reinforcement Learning Methods in a Custom Game Environment

Faculty of Electrical Engineering, Media and Information Technology

Fakultät für Elektro-, Medien- und Informationstechnik

HAMBURG UNIVERSITY
OF APPLIED SCIENCES

Hochschule für Angewandte
Wissenschaften Hamburg

Omar Kandil

Comparative Analysis of Deep Reinforcement Learning Methods in a Custom Game Environment

Bachelor Thesis based on the examination and study regulations
for *Bachelor of Science Information Engineering*
at the Faculty of Electrical Engineering, Media and Information Technology
of Hamburg University of Applied Sciences
Supervising examiner: Prof. Dr. Marc Hensel
Second examiner: M.Sc. Lev Lazar

Day of delivery: 14. August 2026

Omar Kandil

Title of Thesis

Comparative Analysis of Deep Reinforcement Learning Methods in a Custom Game Environment

Keywords

Deep Reinforcement Learning, Deep Q-Network, Policy Gradient, Actor-Critic, Custom Game Environment, Neural Networks, Comparative Analysis

Abstract

This thesis presents the design of a custom game environment for deep reinforcement learning and a comparative evaluation of selected learning methods. The work investigates how different reinforcement learning approaches perform on the same task under identical experimental conditions. The implemented methods include Deep Q-Networks (DQN), policy-gradient methods, and actor-critic methods.

Omar Kandil

Thema der Arbeit

Vergleichende Analyse von Deep-Reinforcement-Learning-Methoden in einer benutzerdefinierten Spielumgebung

Stichworte

Deep Reinforcement Learning, Deep Q-Network, Policy Gradient, Actor-Critic, Custom Game Environment, Neuronale Netze, Vergleichende Analyse

Kurzzusammenfassung

Diese Arbeit präsentiert den Entwurf einer maßgeschneiderten Spielumgebung für Deep Reinforcement Learning und eine vergleichende Evaluierung ausgewählter Lernmethoden. Untersucht wird die Leistung verschiedener Reinforcement-Learning-Ansätze bei derselben Aufgabe unter identischen experimentellen Bedingungen. Zu den implementierten

Methoden gehören Deep Q-Networks (DQN), Policy-Gradient-Verfahren und Actor-Critic-Verfahren.

Contents

List of Figures **ix**

List of Tables **xi**

1 Introduction **1**

1.1 Motivation 1
1.2 Problem Statement 2
1.3 Objectives 2

2 Fundamentals **3**

2.1 Artificial Intelligence and Machine Learning 3
2.1.1 Conceptual Classification 3
2.1.2 Learning Paradigms 4
2.2 Neural Networks and Deep Learning 6
2.2.1 Structure of Neural Networks 6
2.2.2 Training, Loss Functions, and Optimization 8
2.3 Reinforcement Learning 10
2.3.1 Agent-Environment Interaction 10
2.3.2 Markov Decision Processes 12
2.3.3 Policies, Returns, and Value Functions 14
2.3.4 Exploration and Exploitation 16
2.4 Deep Reinforcement Learning Methods 17
2.4.1 Deep Q-Networks 17
2.4.2 Policy Gradient Methods 19
2.4.3 Actor-Critic Methods 22

3 Research Questions, Objectives, and Experimental Requirements **25**

3.1 Research Aim and Research Questions 25
3.2 Experimental System Context 26

3.3 Stakeholders . . . 27
3.4 Objectives Analysis . . . 29
3.4.1 Development of a Custom GridWorld Environment . . . 29
3.4.2 Suitability for Reinforcement Learning . . . 30
3.4.3 Comparability of Learning Methods . . . 30
3.4.4 Boundary Conditions . . . 31
3.5 Requirements Overview . . . 31
3.5.1 Functional Requirements . . . 31
3.5.2 Non-Functional Requirements . . . 32

4 Concept and Design **33**
4.1 Overall Approach . . . 33
4.2 System Architecture . . . 35
4.3 Concept of Game Logic and Environment . . . 37
4.3.1 GridWorld as a Reinforcement Learning Task . . . 38
4.3.2 State Space and Action Space . . . 39
4.3.3 Reward System . . . 40
4.3.4 Difficulty Configurations . . . 41
4.4 Comparison Criteria . . . 43

5 Environment Development **45**
5.1 GridWorld Environment . . . 45
5.1.1 Approach . . . 46
5.1.2 Environment Workflow . . . 46
5.1.3 Implementation . . . 46
5.2 Environment Configurations and Randomization . . . 49
5.2.1 Configuration Approach . . . 49
5.2.2 Randomization Workflow . . . 51
5.3 Rendering and Manual Environment Testing . . . 52
5.3.1 Rendering Approach . . . 53
5.3.2 Manual Testing Workflow . . . 54

6 Experimental Framework and Learning Agent Development **55**
6.1 Development Setup and Project Structure . . . 55
6.2 Random Agent . . . 59
6.2.1 Approach . . . 59
6.2.2 Execution Workflow . . . 60

6.2.3 Implementation . . . 61
6.3 DQN Agent . . . 61
6.3.1 Approach . . . 61
6.3.2 Network Architecture and Hyperparameters . . . 62
6.3.3 Training Workflow . . . 63
6.3.4 Implementation . . . 66
6.4 Policy-Gradient Agent . . . 68
6.4.1 Approach . . . 68
6.4.2 Network Architecture and Hyperparameters . . . 69
6.4.3 Training Workflow . . . 69
6.4.4 Implementation . . . 71
6.5 Actor-Critic Agent . . . 73
6.5.1 Approach . . . 74
6.5.2 Network Architecture and Hyperparameters . . . 75
6.5.3 Training Workflow . . . 76
6.5.4 Implementation . . . 78

7 Evaluation **81**
7.1 Evaluation Methodology . . . 82
7.2 Agent Development Observations . . . 82
7.2.1 DQN Development . . . 83
7.2.2 Policy Gradient Development . . . 83
7.2.3 Actor-Critic Development . . . 84
7.3 Experimental Results and Analysis . . . 85
7.3.1 Overview of Final Evaluation Results . . . 86
7.3.2 Environments Evaluation . . . 88
7.3.3 Stochastic Policy Evaluation . . . 95
7.4 Evaluation with respect to objectives and requirements . . . 96

8 Conclusion and Outlook **100**
8.1 Conclusion . . . 100
8.2 Outlook . . . 101

Bibliography **103**

A Training Plots **107**

Declaration **118**

List of Figures

2.1 Relationship between artificial intelligence, machine learning, deep learning and reinforcement learning . . . 4
2.2 A multiple layer neural network . . . 7
2.3 Scatter plot of input vs. actual and predicted values for the neural net approximation . . . 8
2.4 Reinforcement learning interaction loop between agent and environment . 11
2.5 A simplified MDP diagram with three states and two actions . . . 13
2.6 DQN architecture diagram . . . 19
2.7 Policy gradient action-selection process . . . 22
2.8 Actor-critic architecture . . . 24

3.1 System context diagram. . . . 27

4.1 System architecture of the custom reinforcement learning framework. . . . 37
4.2 An example layout for the GridWorld components. . . . 39

5.1 GridWorld step workflow activity diagram. . . . 47
5.2 Class diagram of the GridWorld environment implementation. . . . 50
5.3 Activity diagram of randomized start-goal sampling. . . . 53

6.1 Package diagram of the implemented GridWorld framework. . . . 58
6.2 Activity diagram of the DQN training workflow. . . . 65
6.3 Activity diagram of the policy gradient training workflow. . . . 72
6.4 Architecture diagram of the implemented actor-critic network. . . . 78

7.1 GridWorld easy layout . . . 89
7.2 GridWorld standard layout . . . 90
7.3 GridWorld open random layout . . . 91
7.4 Gridworld random obstacles layouts . . . 93
7.5 GridWorld short-term vs. long-term reward layout . . . 94

7.6 GridWorld fixed multi-target layout 95

A.1 Recent success rate during training in the random obstacles environment for DQN, policy gradient, and actor-critic 109
A.2 Entropy decay during training in the random obstacles environment for policy gradient and actor-critic 110
A.3 Recent average reward during training in the short-term vs. long-term reward environment for DQN, policy gradient, and actor-critic 111
A.4 Episode steps during training in the short-term vs. long-term reward environment for DQN, policy gradient, and actor-critic 112
A.5 Episode steps during training in the standard environment for DQN, policy gradient, and actor-critic 113
A.6 Recent success rate during training in the standard environment for DQN, policy gradient, and actor-critic 114
A.7 Recent average reward during training in the standard environment for DQN, policy gradient, and actor-critic 115
A.8 Recent average reward during training in the open random environment for DQN, policy gradient, and actor-critic 116
A.9 Entropy decay during training in the open random environment for policy gradient and actor-critic 117

List of Tables

4.1 Role of the selected methods in the thesis 35
4.2 Main GridWorld components 38
4.3 Action space 40
4.4 Reward concept including passable movement types 41
4.5 Designed environment configurations 42
4.6 Evaluation metrics used 44

5.1 Main methods of the GridWorld environment 48
5.2 Environment configuration methods 52

6.1 Main project modules 56
6.2 Main execution scripts 57
6.3 Shared training and evaluation settings 59
6.4 Programming language, libraries, and supporting modules used in the implementation 60
6.5 DQN network architecture 63
6.6 DQN hyperparameters 64
6.7 Policy gradient network architecture 69
6.8 Policy gradient hyperparameters 70
6.9 Actor-critic network architecture 76
6.10 Actor-critic hyperparameters 77

7.1 Summary of agent development observations and resulting decisions . . . 86
7.2 Random Agent reference evaluation 87
7.3 Greedy evaluation results across all environments 88
7.4 Stochastic evaluation results for policy-based methods 96
7.5 Evaluation of objectives and requirements 99

1 Introduction

This chapter first motivates the topic and explains why a custom environment is useful for comparing deep reinforcement learning methods. It then defines the problem addressed by the thesis and states the main objectives of the work.

1.1 Motivation

Reinforcement learning addresses sequential decision-making problems in which an agent learns behavior through interaction with an environment and feedback in the form of rewards. It became an important framework for studying autonomous decision-making, especially where explicit programming of the desired behavior is difficult. Deep reinforcement learning has extended this idea by combining neural networks with reinforcement learning, allowing agents to learn from more complex state representations and larger problem spaces.

Various categories of deep reinforcement learning methods exist, including value-based approaches such as Deep Q-Networks, policy-based approaches such as policy-gradient methods, and hybrid approaches like actor-critic methods represent different ways of learning decision policies. The original DQN work [17] showed that a neural network can be trained to estimate action values directly from high-dimensional input, while policy-gradient methods explicitly optimize the policy itself, and actor-critic methods combine policy learning with value estimation.

The motivation of this thesis is therefore to develop a custom environment for deep reinforcement learning and to use it as a basis for a structured comparison of selected reinforcement learning methods. By analyzing DQN, a policy-gradient method, and an actor-critic method within the same task setting, the thesis aims to contribute to a clearer understanding of the strengths and limitations of these approaches in a controlled environment.

1.2 Problem Statement

Although many reinforcement learning environments already exist, this thesis does not focus on using a predefined benchmark directly. Instead, the task is to design and implement a custom environment as part of the work itself and to use it as the basis for training and evaluation.

The central problem addressed in this thesis is how selected deep reinforcement learning methods can be implemented and compared equally in a self-developed environment under similar task conditions. This requires the definition of a suitable environment, as well as the development of an evaluation framework that allows meaningful comparison between the selected methods.

1.3 Objectives

The objective of this thesis is to develop a custom environment for deep reinforcement learning and use this environment as the basis for a comparative evaluation of selected reinforcement learning methods. For this purpose, the environment is designed and implemented with explicitly defined observations, actions, rewards, and termination conditions. On this basis, a Deep Q-Network, a policy-gradient method, and an actor-critic method are implemented and evaluated under identical conditions. The aim is to analyze the methods with regard to learning behavior, convergence, stability, and overall task performance, and then to identify their respective strengths and limitations in the context of a custom reinforcement learning environment.

2 Fundamentals

This chapter presents the theoretical foundations required to understand the design, implementation, and evaluation of the reinforcement learning system developed in this thesis. The chapter first explains the relevant concepts from artificial intelligence, machine learning, neural networks, reinforcement learning, and deep reinforcement learning. Finally, the chapter discusses reward design, robustness toward reward systems, and the selection of methods used for the experiments in this thesis.

The purpose of this chapter is not to cover the entire field of reinforcement learning. Instead, it focuses on the concepts that are directly relevant to this thesis: deep Q-Networks, policy gradient methods, actor-critic methods, and reward shaping.

2.1 Artificial Intelligence and Machine Learning

2.1.1 Conceptual Classification

Artificial intelligence is a broad field that is concerned with the development of systems that can perform tasks normally associated with intelligent behavior. Such tasks can include perception, planning, decision-making, problem solving, language processing, and autonomous control. Machine learning is a subfield of artificial intelligence in which systems improve their performance based on data or experience rather than being explicitly programmed for every possible situation [33]. Figure 2.1 abstractly demonstrates the conceptual classification.

In traditional programming, the developer writes explicit rules that determine how the program behaves. In Machine learning, the developer defines a model structure, an objective, and a training procedure. The model then adjusts its internal parameters based

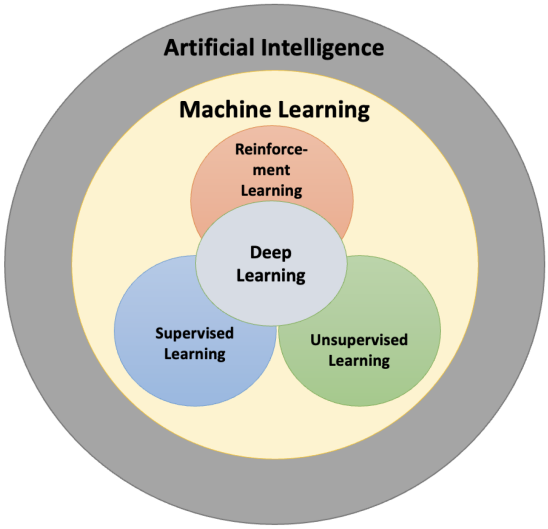


Figure 2.1: Relationship between artificial intelligence, machine learning, deep learning and reinforcement learning, adapted from [19].

on data or experience. This distinction is important because reinforcement learning systems do not rely on manually programmed action rules for every possible state. Instead, the agent learns behavior through repeated interaction with the environment.

Machine learning is commonly divided into supervised learning, unsupervised learning, and reinforcement learning. Supervised learning uses labeled data, where each input is paired with a desired output. Unsupervised learning uses unlabeled data and aims to discover structure or patterns [33]. Reinforcement learning is different because the learning system is an agent that actively interacts with an environment. The agent receives rewards, and the objective is to learn behavior that maximizes long-term return.

This thesis belongs to the area of deep reinforcement learning. It uses reinforcement learning because the agent learns through interaction with the environment. It uses deep learning because neural networks are used to represent value functions or policies. The resulting system combines decision-making, reward-based learning, and neural-network function approximation.

2.1.2 Learning Paradigms

Supervised learning, unsupervised learning, and reinforcement learning differ mainly in the type of feedback available during learning. In supervised learning, the model receives

the correct target output. A supervised model can therefore be trained by directly comparing its predictions with known labels. This is suitable for tasks such as image classification, regression, and object detection [8].

In this thesis, the practical learning task is represented by a GridWorld environment. A GridWorld is a discrete navigation environment in which an agent moves on a two-dimensional grid. The agent occupies one cell at a time and can choose from a small set of movement actions. The agent receives rewards or penalties based on the consequences of its actions.

The GridWorld task is not naturally formulated as a supervised learning problem. There is no fixed dataset containing the correct action for every possible state. Even if such a dataset were manually created, it would remove the main learning problem. The purpose of reinforcement learning is that the agent discovers useful behavior through interaction. The quality of an action may also depend on future consequences. For example, moving toward the goal may not immediately produce a positive reward, but it can be necessary to reach the goal several steps later [24].

Unsupervised learning is also not the main paradigm for this thesis. In unsupervised learning, the model usually searches for hidden structure in data. However, the GridWorld task is not mainly about discovering patterns in a static dataset. It is about learning how to act in an environment in order to maximize reward [24].

Reinforcement learning is the appropriate learning paradigm because it directly addresses sequential decision-making. The agent selects actions, observes consequences, and improves its behavior based on reward feedback. This makes it suitable for games, control systems, navigation tasks, and other problems in which decisions influence future states [24].

One important difference between reinforcement learning and supervised learning is that the agent's own behavior influences the data it receives. In supervised learning, the training data is usually fixed. In reinforcement learning, the agent generates experience by interacting with the environment. If the agent behaves poorly, it may repeatedly visit bad states and collect unhelpful experience. If it explores successfully, it may discover better states and improve faster. This dependency between behavior and training data is one reason why reinforcement learning is often more difficult to stabilize than supervised learning [24].

In this thesis, the reinforcement learning setting is combined with neural networks. This leads to Deep Reinforcement Learning. The DQN agent uses a neural network to estimate action values [18]. The Policy Gradient agent uses a neural network to represent a stochastic policy [31]. The Actor-Critic method combines both ideas by using one component for policy learning and another for value estimation [14].

2.2 Neural Networks and Deep Learning

2.2.1 Structure of Neural Networks

Artificial neural networks are computational models composed of connected layers of artificial neurons. A typical feedforward neural network contains an input layer, one or more hidden layers, and an output layer. The input layer receives the data, the hidden layers transform it, and the output layer produces the final prediction or decision-related values [8]. Figure 2.2 illustrates the structure of a feed-forward neural network.

Each neuron computes a weighted sum of its inputs and adds a bias term. For inputs $x_1, x_2, \ldots, x_n$, weights $w_1, w_2, \ldots, w_n$, and bias b, the pre-activation value is:

$$z = \sum_{i=1}^{n} w_i x_i + b \tag{2.1}$$

The neuron then applies an activation function a:

$$a = \sigma(z) \tag{2.2}$$

The activation function introduces non-linearity. Without non-linear activation functions, a network with multiple layers would still behave like a linear function. Non-linearity allows the network to approximate more complex relationships between inputs and outputs [8].

A commonly used activation function is the Rectified Linear Unit:

$$ReLU(x) = \max(0, x) \tag{2.3}$$

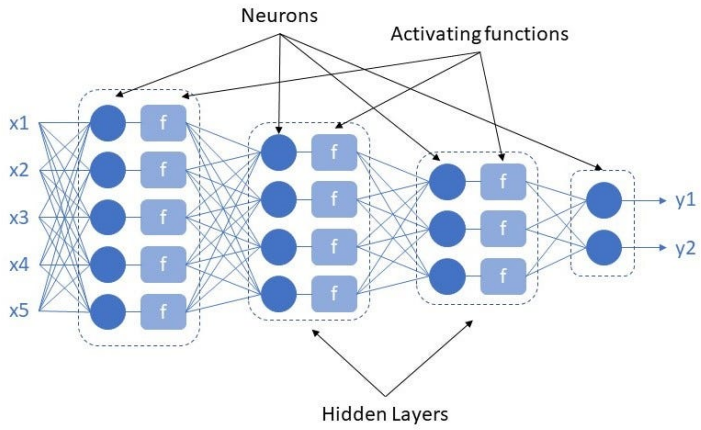


Figure 2.2: A multiple layer neural network, adapted from [32].

ReLU keeps positive values unchanged and maps negative values to zero. It is computationally simple and widely used in deep learning models. In the neural networks used in this thesis, ReLU activation is used in hidden layers [20, 8].

Neural networks are used in this thesis as function approximators. Function approximation means that the network learns a mapping from an input to an output relevant for the task. In reinforcement learning, this output may be a value estimate, a policy distribution, or both. This is useful because the model does not need a separate table entry for every possible state. Instead, the neural network learns a general mapping from states to decision-related outputs [8, 24]. Figure 2.3 shows the input versus the predicted values for the neural network approximation.

The current GridWorld uses a coordinate-based state representation. This means that the input is low-dimensional. Therefore, a feedforward network is sufficient. This is different from image-based reinforcement learning tasks, where convolutional neural networks are often used to process visual information. The decision not to use image input in the first version of the environment keeps the focus on algorithm comparison rather than computer vision [18].

The output layer depends on the selected method. In DQN, the output layer contains four values, one for each action. These values estimate the expected future return of each action [18]. In policy gradient, the output layer contains action logits. Logits are the raw, unbounded numerical scores output by a neural network before applying an activation function. These logits are converted into probabilities and used to sample

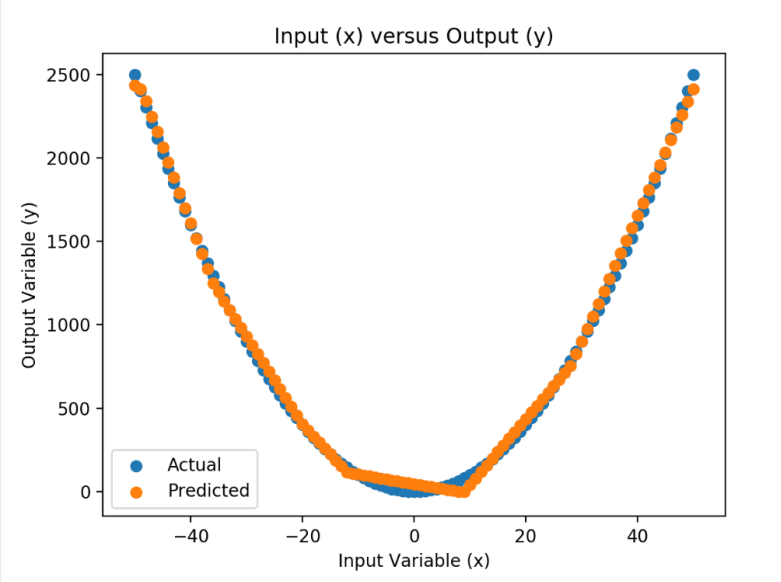


Figure 2.3: Scatter plot of input vs. actual and predicted values for the neural net approximation, adapted from [4].

actions [31]. In actor-critic methods, the architecture can contain an actor output and a critic output. The actor output represents the policy, while the critic output represents a value estimate [14, 24].

This flexibility is one of the reasons neural networks are useful in Deep Reinforcement Learning. The same basic model structure can be adapted to different learning objectives. The network input may remain the same, while the output interpretation and loss function change depending on the algorithm. [24, 18]

2.2.2 Training, Loss Functions, and Optimization

Training a neural network means adjusting its parameters so that its output becomes more useful for the task. The parameters include weights and biases. During training, the model produces an output, a loss function evaluates the quality of that output, and an optimizer updates the model parameters using gradient information [8].

The role of the loss function depends on the learning method. In supervised learning, a loss function might compare a predicted label to a known target label. In reinforcement learning, the situation is more complex because the target is often derived from rewards

and future estimates. The loss function therefore differs between DQN, policy gradient, and actor-critic methods [24].

In DQN, the network predicts Q-values. The loss measures the difference between the predicted Q-value and a target value based on the Bellman update [24, 18]. In the implementation of this thesis, a Huber-style loss is used because it is less sensitive to large errors than pure mean squared error [12].

A general form can be written as:

$$L_{\text{DQN}} = \text{Huber}(Q_{\text{policy}}(s, a), y) \tag{2.4}$$

where:

- $Q_{\text{policy}}(s, a)$ is the Q-value predicted by the policy network
- y is the target value

The Huber loss is defined as:

$$\text{Huber}(x, y) = \begin{cases} \frac{1}{2}(x - y)^2, & \text{if } |x - y| \leq \delta \\ \delta \cdot \left(|x - y| - \frac{1}{2}\delta\right), & \text{otherwise} \end{cases}$$

where:

- x is the predicted value
- y is the target value
- δ is the threshold at which the loss changes from quadratic to linear behavior

The Huber loss behaves like mean squared error for small prediction errors and like mean absolute error for larger errors. This makes it less sensitive to large temporal-difference errors than pure mean squared error, which can improve stability during DQN training.

In policy gradient, the loss is not based on a fixed target value. Instead, it depends on the log probability of selected actions and the returns obtained after those actions. Actions that lead to better returns should become more likely, while actions that lead to poor returns should become less likely [31, 24].

Actor-critic methods use at least two learning objectives. The actor is trained to improve the policy, while the critic is trained to improve value estimation. This means that actor-critic methods usually combine a policy loss and a value loss [14, 24].

Optimization is performed using gradient-based methods. The optimizer updates the network parameters in a direction that reduces the loss [8]. In practice, reinforcement learning training can be unstable because the training data is generated by the agent itself. As the policy changes, the distribution of visited states also changes. This makes reinforcement learning different from supervised learning, where the dataset is usually fixed before training [24].

For this reason, the implementation and evaluation of reinforcement learning systems require additional care. Training curves, evaluation metrics, and reproducibility measures are necessary to understand whether an agent is actually learning useful behavior. A decreasing loss alone is not enough to prove that a reinforcement learning agent performs well.

2.3 Reinforcement Learning

2.3.1 Agent-Environment Interaction

Reinforcement learning is based on interaction between an agent and an environment. The agent selects actions, and the environment responds with new observations and rewards. The agent's goal is to learn behavior that maximizes accumulated reward over time. This makes reinforcement learning suitable for sequential decision-making problems. Figure 2.4 shows the agent sending an action to the environment and receiving a state and reward in return. At time step t, the agent observes a state S_t, selects an action A_t, receives a reward R_{t+1}, and observes the next state S_{t+1} [24].

This interaction can be written as:

$$S_t \rightarrow A_t \rightarrow R_{t+1}, S_{t+1} \tag{2.5}$$

The process continues until the episode ends. In the custom GridWorld environment, an episode ends when the agent reaches the goal, reaches a trap, or reaches the maximum number of steps.

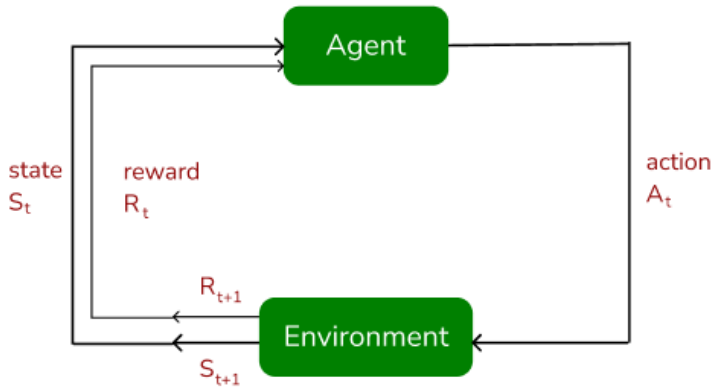


Figure 2.4: Reinforcement learning interaction loop between agent and environment, adapted from [7].

This is one of the most important figures in the chapter because it connects the abstract reinforcement learning framework to the implemented GridWorld.

Sutton and Barto [24] describe the agent-environment interface as the central structure of reinforcement learning, with the agent selecting actions and the environment returning rewards and new states. This structure is also used in the practical DRL book [33], where the fundamentals are introduced through interaction, reward, state transitions, and later deep methods such as DQN, policy gradient, and actor-critic.

The agent-environment interaction is important because it explains why reinforcement learning differs from supervised learning. The agent does not receive a correct action label. Instead, it receives feedback after acting. In many cases, this feedback is delayed. For example, in the GridWorld, the agent may need to take several steps before it receives the positive goal reward. The learning algorithm must therefore connect earlier actions with later consequences [24].

The reward signal defines what the agent should learn. If the reward function is designed poorly, the agent may learn behavior that maximizes the reward but does not match the intended task [24].

An episode is one complete attempt to solve the task. In the GridWorld, each episode begins with the agent at the start position. The episode ends successfully if the agent

reaches the goal. It ends unsuccessfully if the agent reaches a trap. It is truncated if the maximum number of steps is reached. This episode-based structure is suitable for training and evaluating the agents because each episode produces measurable outcomes: success, trap, timeout, total reward, and number of steps.

2.3.2 Markov Decision Processes

Many reinforcement learning problems can be formalized as Markov decision processes. A Markov decision process provides a mathematical description of a sequential decision-making problem [21, 27].

It is commonly defined as:

$$(S, A, P, R, \gamma) \tag{2.6}$$

where:

- S is the set of states
- A is the set of actions
- P is the transition probability function
- R is the reward function
- γ is the discount factor

Sutton and Barto [24] use finite Markov decision processes as the formal foundation for reinforcement learning, including the agent-environment interface, rewards, returns, policies, and value functions.

The transition function describes how likely it is to move from one state to another after selecting an action. In deterministic environments, an action may always lead to the same next state, except when blocked by walls or boundaries. In stochastic environments, the same action may lead to different outcomes with certain probabilities [21, 13]. The current GridWorld environment is mostly deterministic because movement actions have predictable effects.

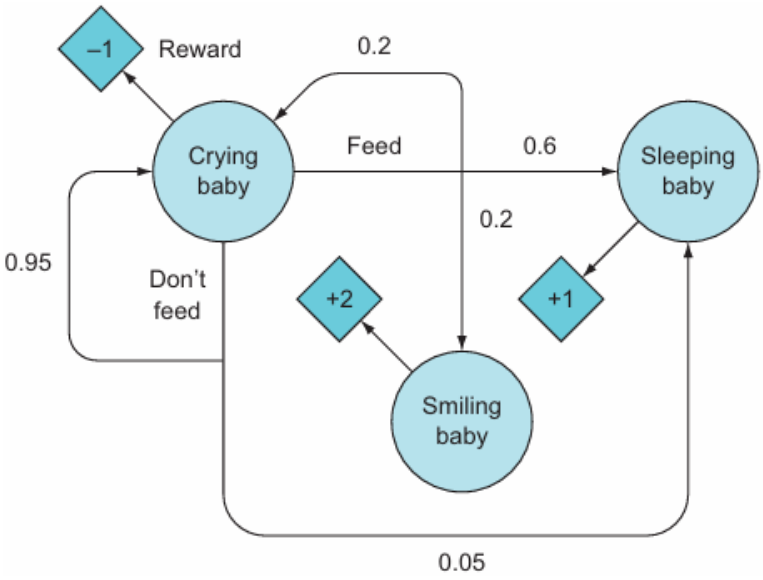


Figure 2.5: A simplified MDP diagram with three states and two actions, adapted from [33].

In Figure 2.5 we model the parenting decision process for taking care of an infant. If the baby is crying, we can either administer food or not, and with some probability the baby will transition into a new state, and we'll receive a reward of -1, +1, or +2 (based on the baby's satisfaction).

The Markov property states that the next state depends only on the current state and action, not on the complete history of previous states [21]. In the GridWorld, the next state is determined by the current agent position, the selected movement action, and the grid layout. The environment does not need to know the full path that led to the current position in order to compute the next state.

The MDP formulation is useful because it provides a common language for the algorithms used in this thesis. DQN estimates action values in an MDP. Policy gradient optimizes a parameterized policy in an MDP. Actor-critic methods use both a policy and a value estimate in an MDP [13, 27]. Therefore, the MDP framework connects the environment design to the learning methods.

In the custom GridWorld, the state space consists of all possible valid combinations of agent and goal positions. The action space consists of four discrete movement actions. The transition function is determined by the grid layout, walls, boundaries, and traps.

The reward function is defined by goal reaching, step penalties, invalid movement penalties, and trap penalties. The discount factor controls the importance of future rewards during learning.

This formal interpretation is helpful because it shows that the GridWorld is not just a visual game. It is a reinforcement learning problem with a defined state space, action space, transition function, reward function, and episode structure [21, 27].

2.3.3 Policies, Returns, and Value Functions

A policy π defines how the agent selects actions. A deterministic policy maps each state s to one action a:

$$a = \pi(s) \tag{2.7}$$

A stochastic policy defines a probability distribution over actions:

$$\pi(a|s) = P(A_t = a \mid S_t = s) \tag{2.8}$$

The distinction between deterministic and stochastic policies is important for the comparison in this thesis. DQN learns action values and typically selects the action with the highest estimated value during evaluation [30, 18]. Policy gradient methods learn a stochastic policy directly. This means the policy can assign different probabilities to different actions, and actions can be sampled from this distribution [31]. The return describes the accumulated reward from a time step onward.

For an episodic task, the discounted return can be described as the discounted sum of future rewards [27, 3] as shown in Equation 2.9:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1} \tag{2.9}$$

where:

- G_t is the return at time step t

- R_{t+k+1} is the reward received at a future time step $t + k + 1$
- T is the final time step of the episode
- γ is the discount factor

The discount factor controls the importance of future rewards. A discount factor close to 1 means that future rewards are important. A discount factor close to 0 means that the agent mainly considers immediate rewards [3].

The return is especially important in the GridWorld task because successful behavior produces delayed reward. The agent receives the main positive reward only after reaching the goal. Earlier actions may receive negative step rewards, but they can still be necessary for reaching the goal later. The learning method must therefore assign credit to earlier actions that contributed to later success. This is known as the credit assignment problem [23].

A state-value function estimates the expected return from a state under a policy [3, 27]:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s] \tag{2.10}$$

where:

- $V^{\pi}(s)$ is the value of state s under policy π
- $\mathbb{E}_{\pi}$ is the expected value if the agent follows policy π

An action-value function estimates the expected return after taking a specific action in a specific state [30, 27]:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a] \tag{2.11}$$

Value functions are central to value-based reinforcement learning methods. DQN uses a neural network to approximate the action-value function [30, 18]. In contrast, Policy gradient methods optimize the policy directly [31]. Actor-critic methods use both ideas by learning a policy and using value estimates to guide the policy update [14].

The difference between a value function and a policy is important. A value function estimates how good a state or action is. A policy defines how the agent behaves. Some

methods first learn values and then derive behavior from those values. Other methods learn the behavior directly [27, 14]. This distinction is one of the main reasons why DQN and Policy gradient behave differently in the experiments.

In the GridWorld task, the value of an action depends on long-term consequences. Moving right may be good in one state if it leads toward the goal, but bad in another state if it leads toward a trap or wall. Therefore, the learning algorithm must consider the state-action relationship rather than simply learning that one action is always good [30, 23].

2.3.4 Exploration and Exploitation

A reinforcement learning agent must balance exploration and exploitation. Exploration means trying actions whose consequences are not yet fully known. Exploitation means choosing actions that currently appear to be good. Both are necessary for successful learning [2, 28].

If the agent explores too little, it may converge prematurely to a poor policy. For example, in the GridWorld, the agent may repeatedly choose a path that leads to a trap simply because it has not discovered a better path to the goal. If the agent explores too much, it may behave randomly for too long and fail to exploit useful knowledge [28, 13].

Different learning methods use different exploration mechanisms. A random agent explores completely because it selects all actions uniformly at random. DQN commonly uses epsilon-greedy exploration [18, 29]. With probability ϵ, the agent selects a random action. With probability $1-\epsilon$, it selects the action with the highest predicted Q-value:

$$a = \begin{cases} \text{random action}, & \text{with probability } \epsilon \\ \arg\max\limits_{a} Q(s,a), & \text{with probability } 1-\epsilon \end{cases} \tag{2.12}$$

This strategy allows the agent to exploit the currently best estimated action most of the time, while still occasionally selecting random actions to explore alternatives [29, 18].

Policy gradient methods explore differently. They use a stochastic policy, so actions are sampled from a probability distribution. If one action has a probability of 70%, it will usually be selected, but other actions can still be sampled. This allows exploration to be built into the policy itself [26].

Exploration also influences how the results should be interpreted. A method that fails early may not necessarily be incapable of solving the task. It may have failed to explore useful trajectories. Similarly, a method that succeeds during training but fails during greedy evaluation may have learned useful action probabilities without forming a stable deterministic policy [13, 28].

2.4 Deep Reinforcement Learning Methods

Deep reinforcement learning combines reinforcement learning with neural networks. Instead of storing policies or value functions in tables, neural networks are used as function approximators. This allows reinforcement learning methods to handle larger or more complex state spaces [6, 1]. In this thesis, the state space is still compact, but neural networks are used to compare deep reinforcement learning method families under the same environment conditions.

2.4.1 Deep Q-Networks

Deep Q-networks are value-based reinforcement learning methods. They combine Q-learning with neural networks. Q-learning learns an action-value function $Q(s, a)$, which estimates the expected return for selecting action a in state s [30]. A DQN approximates this function using a neural network [18].

In the custom GridWorld, the DQN receives the state vector:

$$s = (\text{agent_row}, \text{agent_col}, \text{goal_row}, \text{goal_col}) \tag{2.13}$$

The output layer contains one value for each action:

$$Q(s, \text{up}), Q(s, \text{down}), Q(s, \text{left}), Q(s, \text{right}) \tag{2.14}$$

During greedy action selection, the selected action is:

$$a = \arg\max_{a} Q(s, a) \tag{2.15}$$

This makes DQN easy to interpret in the GridWorld environment. The network estimates how promising each movement action is, and the agent selects the action with the highest estimated value. Figure 2.6 shows the state vector as input, a feedforward neural network, and Q-values as output. It also shows that the agent chooses the maximum Q-value during greedy evaluation.

The DQN target is based on the immediate reward and the best estimated future value [30, 18]:

$$y = r + \gamma \max_{a'} Q_{\text{target}}(s', a') \tag{2.16}$$

where:

- r is the reward after executing the action
- γ is the discount factor (between 0 and 1), which determines how much the agent values future rewards versus immediate ones
- s' is the next state observed after the action
- a' is the action in the next state that produces the highest possible Q-value
- Q_{target} is the target network

The policy network is trained to make its prediction closer to this target.

Mnih et al. [18] introduced DQN as a method that combines reinforcement learning with deep neural networks. Their work showed that a single deep reinforcement learning agent could learn to play multiple Atari games using visual input. Two important stabilizing mechanisms in DQN are experience replay and target networks.

Experience replay stores transitions in a replay buffer. A transition has the form:

$$(s, a, r, s', \text{done}) \tag{2.17}$$

During training, the agent samples random mini-batches from the replay buffer. This reduces the correlation between consecutive training samples and allows the model to reuse earlier experiences [15, 18]. This is important because reinforcement learning data is generated sequentially, and consecutive states are often highly related.

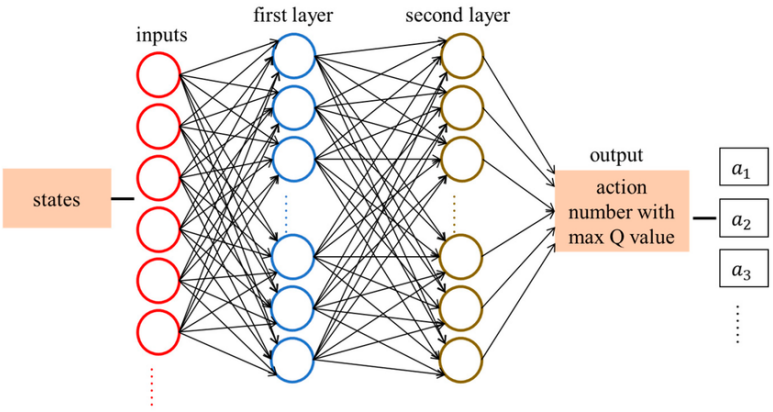


Figure 2.6: DQN architecture diagram, adapted from [5].

The target network improves stability by separating the network used for action-value prediction from the network used to compute target values. The policy network is updated frequently. The target network is updated only periodically by copying weights from the policy network. This makes the training target more stable [18].

However, DQN also has limitations. It is naturally designed for discrete action spaces. It also requires mechanisms such as experience replay, target networks, and exploration scheduling. These mechanisms make the method more complex than simple tabular Q-learning, but they are necessary for stable learning with neural networks [18, 1].

Another limitation is that DQN does not directly learn a stochastic policy. It learns values and then derives behavior from those values. During evaluation, the action with the highest Q-value is usually selected. This works well in many discrete control tasks, but it differs from policy-based methods, where the policy itself is directly optimized [30, 26].

2.4.2 Policy Gradient Methods

Policy gradient methods are policy-based reinforcement learning methods. Instead of learning action values and deriving a policy from them, they optimize the policy directly [25, 31].

A parameterized policy can be written as:

$$\pi_\theta(a|s) \tag{2.18}$$

where θ represents the trainable parameters of the policy network [25].

The policy network outputs action logits. These logits are converted into a probability distribution over the four actions. During training, the agent samples an action from this distribution.

This difference is important. DQN answers the question: "How valuable is each action in this state?" In other words, the output of the DQN is an estimate of the action-value function $Q(s, a)$, which represents the expected discounted return after taking action a in state s. Policy gradient answers the question: "How likely should each action be in this state?" DQN learns values, while policy gradient learns action probabilities.

The implemented policy gradient method is based on REINFORCE. REINFORCE is a Monte Carlo policy-gradient method introduced by Williams [31]. The method updates the policy after a complete episode. During the episode, the agent stores the log probability of each selected action and the rewards received after each action. After the episode ends, the discounted returns are computed and used to update the policy [31].

A simplified REINFORCE loss is:

$$L = -\log \pi_\theta(a_t|s_t)G_t \tag{2.19}$$

The negative sign appears because neural network optimizers minimize loss, while reinforcement learning aims to maximize return. If an action leads to a high return, minimizing the loss increases the probability of that action. If an action leads to a poor return, its probability is reduced.

A major weakness of basic REINFORCE is high variance. Since the update is based on complete episode returns, the learning signal can be unstable. This is especially problematic in environments with sparse or delayed rewards [9]. The standard GridWorld configuration has this property because the main positive reward is received only after reaching the goal, while intermediate actions receive negative step rewards.

To improve the stability of the policy gradient update, a moving-average baseline is used. In this context, the baseline is not a baseline method. Instead, it is a running reference

value computed from previous episode returns. It represents an estimate of the return that the agent usually achieves. The return of the current episode is then compared with this moving average to determine whether the selected actions performed better or worse than expected [9, 25].

The learning signal is computed as the difference between the discounted return and the moving-average baseline:

$$A_t = G_t - b_t \tag{2.20}$$

where:

- G_t is the discounted return from time step t
- b_t is the moving-average baseline
- A_t indicates whether the obtained return is above or below the recent average performance of the policy

If the return is better than the baseline, the selected actions are reinforced. If the return is worse than the baseline, their probabilities are reduced.

The improved loss also includes entropy regularization:

$$L = -\log \pi_\theta(a_t|s_t)A_t - \beta H(\pi_\theta(\cdot|s_t)) \tag{2.21}$$

where:

- H is the entropy of the policy distribution
- β is the entropy coefficient

Entropy measures how spread out the action probabilities are. A high-entropy policy assigns similar probabilities to multiple actions. A low-entropy policy strongly prefers one action. Entropy regularization encourages exploration by preventing the policy from becoming deterministic too early [16, 11].

Figure 2.7 shows that once an action is sampled from the policy network's probability distribution, it produces a new state and reward. The reward signal is used to reinforce

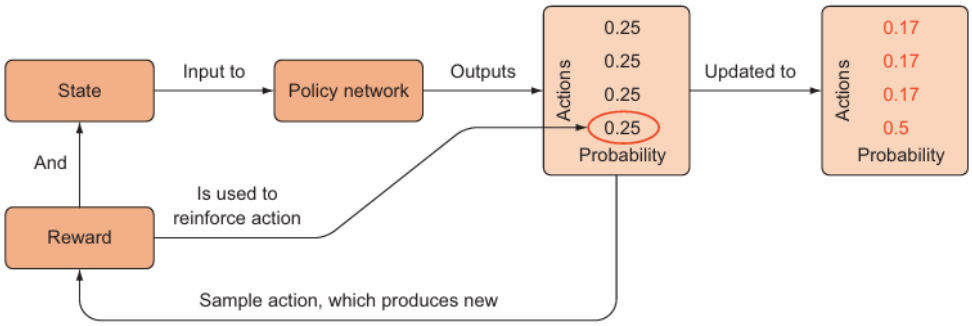


Figure 2.7: Policy gradient action-selection process, adapted from [33].

the action that was taken, that is, it increases the probability of that action given the state if the reward is positive, or it decreases the probability if the reward is negative.

The evaluation of policy gradient requires special attention. Since the learned policy is stochastic, two evaluation modes can be meaningful. In greedy evaluation, the action with the highest probability is selected. In stochastic evaluation, actions are sampled from the learned distribution. Greedy evaluation tests whether the policy has learned a stable deterministic path. Stochastic evaluation tests whether useful behavior is present in the probability distribution.

Policy gradient methods are important for comparison because they represent a different learning principle from DQN. They directly optimize the policy and can naturally represent stochastic behavior [25, 31]. However, the experiments show that they can be more sensitive to the reward structure.

2.4.3 Actor-Critic Methods

Actor-critic methods combine policy-based and value-based reinforcement learning. The actor learns a policy and selects actions. The critic estimates value information and evaluates the actor's behavior. This combination is designed to keep the advantages of direct policy learning while reducing the high variance of basic policy gradient methods [14, 10].

The actor can be represented as a parameterized policy:

$$\pi_\theta(a|s) \tag{2.22}$$

The critic can estimate a state-value function:

$$V_w(s) \tag{2.23}$$

where w represents the critic parameters.

The key idea is that the critic provides feedback about whether an action was better or worse than expected. A central concept is the advantage function [22]:

$$A(s,a) = Q(s,a) - V(s) \tag{2.24}$$

If the advantage is positive, the action was better than expected. If it is negative, the action was worse than expected. The actor can use this information to update the policy more efficiently than with raw returns alone.

Actor-critic methods are a natural next step after policy gradient. In REINFORCE, the policy update depends on full episode returns. This can produce high-variance updates and slow learning. In actor-critic methods, the critic provides a value estimate that helps reduce this variance [10, 22]. This makes the method more stable in many settings.

Mnih et al.[16] proposed asynchronous actor-critic methods and showed that actor-critic variants can train neural network controllers effectively across several reinforcement learning tasks. Their asynchronous actor-critic approach also demonstrated the stabilizing effect of using multiple actor-learners and value estimates .

Figure 2.8 shows the general overview of actor-critic models. First, the actor predicts the best action and chooses the action to take, which generates a new state. The critic network computes the value of the old state and the new state. The relative value of S_{t+1} is called its advantage, and this is the signal used to reinforce the action that was taken by the actor.

Actor-Critic methods are more complex than DQN or basic policy gradient because they require two learning components. The actor and critic must both be trained, and the quality of the actor update depends on the quality of the critic estimate. If the critic estimates values poorly, the actor may receive misleading feedback. Despite this complexity, actor-critic methods are important because they often provide better learning stability than pure policy gradient methods.

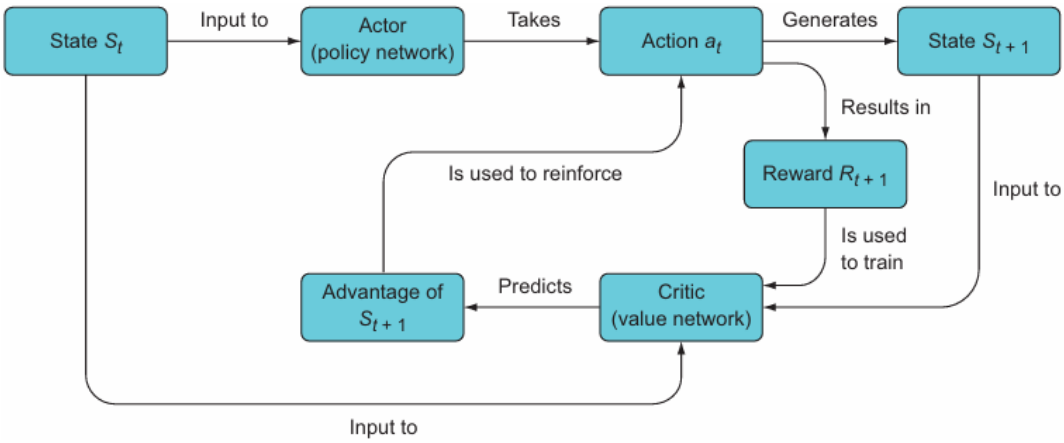


Figure 2.8: Actor-critic architecture, adapted from [33].

3 Research Questions, Objectives, and Experimental Requirements

This chapter defines the research direction, objectives, and experimental requirements of the thesis. Since the purpose of the work is to gain insights into the behavior of selected deep reinforcement learning methods, the chapter does not only describe software requirements. Instead, it first formulates the research aim and the guiding research questions, then explains the objectives that must be achieved in order to answer these questions, and finally derives the experimental and implementation-related requirements for the developed framework.

The requirements in this chapter are therefore not understood only as technical system requirements. They also reflect the conditions that must be fulfilled so that the custom environment and the implemented learning agents can support a meaningful comparison of DQN, policy gradient, and actor-critic under controlled conditions.

3.1 Research Aim and Research Questions

The overall aim of this thesis is to investigate how selected deep reinforcement learning methods behave when they are trained and evaluated in the same custom GridWorld environment. The focus is not only on whether an agent can reach the goal, but also on how the learned behavior differs between value-based, policy-based, and actor-critic approaches.

The thesis is guided by the following research questions:

- **RQ1:** How can a custom GridWorld environment be designed so that different deep reinforcement learning methods can be compared under controlled conditions?

- **RQ2:** How do DQN, policy gradient, and actor-critic differ with respect to success rate, accumulated reward, episode length, and stability?
- **RQ3:** How do the selected methods react to more difficult environment configurations, such as randomized start-goal positions, delayed rewards, and multiple valid targets?
- **RQ4:** How does the difference between greedy and stochastic evaluation influence the interpretation of policy-based methods?

These research questions define the direction of the thesis. The following sections describe the system context, objectives, and requirements that are necessary to answer them.

3.2 Experimental System Context

The experimental system context describes the main components that are required to investigate the research questions defined above. The developed framework consists of a custom GridWorld environment, a set of learning agents, training and evaluation scripts, result logging, and visualization utilities.

The experimental setup consists of a custom GridWorld environment, a set of learning agents, training and evaluation scripts, result logging, and visualization utilities. The GridWorld environment provides the tasks through which the behavior of the methods can be observed. The learning agents represent the selected method families investigated in this thesis, namely DQN, policy gradient, and actor-critic. The training and evaluation scripts ensure that all methods are executed in a consistent way, while the recorded metrics form the basis for the later comparative analysis.

Figure 3.1 gives a compact overview of this context. It shows that the learning agents interact with the GridWorld environment during training and evaluation. The experiment configuration controls the selected environment variant, method, and parameters. The resulting metrics are stored and later used for comparison and analysis.

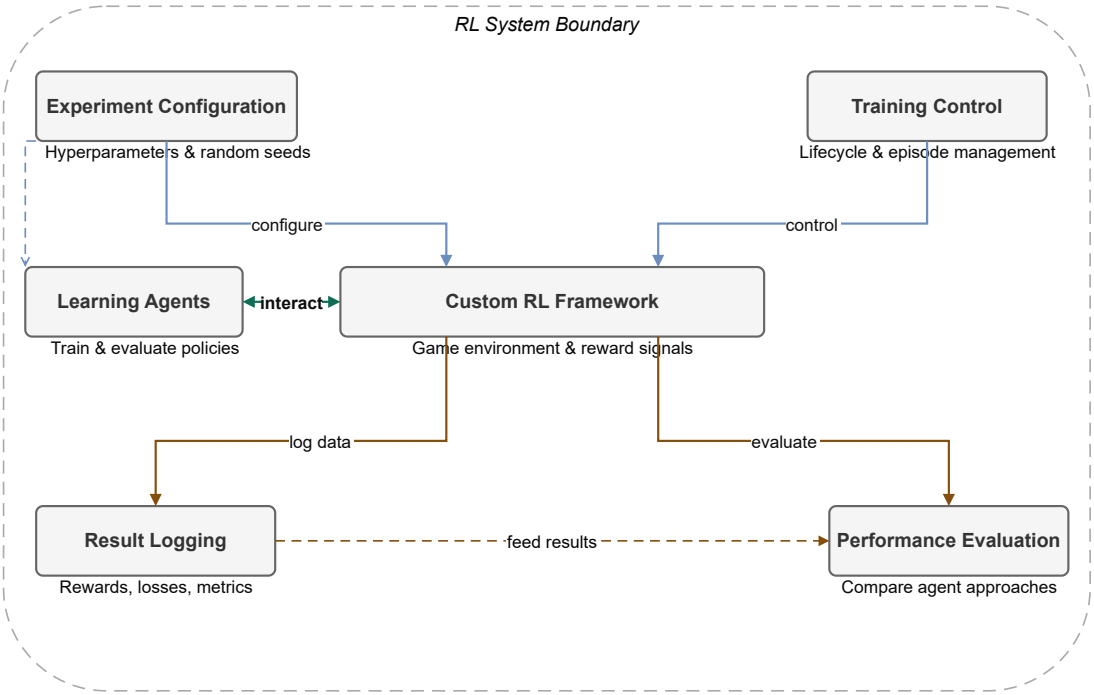


Figure 3.1: System context diagram.

3.3 Stakeholders

The stakeholders of this thesis are the people who influence the direction of the work, evaluate its outcome, or may benefit from the developed framework in the future. Since the thesis is mainly research-oriented, the stakeholders are not only considered with respect to system requirements. Their expectations are also relevant for defining the objectives of the work, the research questions, and the value of the final results.

The First Supervisor: Prof. Dr. Marc Hensel is the first supervisor of this thesis and therefore the main academic stakeholder. His motivation for the topic is to investigate reinforcement learning methods beyond DQN and to understand their advantages, disadvantages, and possible usefulness for future projects. Therefore, the thesis should not only show whether the agents can solve the GridWorld tasks, but also provide a structured comparison of different method families.

Another motivation is the educational value of the work. The professor is preparing teaching material on reinforcement learning and would like to guide students through

different learning approaches, including value-based, policy-based, and actor-critic methods. For this reason, the thesis should provide a clear and reproducible example that can help students understand how such methods are implemented, trained, evaluated, and compared in a custom environment.

A further motivation is the academic curiosity to observe the resulting behavior of the different agents and to relate the findings to the theoretical characteristics of the methods. From this perspective, the thesis should provide not only implementation results, but also an interpretation of why the agents behave differently under the selected environment configurations.

The Developer: The developer is the student and author of this thesis. The personal motivation for working on this topic is to gain practical experience with deep reinforcement learning and to understand how different method families behave when they are applied to the same custom environment. The work is therefore not limited to implementing algorithms, but also includes designing the environment, testing different configurations, interpreting the results, and learning how reinforcement learning experiments can be structured and evaluated. The developer's objective is to create a complete and understandable framework that supports the comparison of different learning methods.

The Second Examiner: The second examiner, Mr. Lev Lazar, provides an additional technical and practical perspective on the work. His interest is that the developed framework is not only academically understandable, but also structured in a way that can be inspected, reproduced, and potentially extended. This includes clear code organization, understandable experiment execution, useful result logging, and a design that can serve as a basis for future improvements or related work.

Future Students and Researchers: Future students and researchers are potential users of the developed framework and the results of this thesis. This includes students who want to understand basic deep reinforcement learning methods, students who may extend the framework in a future thesis, and researchers or interested readers who want to compare learning behavior in a controlled GridWorld environment. Their expectation is that the work provides a clear explanation of the environment, the implemented agents, the evaluation procedure, the observed differences between the methods, and the framework should be understandable and executable on standard computers without dedicated hardware or GPU acceleration. For this reason, documentation, reproducibility, and a clear connection between research questions, experiments, and results are important.

3.4 Objectives Analysis

The general objective of this thesis is to design and implement a custom reinforcement learning environment and to use it as a basis for comparing selected deep reinforcement learning methods. To make this objective more solid, it is divided into several detailed objectives. For each objective, it must be possible to determine whether it has been achieved. Therefore, this section analyzes the intended results, possible verification criteria, boundary conditions, and consequences for the later requirements.

3.4.1 Development of a Custom GridWorld Environment

The first objective is to develop a custom GridWorld environment that can be used as a controlled platform for reinforcement learning experiments. The environment should not be taken from a ready-made benchmark, but should be implemented as part of the thesis.

This decision has several reasons. First, a self-developed environment provides full control over the task structure. The grid layout, state representation, action space, reward function, terminal events, randomization behavior, and evaluation conditions can be adapted to the research questions of the thesis. This would be more difficult when using a predefined benchmark with fixed assumptions.

Second, implementing the environment makes it possible to understand the agent-environment interaction at code level. This includes the reset function, the step function, state transitions, reward calculation, termination, truncation, and result logging. Since reinforcement learning strongly depends on the interaction between agent and environment, this understanding is important for interpreting the later results.

Third, the development of a custom environment provides practical experience in building simplified virtual environments. Such environments can be understood as small experimental models of larger systems. This is relevant because future reinforcement learning applications often require custom simulation environments before they can be applied to real-world problems. Therefore, the GridWorld environment provides a compact first step toward understanding how such virtual environments can be created and used for learning experiments.

This objective is achieved if the environment can be reset, accepts valid agent actions, updates its internal state correctly, calculates rewards, identifies terminal or truncated episodes, and supports the environment configurations required for the comparison.

The main boundary condition is that the environment must remain simple enough to be implemented, executed, and understood without dedicated hardware or special computing resources. At the same time, it must not be so trivial that all methods solve it without meaningful learning. This balance is important because the framework should be useful not only for this thesis, but also for future students who may want to understand or extend the environment in later work.

3.4.2 Suitability for Reinforcement Learning

The second objective is to ensure that the environment is suitable for reinforcement learning. This means that the agent must be able to interact with the environment through a repeated action-feedback loop. After each action, the environment must return a new observation, a reward, and information about whether the episode has ended. This structure follows the standard agent-environment interaction used in reinforcement learning, where an agent learns by selecting actions and receiving rewards over time.

This objective can be considered achieved if the environment supports repeated episodes and if a random agent can interact with it without errors. In addition, the random agent results should show that the environment is neither impossible nor completely easy. The easy configuration is then used to verify that the learning pipeline can be tested, while the other configurations provide a more challenging scenario.

3.4.3 Comparability of Learning Methods

The third objective is to make the environment suitable for comparing different reinforcement learning methods. Since the thesis focuses on an evolution from DQN, policy-gradient methods, and actor-critic methods, the environment must allow these approaches to be trained and evaluated under identical task conditions. DQN and its extensions are especially suitable for discrete action spaces, so the initial environment should provide a small and clearly defined set of discrete actions.

This objective is achieved if all selected learning methods can use the same state representation, action space, reward function, and evaluation criteria. The comparison should not depend on different environments or manually adjusted rules for individual algorithms. Instead, the same environment configuration should be used for each method.

3.4.4 Boundary Conditions

Several boundary conditions limit the scope of the thesis. First, the work must be achievable within the time frame of a bachelor thesis. Accordingly, the initial implementation focuses on a single-agent GridWorld environment rather than a multi-agent scenario. Second, the environment should be implemented by the author and not used as a ready-made Gymnasium benchmark. Third, the action space should remain discrete in the first version, because this supports the planned DQN implementation. Finally, the number of algorithms must remain limited so that each method can be explained, implemented, and evaluated properly.

These boundary conditions lead to a phased development strategy. The environment is first implemented and validated without learning algorithms. After that, DQN can be implemented and evaluated before extending the comparison to additional methods.

3.5 Requirements Overview

Based on the system environment, stakeholders analysis and objectives analysis, the following functional and non-functional requirements are derived. Functional requirements describe the behavior that the system must provide. Non-functional requirements describe quality-related constraints such as reproducibility, modularity, and feasibility. They have an impact on decisions on project design and architecture, which may or may not have an impact on technical implementation.

3.5.1 Functional Requirements

FR1: The system shall provide a custom environment with a defined observation space, action space, reward function, and episode termination logic.

FR2: The environment shall provide a configurable reward system so that different reward structures can be implemented and compared.

FR3: The environment shall provide a reinforcement learning interface that is compatible with Gymnasium environment API[1]. This includes functionality for resetting an episode, processing an action step, returning observation and reward information, distinguishing between terminated and truncated episodes, and rendering the environment state.

FR4: The system shall implement the selected deep reinforcement learning methods.

FR5: The system shall record performance metrics during training and evaluation.

FR6: The system shall support textual or graphical visualization of the environment state. This is required for manual inspection, debugging, and verification of the implemented environment configurations.

FR7: The system shall support a comparative evaluation of the implemented methods under identical task conditions.

FR8: The system shall store and report relevant experiment results, including success rate, timeout rate, trap rate, average reward, average episode length, and method-specific training values where applicable.

3.5.2 Non-Functional Requirements

NFR1: The system shall be implemented in a modular way so that the environment and algorithms can be modified independently.

NFR2: The system shall allow reproducible experiments through random seeds and documented hyperparameters.

NFR3: The framework shall be extensible so that additional environment configurations or learning methods can be added without changing the complete system structure.

NFR5: The implementation and documentation shall be understandable for future students or researchers who want to reproduce, inspect, or extend the work. This includes clear project structure, meaningful file names, readable code organization, and explanations of the main environment and agent components.

[1] https://gymnasium.farama.org/

4 Concept and Design

This chapter presents the concept and design of the reinforcement learning system developed in this thesis. The previous chapter defined the objectives, stakeholders, boundary conditions, and requirements of the system. Based on these requirements, this chapter explains how the system is designed before the actual implementation is described in Chapters 5 and 6.

The main purpose of the design is to create a controlled experimental framework for comparing selected deep reinforcement learning methods in a custom GridWorld environment. The framework is not limited to the environment itself. It also includes the agents, training procedures, evaluation procedures, result logging, model saving, plotting, and visualization. This is necessary because the thesis does not only aim to train individual agents, but to compare several methods under equal and reproducible conditions.

The design follows a modular structure. The environment is separated from the learning algorithms, and the training process is separated from the evaluation process. This allows different agents to interact with the same environment interface and makes the results comparable. The same state representation, action space, reward system, and evaluation metrics are used for all methods. As a result, observed performance differences can be related mainly to the learning methods and not to different task conditions.

4.1 Overall Approach

The overall approach of this thesis is to design and implement a custom GridWorld environment and use it as a common test environment for selected deep reinforcement learning methods. The environment represents a discrete navigation task. An agent starts at a defined position and has to reach a goal position by selecting movement

actions. Depending on the difficulty configuration, the environment can include walls, invalid movements, a trap, and a maximum step limit.

A GridWorld environment was selected because it is simple enough to be understandable, but still suitable for studying important reinforcement learning behavior. The task requires the agent to learn a sequence of decisions instead of a single isolated action. The agent must explore the environment, discover useful movement directions, avoid inefficient actions, and learn how to reach the goal before the episode ends.

The environment is custom-built instead of using an existing benchmark directly. This gives full control over the grid layout, reward values, state representation, terminal states, and difficulty settings. This is important for the thesis because the environment itself is part of the developed system. It also allows the experiments to focus on the comparison of learning methods under clearly defined and transparent conditions.

The learning methods are added step by step. First, a random agent is used as a non-learning baseline. It selects actions randomly and therefore shows how difficult the environment is without learning. After that, DQN is implemented as the value-based method. Policy gradient is implemented as the policy-based method. Actor-critic is implemented as the hybrid method that combines policy learning with value estimation.

The general workflow of the thesis is shown in the following steps:

1. Design a custom GridWorld environment
2. Define multiple difficulty configurations
3. Validate the environment with a random agent
4. Implement and train DQN
5. Implement and train policy gradient
6. Implement and train actor-critic
7. Evaluate all methods using common metrics
8. Compare final performance, learning behavior, and robustness toward the reward system

The comparison is not based only on whether an agent eventually reaches the goal. It also considers how often the agent fails, how it fails, how many steps it needs, and how stable the learning process is. This is important because two methods can reach the same final success rate but still differ strongly during training. For example, one method may solve the task directly with the original reward system, while another may require reward shaping or additional stabilization techniques. Table 4.1 summarizes the role of each selected method in the thesis.

4.2 System Architecture

The system architecture is designed as a modular reinforcement learning framework. The main components are the GridWorld environment, the agents, the training scripts, the evaluation scripts, the result logs, the saved models, the plotting scripts, and the visualization component.

The GridWorld environment defines the task. It contains the grid layout, the agent position, the goal position, the trap position, the reward values, and the step limit. It provides a reset operation for starting a new episode and a step operation for applying an action. The environment returns the next state, reward, episode status, and event information after each action.

The agents define the decision-making behavior. Each agent receives the current state and selects an action. The random agent selects actions uniformly at random. The DQN agent selects actions based on learned Q-values. The policy gradient agent selects actions

Method	**Method family**	**Role in the thesis**
Random Agent	Non-learning baseline	Provides a reference for task difficulty without learning
Deep Q-Network	Value-based learning	Represents action-value learning in a discrete action space
Policy Gradient	Policy-based learning	Represents direct policy optimization and stochastic action selection
Actor-Critic	Hybrid learning	Represents policy learning supported by value estimation

Table 4.1: Role of the selected methods in the thesis

from a learned probability distribution. The actor-critic agent also selects actions from a learned policy, but additionally uses a critic to estimate the value of states.

The training scripts coordinate the learning process. A training script creates the selected environment, initializes the corresponding agent, runs training episodes, updates the model, logs episode results, and saves the trained model. Each learning method has its own training script because the update process differs between DQN, policy gradient, and actor-critic.

The evaluation scripts are separated from the training scripts. During evaluation, the trained model is loaded and tested without further learning. This separation is important because training behavior includes exploration and parameter updates, while evaluation should measure the learned behavior under fixed conditions.

The result logging component stores training and evaluation results in CSV files. These files contain episode-level information such as total reward, number of steps, final event, success, trap, timeout, and recent success rate. Depending on the method, additional values such as epsilon, loss, entropy, advantage, or value estimates are also stored.

The plotting scripts use the saved CSV files to generate graphs. These plots are used to analyze learning progress and compare methods. The visualization component is used to display the GridWorld and demonstrate how an agent moves through the environment. Visualization is kept separate from training so that the learning process does not depend on graphical rendering.

Figure 4.1 illustrates the overall structure of the developed reinforcement learning framework and the interaction between its main components. The training and evaluation scripts coordinate the execution process, while the agent and the GridWorld environment exchange states, actions, and rewards during each episode. Training results and models are stored for later analysis, and the logged data are used to generate plots and visualizations. This modular architecture supports comparability, reproducibility, and the integration of additional algorithms or environment configurations in the future.

This architecture supports the requirements of modularity, comparability, and reproducibility. Since the agents use the same environment interface, all methods can be compared under identical conditions. Since the training and evaluation scripts are separated, the final performance can be measured independently from the learning process. Since result files and models are saved, experiments can be inspected and repeated later.

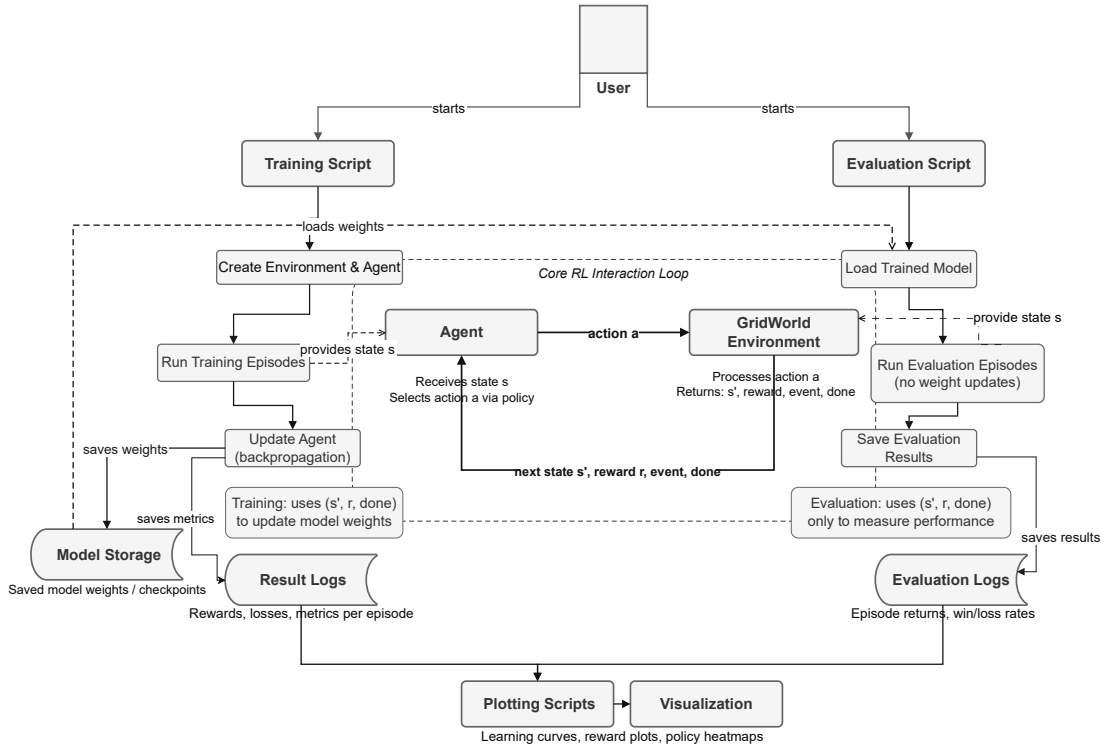


Figure 4.1: System architecture of the custom reinforcement learning framework.

The architecture also supports future extensions. Additional algorithms can be added by implementing new agent and experiment scripts. Additional difficulty levels can be added by defining new environment configurations. More visualizations can also be added without changing the core learning logic.

4.3 Concept of Game Logic and Environment

The GridWorld environment is designed as the main task environment of the thesis. It provides the agent with a discrete navigation problem in which the objective is to reach a goal cell from a start cell. The environment is intentionally simple in its visual structure but meaningful in its learning behavior.

The environment contains the basic elements required for a reinforcement learning task: states, actions, rewards, and episode-ending conditions. The agent observes the current state, selects an action, receives feedback from the environment, and continues until the

episode ends. This creates the repeated interaction loop needed for training reinforcement learning agents.

The environment design also supports clear evaluation. Each episode ends with a specific event, such as reaching the goal, reaching the trap, or reaching the step limit. These events make it possible to calculate success rate, trap rate, and timeout rate. Therefore, the evaluation is not limited to total reward alone.

4.3.1 GridWorld as a Reinforcement Learning Task

The GridWorld is modeled as an episodic navigation task. At the beginning of each episode, the environment is reset and the agent is placed at the start position. The agent then selects actions until it reaches the goal, reaches a trap, or exceeds the maximum number of allowed steps.

The goal represents successful task completion. The trap represents an undesirable terminal state. Invalid movements, such as trying to move outside the grid or into blocked cells, are penalized and do not move the agent. The maximum step limit prevents endless episodes and creates a timeout condition for agents that fail to solve the task efficiently. Table 4.2 lists the main components of the GridWorld task and Figure 4.2 shows an example layout for these components.

Component	**Description**
Agent '*A*'	Entity controlled by the selected method
Start position ■	Initial position at the beginning of each episode
Goal position ■	Target cell that defines success
Free cells	Valid cells through which the agent can move
Walls or boundaries '#'	Areas that block movement
Trap ■	Negative terminal state in the Standard configuration
Step limit	Maximum number of actions before timeout

Table 4.2: Main GridWorld components

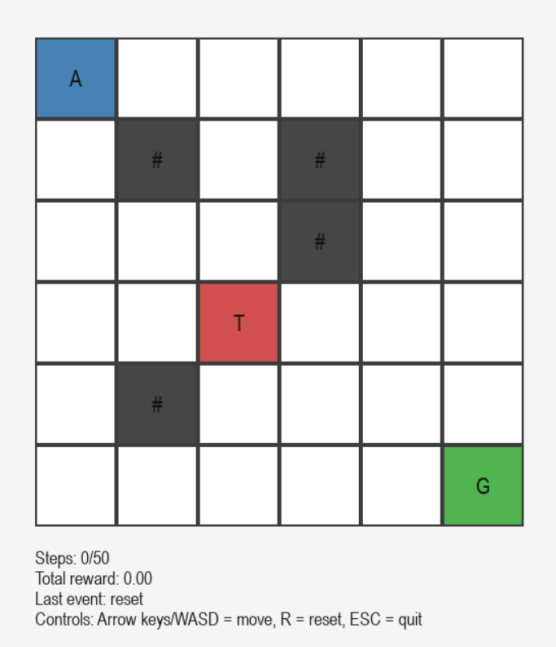


Figure 4.2: An example layout for the GridWorld components.

4.3.2 State Space and Action Space

The state representation is coordinate-based. The environment returns the position of the agent and the position of the goal. The state is represented as:

$$s = (\text{agent_row}, \text{agent_col}, \text{goal_row}, \text{goal_col}) \tag{4.1}$$

This compact representation is sufficient for the current GridWorld because the agent only needs to know its own position and the goal position to learn useful movement behavior. The state is also suitable for all selected neural network methods because it can be used directly as input to feedforward networks.

In addition to the original single-goal state representation, a later environment required small extensions of the observation design.

In fixed multi-target environment, the state representation is extended because more than one terminal target exists. In this case, the agent observes its own position and the

positions of both available targets:

$$s = (\text{agent_row}, \text{agent_col}, \text{target1_row}, \text{target1_col}, \text{target2_row}, \text{target2_col})$$

The action space consists of four discrete movement actions. Each action attempts to move the agent by one cell in one direction. Table 4.3 summarizes the GridWorld action space.

The same state and action spaces are used for random agent, DQN, policy gradient, and actor-critic. This is important for fairness because the methods should be compared under the same information and action conditions.

4.3.3 Reward System

The reward system defines the feedback signal for the agent. It is designed to encourage reaching the goal efficiently and discourage invalid or unsafe behavior. Table 4.4 summarizes the reward concept used in the GridWorld environment.

The agent receives a positive reward when it reaches the goal. It receives a negative step reward for normal movement, which encourages shorter paths. Invalid movements receive a stronger penalty, because repeatedly trying impossible movements should not be useful. In addition to these basic events, the environment can also distinguish between different types of passable movement. This means that some movements are allowed and preferred, while other movements are allowed but discouraged through additional penalties. This distinction is useful because not every undesirable movement should immediately end the episode like a trap.

The reward system is important for the comparison because the selected methods do not react to rewards in exactly the same way. DQN learns action values based on immediate

Action ID	**Action**	**Description**
0	Up	Move one cell upward
1	Down	Move one cell downward
2	Left	Move one cell left
3	Right	Move one cell right

Table 4.3: Action space

Event or movement type	Purpose
Goal reached	Encourage successful task completion and terminate the episode successfully.
Normal step	Apply a small step penalty to encourage shorter and more efficient paths.
Preferred passable movement	Represent movements that are allowed and desirable within a specific configuration. These movements can receive normal or positive reward adjustments to guide the agent toward a preferred route.
Discouraged but passable movement	Represent movements through areas or transitions that the agent is allowed to use, but should avoid if a better alternative exists. These movements receive an additional penalty without terminating the episode.
Invalid movement	Discourage blocked or impossible movements, such as moving outside the grid or into a wall.
Trap reached	Penalize unsafe terminal behavior and end the episode unsuccessfully.
Step limit reached	End unsuccessful episodes that take too long.

Table 4.4: Reward concept including passable movement types

and future reward estimates. Policy gradient learns from sampled episode outcomes and can therefore be more sensitive to delayed positive feedback. Actor-critic uses value estimation to support policy learning.

For this reason, reward robustness is part of the evaluation. A method that solves the environment reliably using the original reward system is considered more robust in this setup than a method that requires additional reward shaping or shows unstable behavior when rewards are delayed or path-dependent.

4.3.4 Difficulty Configurations

The GridWorld environment is designed to support several difficulty configurations. These configurations are used to increase the task complexity step by step while keeping the same basic environment interface. This allows the learning methods to be tested first under simple and controlled conditions and later under more challenging conditions. Table 4.5 compares each environment configuration and purpose.

Configuration	Start and goal	Obstacles	Purpose
Easy	Fixed	Walls, no trap	Validate the basic learning pipeline.
Standard	Fixed	Walls and one trap	Main fixed-layout comparison task.
Open Random	Random	No walls or traps	Test general navigation with changing start and goal positions.
Random Obstacles	Random	Walls and one trap	Test general navigation with obstacles and unsafe terminal states.
Short-Term vs. Long-Term Reward	Fixed start and fixed goal	Two reward-based routes	Test whether agents prefer immediate reward or higher accumulated long-term return.
Fixed Multi-Target	Fixed start and two fixed targets	Multiple terminal targets	Test which valid target the agent naturally learns to select.

Table 4.5: Designed environment configurations

The first configuration is the easy environment. It is mainly used to validate the basic learning pipeline. The environment contains a fixed start position and a fixed goal position. It includes walls, but no trap. The reward system uses a small negative step reward, so the agent is encouraged to reach the goal efficiently without making the task too difficult at the beginning.

The second configuration is the standard environment. It also uses fixed start and goal positions, but adds a trap as a negative terminal state. This makes the task more difficult because the agent must not only reach the goal, but also avoid unsafe behavior. The standard environment is therefore used as the main fixed-layout comparison task.

After the fixed layouts were implemented, randomized configurations were added. The purpose of these configurations is to test whether the agents learn a more general navigation behavior instead of only memorizing one fixed route. In the open random configuration, the start and goal positions are sampled randomly at the beginning of each episode. The grid does not contain walls or traps. This configuration tests whether the agent can use the relative position between agent and goal to navigate successfully.

The random obstacles configuration extends the open random environment by adding fixed walls and one trap. The start and goal positions are still sampled randomly. This

makes the task more difficult because the agent must generalize to different start-goal combinations while also avoiding obstacles and unsafe terminal states. To avoid impossible episodes, only reachable start-goal pairs are accepted.

The short-term versus long-term reward configuration was added to test delayed reward behavior more directly. In this environment, the agent has two possible routes to the same goal. The shorter route is designed to appear attractive at first because the initial forward movements along this route receive positive reward adjustments. However, later movements on the same route include negative reward adjustments. As a result, the direct route can still lead to a successful episode, but it is suboptimal with respect to the total accumulated return.

The longer route requires more steps, but it provides smaller positive reward adjustments over several forward movements. These distributed rewards make the longer route produce a higher total return despite its greater length. Therefore, the environment can be used to test whether an agent selects the path with the best long-term return or converges to the shorter path that appears beneficial at the beginning.

To avoid reward farming, the additional rewards and penalties are not assigned only to grid cells. Instead, they are assigned to directed transitions between specific cells. This means that a reward adjustment is applied only when the agent moves forward along a predefined route segment. If the agent moves backward along such a segment, an additional backtracking penalty is applied. This prevents the agent from repeatedly collecting rewards by moving back and forth between the same cells and ensures that the reward structure reflects complete route behavior rather than isolated cell visits.

The fixed multi-target configuration was added as the another environment variant. Instead of one goal, the environment contains two fixed target positions. Reaching either target terminates the episode successfully and gives the same goal reward. The agent is not told which target to choose. This configuration is used to analyze which target each method naturally learns to prefer when multiple valid terminal states are available.

4.4 Comparison Criteria

The methods are compared using common criteria. The main metrics are the success rate, the trap rate, the timeout rate, the average reward, and the average steps. In

Metric	Meaning
Success rate	Percentage of episodes in which the agent reaches the goal
Trap rate	Percentage of episodes in which the agent reaches a trap
Timeout rate	Percentage of episodes that reach the step limit
Average reward	Mean accumulated reward per episode
Average steps	Mean number of steps per episode
Training loss	Optimization loss recorded during training
Recent success rate	Moving success rate over recent training episodes
Selected route	Route learned in the short-term versus long-term reward environment
Target selection rate	Percentage of successful episodes ending at each available target
Selected target	Target reached by the agent in a successful multi-target episode

Table 4.6: Evaluation metrics used

addition, the training behavior is analyzed through learning curves and recent success rates. Table 4.6 illustrates the evaluation metrics used.

For the two additional environment variants, further interpretation criteria are required. In the short-term versus long-term reward environment, the average reward and number of steps are not sufficient alone. The selected route must also be interpreted. A successful episode with a higher average reward and longer path can indicate that the agent learned the long-term reward route, while a successful episode with fewer steps but lower reward can indicate convergence to the shorter but suboptimal route.

For the fixed multi-target environment, the selected target is also recorded during evaluation. Since both targets are valid terminal states, success rate alone does not explain the learned behavior. The target selection frequency shows whether the learned policy consistently prefers Target 1 or Target 2. This makes it possible to compare not only whether the methods solve the environment, but also how they solve it.

These metrics make it possible to compare agents not only by final success, but also by failure type, efficiency, and training stability. This is important because reinforcement learning agents may fail in different ways. One agent may reach traps quickly. Another may avoid traps but repeatedly time out. Another may reach the goal but take too many steps. Looking only at the final reward would hide these differences.

5 Environment Development

This chapter describes the development and implementation of the custom GridWorld reinforcement learning framework. The previous chapter presented the concept and design of the system, including the general environment structure, selected method families, reward concept, and comparison criteria. This chapter therefore focuses on the practical realization of these concepts in software.

This chapter explains the GridWorld environment implementation, including the environment class, step result structure, configuration methods, and rendering.

5.1 GridWorld Environment

The GridWorld environment is the central component of the framework. It represents the task in which an agent moves through a discrete grid and tries to reach a goal position. The environment stores the current state of the agent, the goal position, wall positions, trap positions, reward values, and the maximum number of allowed steps.

The environment was implemented as a custom Python class. Its interface is oriented toward the Gymnasium environment API[1] introduced as a requirement in Section 3.5.1. The environment class is derived from `gymnasium.Env`. This ensures that the self-developed environment follows the standard Gymnasium interface for reinforcement learning environments. The class defines an `action_space` and an `observation_space`. The main interaction methods are `reset()`, `step(action)`, `render()`, and `close()`.

[1] https://gymnasium.farama.org/

5.1.1 Approach

The environment class was designed to keep the interaction loop simple and consistent. At the beginning of an episode, `reset()` initializes the environment and returns the first observation. During the episode, an agent selects an action and passes it to `step(action)`. The environment then applies the action, updates the agent position, calculates the reward, checks whether the episode has ended, and returns a structured result.

This implementation approach separates the responsibility of the environment from the responsibility of the agent. The environment is responsible for state transitions, reward assignment, and episode status. The agent is responsible for action selection and learning.

Table 5.1 summarizes the main methods provided by the custom GridWorld environment and explains their role in the reinforcement learning interaction loop.

5.1.2 Environment Workflow

The environment workflow follows the episode structure used in reinforcement learning experiments as shown in Figure 5.1 . An episode starts with a call to `reset()`. The returned observation is passed to the agent. The agent then selects an action. The action is executed by the environment, and the resulting step output is returned. This loop continues until the episode terminates or is truncated.

The step logic handles several possible events. A normal valid movement changes the agent position and applies the step penalty. An invalid movement keeps the agent in the same position and applies the invalid movement penalty. Reaching the goal terminates the episode successfully. Reaching a trap terminates the episode unsuccessfully. Reaching the maximum number of steps truncates the episode.

5.1.3 Implementation

The return value of `step(action)` is represented by a Python tuple. This contains the observation after the action, the reward, the termination flag, the truncation flag, and an information dictionary. The information dictionary contains additional event data, such as the action name and the current event type.

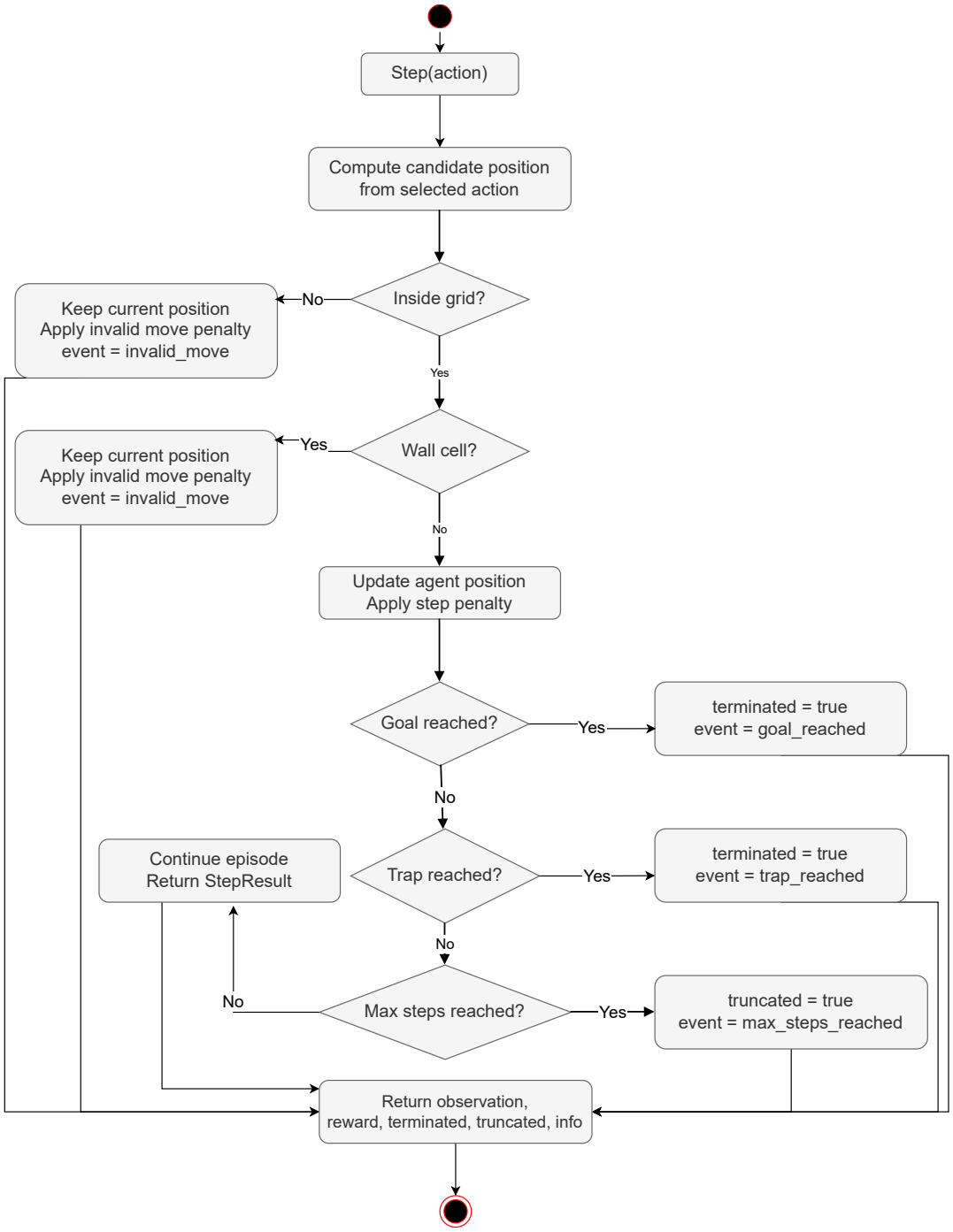


Figure 5.1: GridWorld step workflow activity diagram.

Method or attribute	Implementation role
`gymnasium.Env`	Base class from which the custom GridWorld environment is derived. This keeps the environment interface consistent with the Gymnasium standard while preserving the self-developed GridWorld logic.
`action_space`	Defines the available actions using `gymnasium.spaces.Discrete(4)`. The four discrete actions represent movement up, down, left, and right.
`observation_space`	Defines the shape and value range of the observations using `gymnasium.spaces.Box`. Depending on the configuration, the observation contains the agent position, the goal position, or multiple target positions.
`reset(seed, options)`	Initializes a new episode and returns the initial observation together with an information dictionary.
`step(action)`	Processes one action and returns the next observation, reward, termination flag, truncation flag, and information dictionary.
`render()`	Displays the current grid state for visual inspection and debugging.
`close()`	Closes or cleans up rendering resources if graphical rendering is used.

Table 5.1: Main methods of the GridWorld environment

In the implementation, these values are stored in a small Python data structure before they are used by the training and evaluation scripts. This keeps the internal code readable while preserving the same logical information as the common Gymnasium step return. The information dictionary contains additional event data, such as the action name and the event type. This allows the evaluation scripts to identify whether an episode ended with `goal_reached`, `trap_reached`, or `max_steps_reached`.

This structure is useful because the evaluation scripts can directly identify how an episode ended. Instead of only checking the final reward, the scripts can read the event information and calculate success rate, trap rate, timeout rate, average reward, and average number of steps.

The environment uses a coordinate-based observation:

$$s = (\text{agent_row}, \text{agent_col}, \text{goal_row}, \text{goal_col}) \tag{5.1}$$

The complete grid layout is not included in the observation vector. This keeps the input small and allows all neural network agents to use feedforward networks. In the randomized environments, the goal position changes between episodes, so the agents must learn to use the current agent-goal relationship instead of relying only on a fixed path.

Figure 5.2 shows a UML class diagram of the implemented environment. The diagram includes the GridWorldEnv class, the configuration methods, and the renderer. It illustrates how the environment stores the grid state, processes actions, calculates rewards, and provides the step return information used by the agents and evaluation scripts. The diagram also shows the main attributes and methods that support the Gymnasium-oriented interaction structure.

5.2 Environment Configurations and Randomization

The environment configurations are implemented as separate functions. These functions create GridWorld instances with predefined parameters. Each configuration defines the grid size, start and goal positions, walls, traps, reward values, maximum step count, and whether random start-goal sampling is enabled.

This approach keeps the core environment class independent from specific layouts. A new environment variant can be added by writing a new configuration method instead of changing the internal environment logic.

5.2.1 Configuration Approach

The environment configurations are implemented as separate configuration methods. Each function creates and returns a configured instance of the GridWorld environment. This design keeps the core environment logic independent from the individual experiment layouts. The main environment class defines the general mechanics, such as movement, wall collision, trap handling, reward calculation, termination, truncation, and rendering.

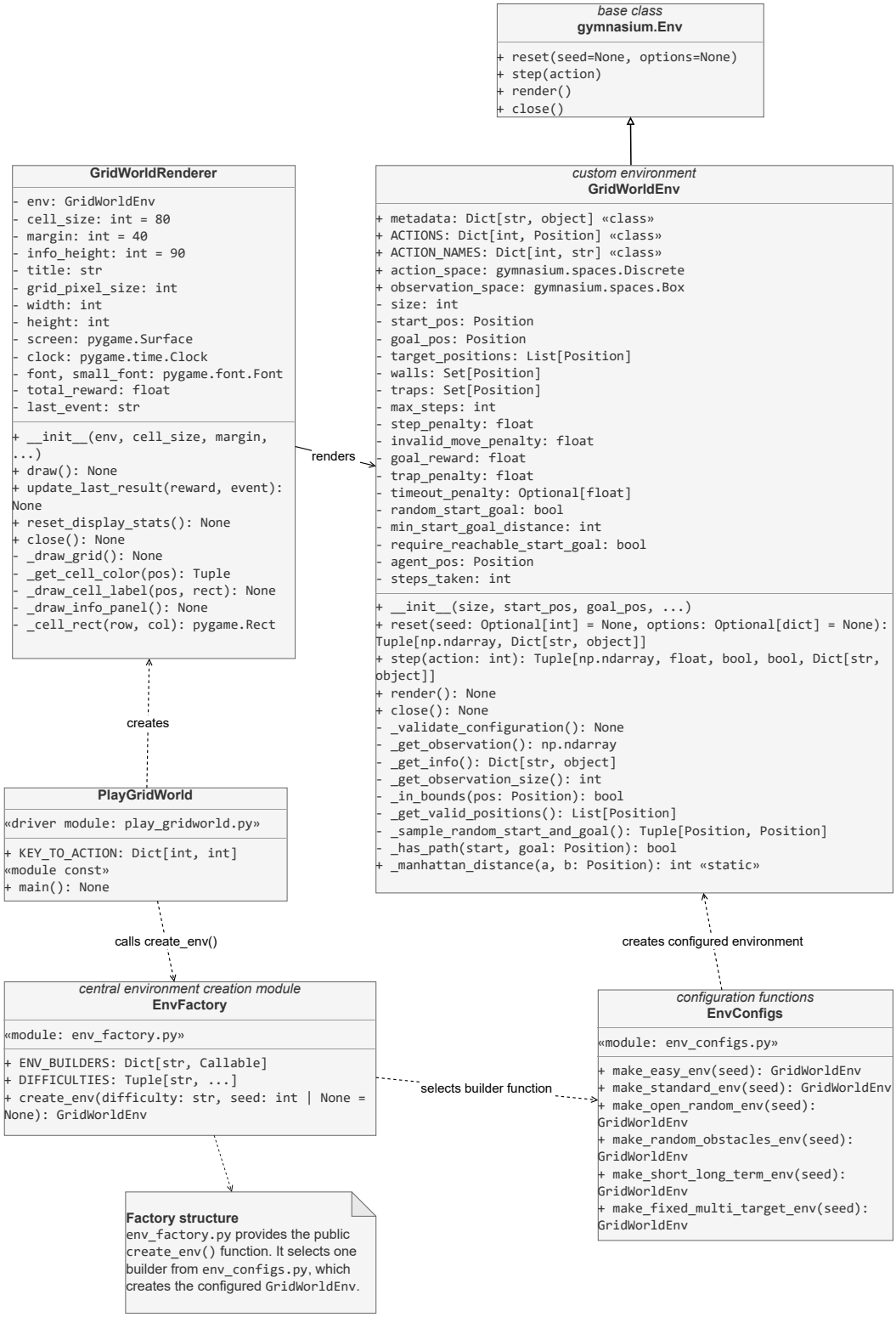


Figure 5.2: Class diagram of the GridWorld environment implementation.

The configuration methods define the concrete layout and reward settings used for a specific experiment.

This separation makes the implementation easier to extend. New experiments can be added by creating a new configuration method without rewriting the environment class itself. For example, the easy and standard configurations use fixed start and goal positions, while the open random and random obstacles configurations activate randomized start-goal sampling. The short-term versus long-term reward configuration adds route-dependent reward transitions, and the fixed multi-target configuration adds multiple terminal targets.

The configuration approach therefore supports a staged experimental design. Simple fixed environments are used first to validate the learning pipeline. Randomized environments are then used to test generalization across changing start and goal positions. The short-term versus long-term reward environment is used to test route preference and delayed reward behavior. Finally, the fixed multi-target environment is used to test target selection behavior when more than one valid terminal state exists.

All configurations use the same action space and the same step interface. This is important because the learning agents can be evaluated under different task conditions without changing the general training and evaluation procedure. Only the environment factory function changes depending on the selected difficulty argument.

Table 5.2 summarizes the configuration methods used to create the different GridWorld configurations.

5.2.2 Randomization Workflow

The randomized configurations extend the behavior of `reset()`. Instead of placing the agent and goal at fixed positions, the environment samples a new start position and a new goal position at the beginning of each episode.

The sampling process uses several checks. First, all valid cells are collected. Walls and traps are excluded. Then, a start and goal position are sampled. These positions must not be identical. A minimum start-goal distance is also required so that the task does not become too trivial.

Configuration method	Implemented variant
`make_easy_env()`	Creates the fixed Easy GridWorld used for validating the training pipeline.
`make_standard_env()`	Creates the fixed Standard GridWorld with walls and a trap.
`make_open_random_env()`	Creates a randomized environment with random start and goal positions and no obstacles.
`make_random_obstacles_env()`	Creates a randomized environment with random start and goal positions, fixed walls, and one trap.
`make_short_long_term_env()`	Creates an environment with two possible routes to the same goal, where the longer route has the higher accumulated return.
`make_fixed_multi_target_env()`	Creates an environment with two fixed terminal targets and records which target the agent selects.

Table 5.2: Environment configuration methods

For the random obstacles environment, a reachability check is added. This check ensures that there is at least one valid path between the sampled start and goal positions. Without this check, the environment could generate impossible episodes where the goal is blocked by walls or traps. Such episodes would make the evaluation unfair because failure would not necessarily indicate poor learning behavior.

Figure 5.3 shows the activity diagram for randomized start-goal sampling, where new start and goal positions are selected for randomized GridWorld configurations before an episode begins.

5.3 Rendering and Manual Environment Testing

Rendering was implemented as a support component for development and debugging. It is not used by the learning agents as input. Instead, it allows the developer to visually inspect the current environment state.

The rendered GridWorld shows the agent, goal, walls, traps, and free cells. This was useful during the implementation of the randomized environments because the start and goal positions change between episodes. Rendering makes it possible to check whether the sampled positions are valid and whether the obstacle layout is displayed correctly.

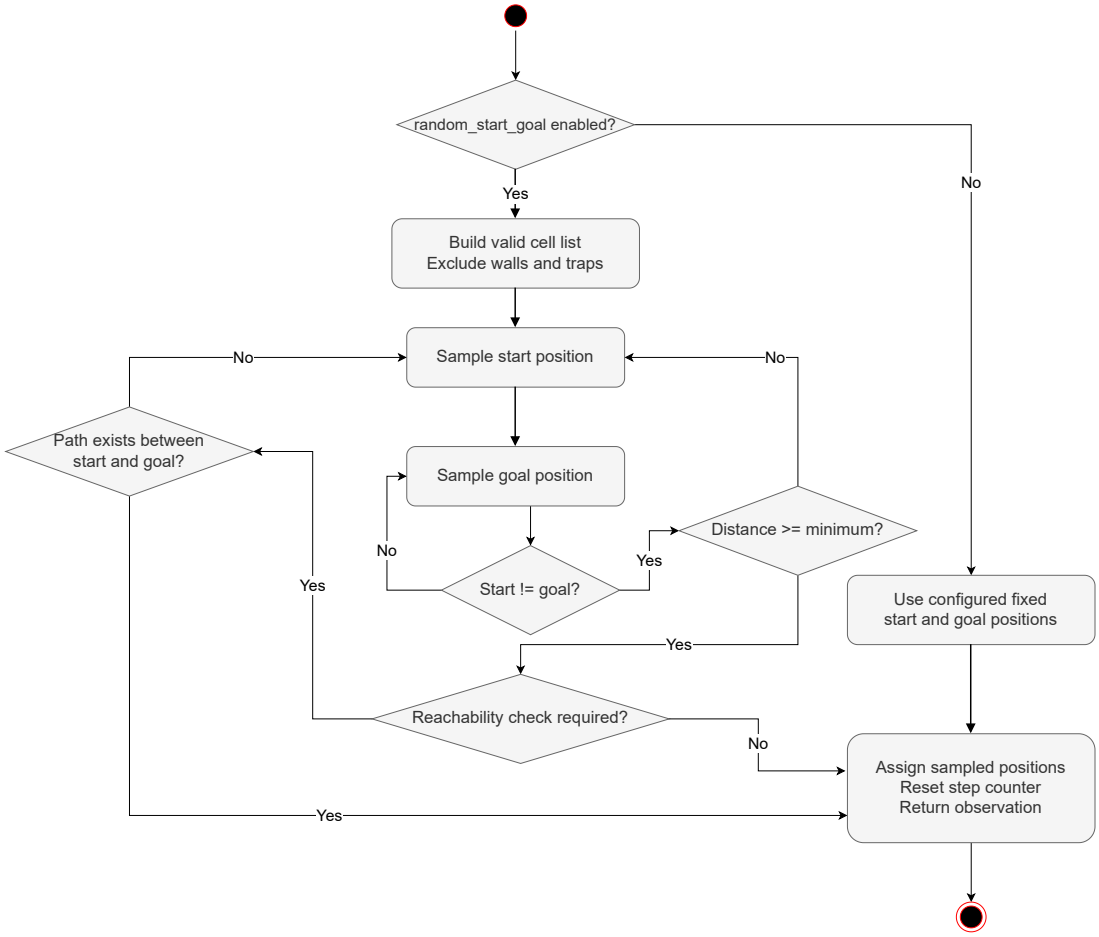


Figure 5.3: Activity diagram of randomized start-goal sampling.

5.3.1 Rendering Approach

The rendering component is separated from the learning logic, but it is integrated into the environment through an optional render mode. By default, the environment is created with `render_mode=None`, so no graphical output is produced during training. This keeps training efficient and avoids unnecessary runtime overhead.

For demonstration purposes, the environment can also be created in human rendering mode using `render_mode="human"`. In this case, the environment updates the graphical view during `reset()` and `step()`. This makes it possible to observe how an agent moves through the GridWorld during training or evaluation. The agents still receive only the coordinate-based observation and do not process rendered images.

This approach keeps rendering optional. It can be disabled for the final experiments and enabled for debugging, documentation, or educational demonstrations of the learning process.

5.3.2 Manual Testing Workflow

Manual testing is performed through the play script. The script creates a selected environment configuration and displays it using the renderer. This allows the developer to inspect the environment independently from the training scripts.

The manual testing workflow is useful for verifying:

- Correct wall placement
- Correct trap placement
- Valid agent and goal positions
- Correct movement behavior
- Correct termination behavior

6 Experimental Framework and Learning Agent Development

This chapter describes the implementation and development of the learning agents used in this thesis. The chapter does not only describe the software modules, but also explains the main design decisions, neural network architectures, hyperparameters, and training procedures used for the random agent, DQN, policy gradient, and actor-critic agents.

For each learning agent, the general approach is first described. Afterwards, the concrete implementation is explained using the most relevant code snippets. These snippets do not show the complete source code, but the parts that are central for the algorithmic behavior, such as action selection, value updates, policy updates, replay-buffer usage, and training-loop interaction with the environment. The chapter also summarizes the initial hyperparameters and explains how they were selected or adjusted during development.

The goal is to make the implementation reproducible and understandable without requiring to inspect the complete source code.

6.1 Development Setup and Project Structure

The custom GridWorld environment was implemented manually instead of using a ready-made Gymnasium benchmark. This decision is important because the development of the environment itself is part of the thesis.

The project is organized in a modular way. Environment logic, learning agents, experiment scripts, saved models, and result files are stored separately. Table 6.1 gives an overview of the main project folders and their responsibilities, while Table 6.2 summarizes the main scripts used to run the baseline, train the reinforcement learning agents, evaluate trained models, and manually inspect the GridWorld environment. This separation improves readability and makes the system easier to extend. For example, new

Module or folder	Purpose
`src/environments`	Contains the GridWorld environment, environment configuration methods, rendering support, and manual play functionality.
`src/agents`	Contains the implemented reinforcement learning agents and neural network definitions.
`src/experiments`	Contains scripts for baseline execution, training, evaluation, plotting, and experiment control.
`src/visualization`	Contains scripts for visualizing and manual testing the game.
`models`	Stores trained neural network model files.
`results`	Stores CSV files & plots containing training and evaluation results.

Table 6.1: Main project modules

environment variants can be added through configuration methods without changing the internal structure of the learning agents.

Figure 6.1 presents the implemented project structure using a UML package diagram to provide a clear overview of the software organization. The diagram illustrates the main components of the framework and their relationships, including the separation between the environment package, agent package, experiment package, model storage, and result storage. It also shows how these packages interact during the experiment workflow, such as how agents communicate with the environment, how experiments manage training and evaluation processes, and how trained models and generated results are stored for later analysis. This diagram helps demonstrate the modular design of the implementation and provides a high-level view of the system architecture.

The agents were developed iteratively. Each method was first implemented in a basic version and tested on the simpler GridWorld configurations. After the implementation produced valid interaction with the environment, the agents were trained on more difficult configurations. During this process, hyperparameters such as learning rate, entropy coefficient, epsilon decay, number of episodes, and reward-shaping settings were adjusted when training behavior indicated unstable learning, premature convergence, or insufficient exploration.

To support reproducibility, all training and evaluation scripts are controlled through command-line arguments. These arguments define the environment configuration, number of episodes, random seed, model path, result path, and evaluation mode. The same

Script	Purpose
`random_agent.py`	Runs the non-learning Random Agent baseline.
`train_dqn.py`	Trains the DQN agent on a selected environment configuration.
`evaluate_dqn.py`	Evaluates a trained DQN model.
`train_policy_gradient.py`	Trains the Policy Gradient agent.
`evaluate_policy_gradient.py`	Evaluates a trained Policy Gradient model in greedy or stochastic mode.
`train_actor_critic.py`	Trains the Actor-Critic agent.
`evaluate_actor_critic.py`	Evaluates a trained Actor-Critic model in greedy or stochastic mode.
`play_gridworld.py`	Starts a rendered GridWorld session for visual inspection and manual testing.

Table 6.2: Main execution scripts

environment interface is used for all agents. Each trained model is saved to a file and evaluated separately using a fixed evaluation script. This separation ensures that final evaluation results are based on fixed trained models and not on agents that are still learning.

Table 6.3 summarizes the settings that are used across the learning agents where applicable. Method-specific parameters, such as the DQN replay-buffer size or the entropy coefficient of the policy-based agents, are described separately in the corresponding agent sections.

Programming Language and Libraries

The implementation was developed in Python. Python was selected because it is widely used in machine learning and reinforcement learning projects and provides suitable libraries for neural network training, numerical computation, file handling, visualization, and command-line execution.

PyTorch was used for the implementation of the neural network-based agents. PyTorch was selected because it provides direct support for neural network implementation, automatic differentiation, and gradient-based optimization. These features are required for all

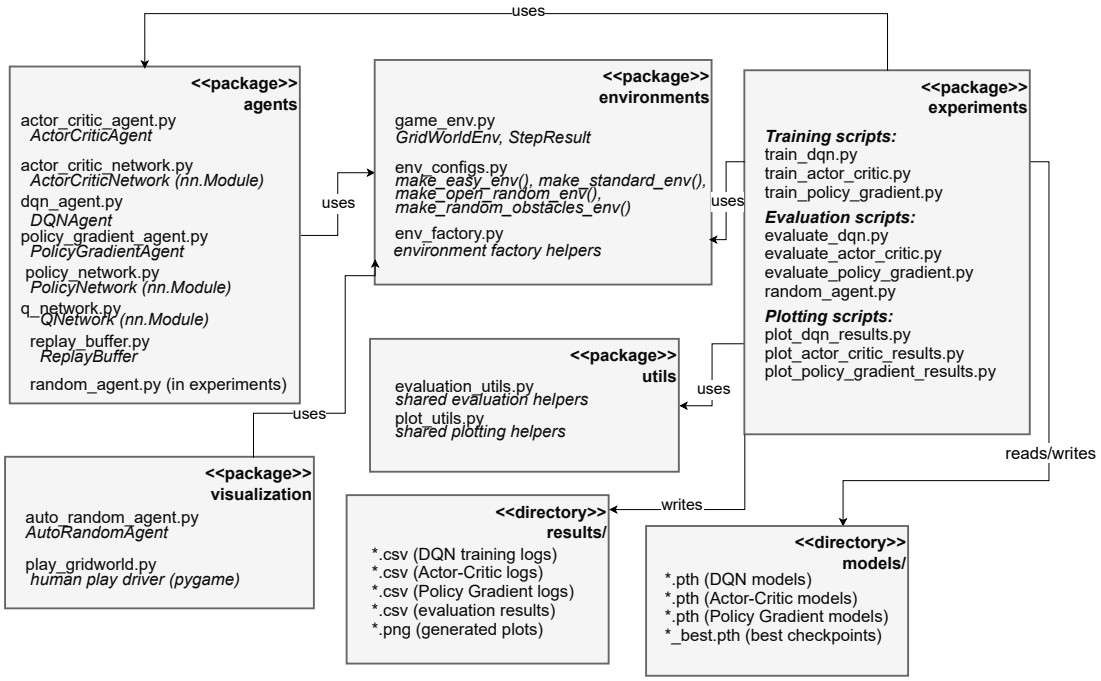


Figure 6.1: Package diagram of the implemented GridWorld framework.

three neural network-based agents. Compared with implementing backpropagation manually, PyTorch reduces implementation complexity and allows the thesis to focus on the reinforcement learning methods, environment design, and experimental comparison.

The implementation uses Python together with selected libraries for reinforcement learning, numerical processing, neural network training, experiment configuration, result storage, and visualization. Gymnasium provides the standard environment interface, NumPy supports numerical operations and observation handling, and PyTorch is used for the neural network-based agents. Training and evaluation results are stored in CSV files and later processed with Matplotlib for plotting. Command-line arguments are used to configure experiments, while the rendering module is kept separate from training and is mainly used for manual inspection and debugging.

Table 6.4 summarizes the programming language, libraries, and supporting modules used to implement the GridWorld environment, reinforcement learning agents, experiment configuration, result storage, and visualization.

Parameter	Value	Purpose
Random seed	42 where not stated otherwise	Used to make training and evaluation runs reproducible.
Discount factor γ	0.99	Gives high importance to future rewards, which is important for path-planning tasks.
Hidden layer size	64 neurons	Provides enough capacity for the small coordinate-based state representation without making the networks unnecessarily large.
Optimizer	Adam	Used for gradient-based optimization of the neural network agents.
Evaluation episodes	500	Provides a stable estimate of success rate, average reward, timeout rate, trap rate, and average episode length.
Training result format	CSV files	Allows later inspection, plotting, and comparison of training behavior.
Model storage	PyTorch model files	Stores trained models so that evaluation can be performed separately from training.

Table 6.3: Shared training and evaluation settings

6.2 Random Agent

The random agent was implemented as the first baseline agent. It does not learn and does not use a neural network. At each step, it samples one of the four possible actions uniformly at random.

6.2.1 Approach

The random agent is used to validate the environment and provide a non-learning reference. It confirms that the environment can run for many episodes without errors. It also shows how difficult each environment configuration is before training a learning agent.

Tool or library	Purpose in the implementation
Python	Main programming language used for the environment, agents, training scripts, evaluation scripts, and result processing.
Gymnasium	Used to provide a standard reinforcement learning environment interface. The custom GridWorld environment derives from `gymnasium.Env` and defines `action_space` and `observation_space`.
NumPy	Used for numerical operations, observation arrays, random sampling through the Gymnasium random generator, and data preparation before tensors are passed to neural networks.
PyTorch	Used for implementing neural networks, automatic differentiation, loss calculation, backpropagation, optimization, and saving/loading trained models.
CSV module	Used to store training and evaluation results in structured CSV files.
Argparse	Used to configure experiments through command-line arguments such as difficulty, episode count, seed, model path, and evaluation mode.
Matplotlib	Used to generate training curves, evaluation plots, and comparison figures from stored result files.
Pygame	Used for graphical visualization of the GridWorld environment during manual inspection and debugging.

Table 6.4: Programming language, libraries, and supporting modules used in the implementation

If a learning method performs only slightly better than the random agent, the method has not learned a meaningful policy. If it performs clearly better, the improvement can be attributed to learning.

6.2.2 Execution Workflow

The random agent script creates the selected environment, runs a fixed number of episodes, samples random actions, records episode outcomes, and prints a summary after all episodes are finished.

6.2.3 Implementation

For each episode, the script records total reward, number of steps, final event, success flag, trap flag, and timeout flag. These values are saved and summarized using the same evaluation metrics later used for the learning agents.

Listing 6.1 shows the core action selection of the random agent. The agent does not contain trainable parameters and does not use a neural network. Instead, it samples one action uniformly from the Gymnasium action space of the environment.

```
1 action = env.action_space.sample()
2 _, reward, terminated, truncated, info = env.step(action)
```

Listing 6.1: Core action selection of the Random Agent

The random agent therefore uses the same environment interface as the learning agents, but it does not update any parameters. This makes it suitable as a non-learning baseline.

6.3 DQN Agent

The DQN agent represents the value-based learning method in the implementation. It was implemented after the random agent baseline and uses a neural network to estimate action values for the four movement actions. The implemented DQN module consists of the Q-network, the target network, the replay buffer, the epsilon-greedy action selection mechanism, the training script, and the evaluation script.

The following subsections describe the approach, the network architecture, the selected hyperparameters, the core implementation, and the training workflow of the DQN agent. The selected code listings show only the central algorithmic parts of the implementation, not the complete source code.

6.3.1 Approach

The DQN agent receives the coordinate-based state vector from the environment. The network output contains one Q-value for each action. During training, the agent does

not always select the action with the highest predicted Q-value. Instead, it uses epsilon-greedy action selection. This means that a random action is selected with probability epsilon, while the action with the highest Q-value is selected otherwise.

The value of epsilon starts at 1.0 and is multiplied by 0.995 after each episode until it reaches the minimum value of 0.05. This means that the agent explores strongly at the beginning of training and gradually shifts toward selecting actions according to the learned Q-values. This implementation was necessary because the agent would otherwise be likely to exploit early and possibly poor value estimates too soon.

The DQN implementation also uses a replay buffer and a target network. The replay buffer stores transitions from previous environment interactions and later samples mini-batches for training. This reduces the dependency between consecutive transitions. The target network is a copy of the Q-network that is updated less frequently and is used to calculate more stable target values. The implemented update follows the Bellman target introduced in Section 2.2.2.

6.3.2 Network Architecture and Hyperparameters

The DQN agent uses a fully connected neural network because the GridWorld state is represented as a compact coordinate-based vector rather than an image. The input size depends on the selected environment configuration. The output size is always four, because the environment provides four discrete movement actions.

The concrete network structure and training settings of the DQN agent are summarized before describing the implementation details. Table 6.5 shows the architecture of the Q-network, while Table 6.6 lists the main hyperparameters used during DQN training.

The hidden layer size of 64 neurons was selected because the state representation is small and coordinate-based. A larger network was not required for the GridWorld task and would have increased the implementation complexity without a clear benefit. The replay buffer and target network were used to stabilize training. The epsilon schedule was selected to provide strong exploration at the beginning while allowing the agent to increasingly rely on the learned Q-values later in training.

Component	Configuration
Input layer	Coordinate-based state vector. The input size depends on the environment configuration.
Hidden layer 1	Fully connected layer with 64 neurons and ReLU activation.
Hidden layer 2	Fully connected layer with 64 neurons and ReLU activation.
Output layer	Four output values, one Q-value for each movement action.
Loss function	Smooth L1 loss between predicted Q-values and target Q-values.
Optimizer	Adam optimizer.

Table 6.5: DQN network architecture

6.3.3 Training Workflow

The DQN training workflow starts by creating the selected GridWorld environment through the environment factory. The state size is derived from the Gymnasium observation space, and the action size is derived from the Gymnasium action space. A DQN agent is then created with the network architecture and hyperparameters shown in Tables 6.5 and 6.6.

At the beginning of each episode, the environment is reset and the initial observation is returned. The agent then interacts with the environment until the episode terminates or is truncated. At each step, the agent selects an action using epsilon-greedy action selection, executes the action in the environment, stores the resulting transition in the replay buffer, and performs a learning update if at least 64 transitions are available.

Mini-batches of 64 transitions are sampled from the replay buffer and used to update the policy network. The target network is synchronized every 250 learning updates by copying the weights of the policy network. After each episode, epsilon is multiplied by 0.995 until it reaches the minimum value of 0.05.

During training, episode-level values such as total reward, number of steps, final event, success flag, trap flag, timeout flag, current epsilon, average loss, recent success rate, and

Hyperparameter	**Value**	**Purpose**
Learning rate	$1 \cdot 10^{-3}$	Controls the update size of the Q-network.
Discount factor γ	0.99	Weights future rewards strongly.
Replay-buffer size	10,000 transitions	Stores previous transitions for mini-batch learning.
Batch size	64	Number of transitions sampled for one update.
Target update interval	250 learning updates	Defines how often the target network is synchronized with the policy network.
Initial epsilon	1.0	Starts training with full exploration.
Minimum epsilon	0.05	Maintains a small amount of exploration.
Epsilon decay	0.995 per episode	Gradually reduces exploration during training.

Table 6.6: DQN hyperparameters

recent average reward are stored in a CSV file. The trained model is saved after training and evaluated separately using the evaluation script.

The final configuration uses a replay-buffer size of 10,000 transitions, a batch size of 64, and a target-network update interval of 250 learning updates. These values provided stable learning behavior in the GridWorld experiments without making the training unnecessarily slow. The epsilon decay was kept gradual because early experiments showed that reducing exploration too quickly can cause the agent to exploit incomplete Q-value estimates. For the standard environment, the number of training episodes was increased compared with the easy environment because the trap layout and longer learning process required more interaction before stable behavior was observed.

Figure 6.2 illustrates this process as an activity diagram, providing a clear visual representation of the sequence of actions, decision points, and possible paths taken during execution.

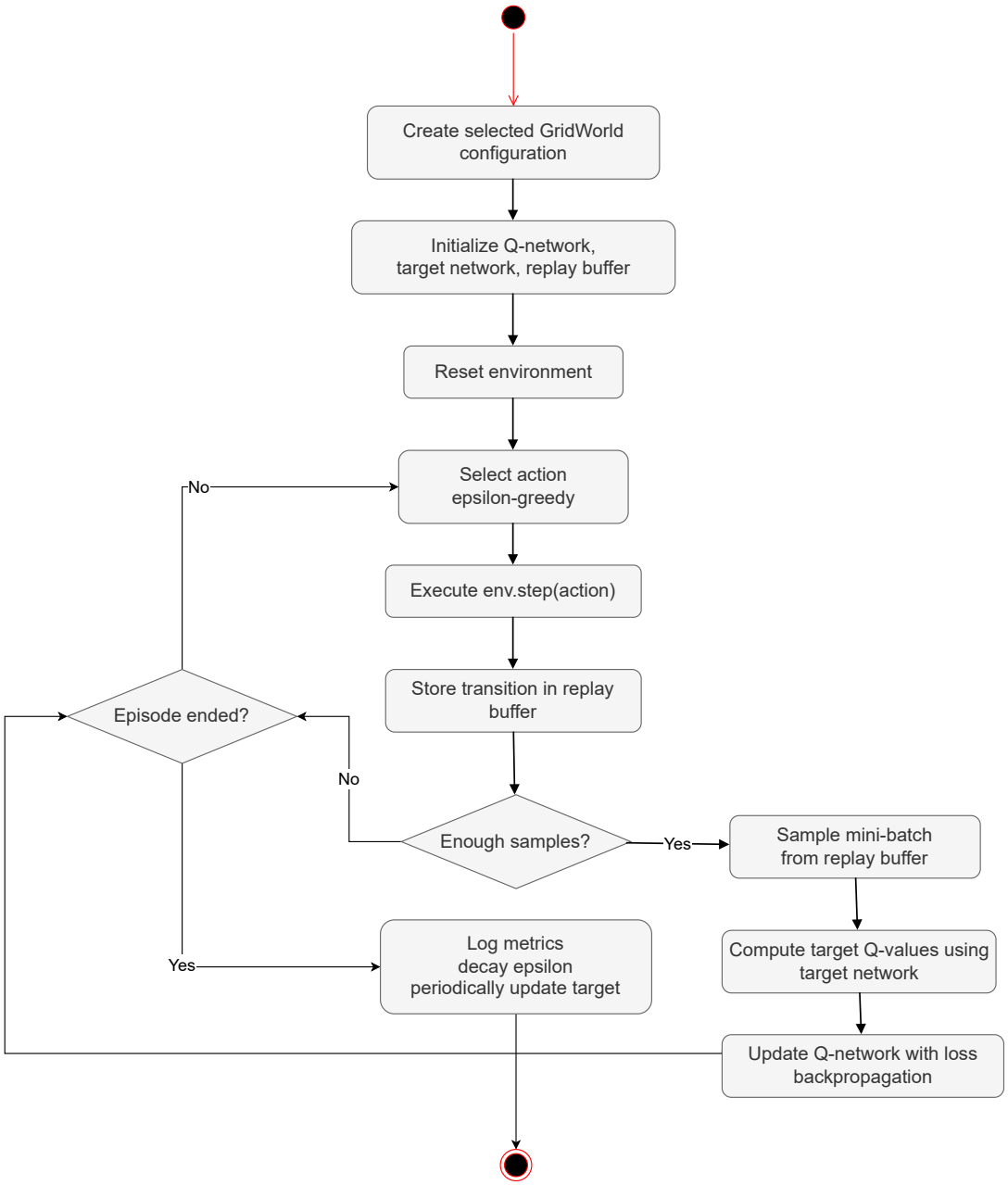


Figure 6.2: Activity diagram of the DQN training workflow.

6.3.4 Implementation

During evaluation, the DQN agent uses greedy action selection. The trained model is loaded from `models/`, and the action with the highest Q-value is selected at each step. No epsilon exploration is used during evaluation. The evaluation script records the same metrics as the random agent baseline, which makes the results directly comparable.

The following listings show the central implementation parts of the DQN agent. They are limited to the algorithmic core and do not show the complete source code.

Listing 6.2 shows the epsilon-greedy action selection. If a random value is smaller than ϵ, the agent explores by selecting a random action. Otherwise, it exploits the current Q-network by selecting the action with the highest predicted Q-value.

Listing 6.3 shows how one environment interaction is stored in the replay buffer. Each transition contains the current state, selected action, received reward, next state, and done flag. The done flag combines termination and truncation and is later used to remove the future value term for completed episodes.

Listing 6.4 shows the calculation of the predicted Q-values and the target Q-values. The policy network predicts the Q-values for the actions that were actually taken. The target network estimates the best future Q-value of the next state. The done tensor removes the future value term for terminal or truncated transitions. The Smooth L1 loss is then used to compare the predicted Q-values with the target values.

```
1  def choose_action(self, state: Tuple[int, ...], epsilon: float) ->
       int:
2
3          if self.rng.random() < epsilon:
4              return self.rng.randrange(self.action_size)
5
6          state_tensor = self._state_to_tensor(state)
7
8          with torch.no_grad():
9              q_values = self.policy_net(state_tensor)
10
11         return int(torch.argmax(q_values, dim=1).item())
```

Listing 6.2: DQN epsilon-greedy action selection

```
done = terminated or truncated

agent.store_transition(
    state=state,
    action=action,
    reward=reward,
    next_state=next_state,
    done=done,
)
```

Listing 6.3: Storing a transition in the replay buffer

```
current_q_values = self.policy_net(states_tensor).gather(
    1,
    actions_tensor,
)

with torch.no_grad():
    next_q_values = self.target_net(next_states_tensor)
    max_next_q_values = next_q_values.max(dim=1, keepdim=True)[0]

    target_q_values = rewards_tensor + self.gamma * max_next_q_values
    * (1.0 - dones_tensor)

loss = self.loss_function(current_q_values, target_q_values)
```

Listing 6.4: DQN target calculation and loss

Listing 6.5 shows how the target network is synchronized with the policy network. In the final DQN configuration, this synchronization is performed every 250 learning updates.

```
self.training_steps += 1

if self.training_steps % self.target_update_every == 0:
    self.target_net.load_state_dict(self.policy_net.state_dict())
```

Listing 6.5: DQN target-network synchronization

6.4 Policy-Gradient Agent

The policy gradient agent represents the policy-based method in the implementation. In contrast to DQN, the policy gradient agent does not estimate action values. Instead, it learns a probability distribution over the four actions.

The implemented Policy Gradient agent is based on the REINFORCE approach introduced in Section 2.4.2. The agent samples actions from a categorical probability distribution during training, stores the log probabilities and received rewards, and updates the policy after a complete episode.

6.4.1 Approach

The policy gradient agent receives the coordinate-based observation from the GridWorld environment. The observation is converted into a tensor and passed to the policy network. The network outputs one action logit for each of the four movement actions. These logits are converted into a categorical probability distribution.

During training, the agent samples an action from this distribution. This means that the agent does not always select the action with the highest probability. Instead, actions with lower probability can still be selected, which supports exploration during learning. The log probability of the selected action is stored because it is later required for the policy-gradient update.

In contrast to DQN, the policy gradient agent does not update after every single environment step. Instead, it collects the rewards of a complete episode and then computes the discounted return for each time step. The policy is updated so that actions followed by higher returns become more likely, while actions followed by lower returns become less likely.

To reduce the variance of the update, the implementation uses a moving-average return baseline. The return is compared with this baseline to calculate an advantage-like learning signal. In addition, an entropy term is used during training to encourage exploration and reduce premature convergence to a deterministic policy.

Component	Configuration
Input layer	Coordinate-based state vector. The input size depends on the environment configuration.
Hidden layer 1	Fully connected layer with 64 neurons and ReLU activation.
Hidden layer 2	Fully connected layer with 64 neurons and ReLU activation.
Output layer	Four action logits, one for each movement action.
Action distribution	Categorical distribution over the four actions.
Optimizer	Adam optimizer.

Table 6.7: Policy gradient network architecture

6.4.2 Network Architecture and Hyperparameters

The concrete network structure and training settings of the policy gradient agent are summarized before describing the implementation details. Table 6.7 shows the architecture of the policy network, while Table 6.8 lists the main hyperparameters used during policy gradient training.

The network architecture is intentionally small because the state representation is compact and coordinate-based. The entropy coefficient and moving-average baseline were added because the Monte Carlo policy gradient update can have high variance. During development, lower learning rates and reward shaping were tested for more difficult configurations when training was unstable or when the sparse reward signal was not sufficient for reliable learning.

6.4.3 Training Workflow

The policy gradient training workflow begins with the creation of the selected environment and the initialization of the policy network. A policy gradient agent is then created with the network architecture and hyperparameters shown in Tables 6.7 and 6.8. During each episode, the current state is passed to the policy network. The network outputs action probabilities, and an action is sampled from this distribution.

Hyperparameter	**Value**	**Purpose**
Discount factor γ	0.99	Weights future rewards strongly when computing discounted returns.
Hidden layer size	64 neurons	Defines the size of both hidden layers in the policy network.
Learning rate	$1 \cdot 10^{-3}$, with selected lower values for more difficult configurations	Controls the update size of the policy network.
Entropy coefficient	0.05 at the beginning of training	Encourages exploration by rewarding higher policy entropy.
Minimum entropy coefficient	0.005	Prevents the entropy term from becoming completely inactive too early.
Entropy decay	0.999 per episode	Gradually reduces exploration pressure during training.
Baseline decay	0.9	Controls the moving-average return baseline.
Gradient clipping	1.0	Limits large gradient updates and improves training stability.

Table 6.8: Policy gradient hyperparameters

The log probability of the selected action is stored together with the received reward. This process continues step by step as the agent interacts with the environment, collecting a trajectory of states, actions, and rewards until the episode terminates or is truncated. After the episode, discounted returns are computed by propagating rewards backward through time, giving higher importance to rewards received earlier in the trajectory.

The policy loss is then calculated using these discounted returns, typically by weighting the stored log probabilities with the corresponding returns. This loss reflects how well the chosen actions contributed to achieving higher rewards. The network parameters are updated through gradient ascent, increasing the likelihood of actions that led to better outcomes and decreasing the likelihood of less effective actions.

Reward shaping can optionally be enabled during training. In this case, an additional signal is added based on whether the agent moves closer to or farther away from the goal. This provides more immediate feedback compared to sparse terminal rewards and helps guide the learning process, especially in environments where reaching the goal is difficult or requires long sequences of actions.

During development, the policy gradient agent was more sensitive to the reward structure than the DQN agent. This was expected because the update is based on sampled complete episodes instead of replayed transition mini-batches. Early training runs showed higher variance in episode rewards, especially in configurations with traps, randomized positions, or delayed reward effects. For this reason, an entropy term, a moving-average baseline, gradient clipping, and optional reward shaping were used to improve stability.

The entropy coefficient starts at 0.05 and is gradually reduced to a minimum of 0.005. This allows stronger exploration at the beginning of training and more stable policy behavior later. Lower learning rates were used in selected more difficult configurations when the policy updates were too unstable with the default learning rate.

Figure 6.3 shows the policy gradient workflow as an activity diagram, illustrating the sequence of steps involved in the learning process, including state observation, action selection based on the current policy, reward collection, and policy update.

6.4.4 Implementation

The following listings show the central implementation parts of the policy gradient agent. They focus on stochastic action selection, return-based policy updating, and optional reward shaping.

Listing 6.6 shows the stochastic action selection of the policy gradient agent. The policy network outputs action logits, which are converted into a categorical distribution. The selected action, its log probability, and the entropy of the distribution are stored until the episode ends.

Listing 6.7 shows the main loss calculation. First, the discounted returns are computed for the episode. The moving-average baseline is then updated and subtracted from the returns. The resulting learning signal is used to increase the probability of actions that performed better than the baseline and decrease the probability of actions that performed worse. The entropy term is subtracted from the loss to encourage exploration.

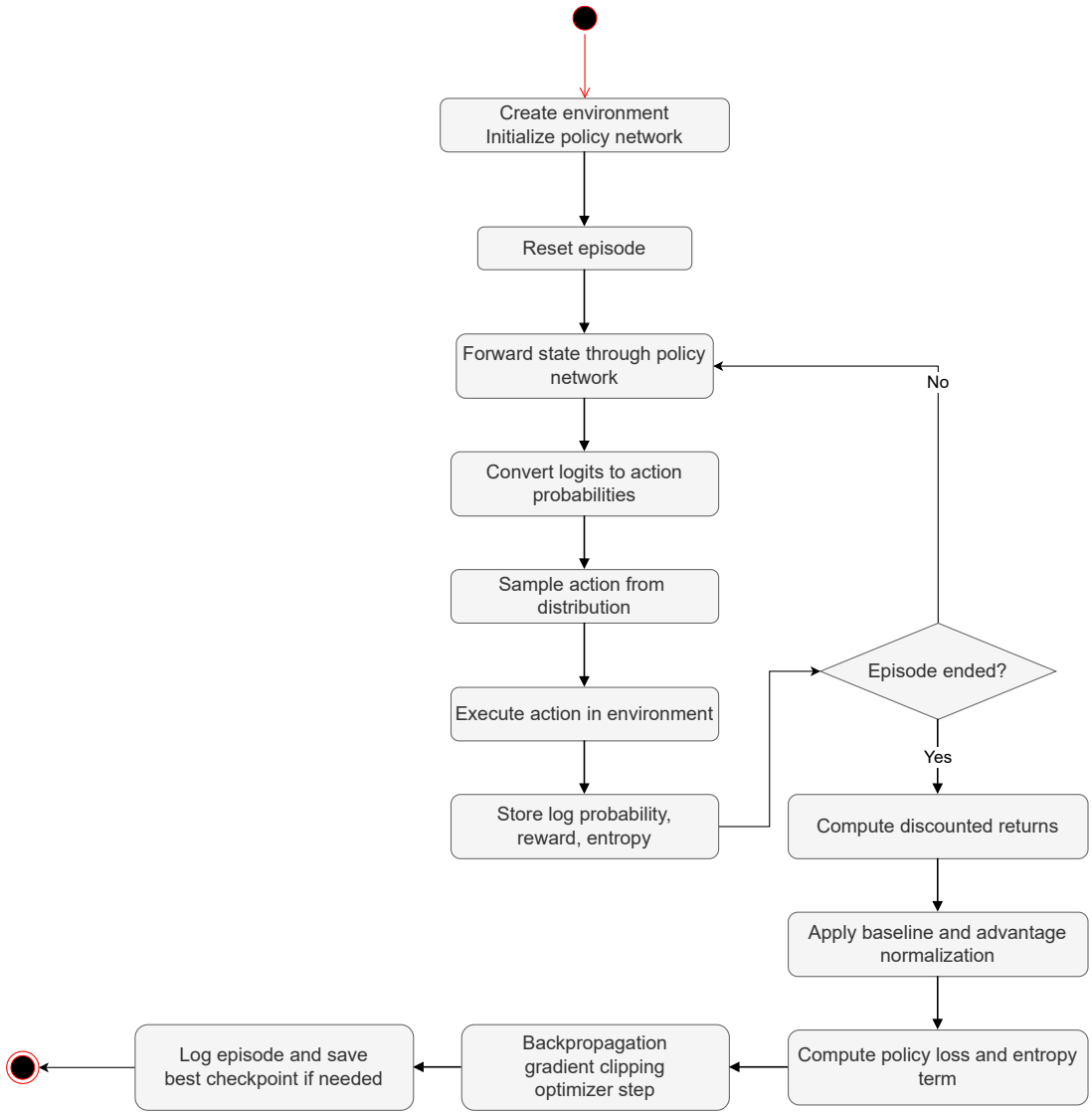


Figure 6.3: Activity diagram of the policy gradient training workflow.

Listing 6.8 shows the optional reward shaping used during selected policy gradient training runs. The original environment reward is still recorded for evaluation, while the shaped reward is used only as the learning signal. This separation ensures that final evaluation results remain comparable across methods.

The evaluation script supports two different action-selection modes. In greedy mode, the action with the highest probability is selected. In stochastic mode, the action is

```
def choose_action(self, state: Tuple[int, ...]) -> int:

        state_tensor = self._state_to_tensor(state)

        logits = self.policy_net(state_tensor)
        distribution = Categorical(logits=logits)

        action = distribution.sample()
        log_probability = distribution.log_prob(action)
        entropy = distribution.entropy()

        self.saved_log_probs.append(log_probability)
        self.saved_entropies.append(entropy)

        return int(action.item())
```

Listing 6.6: Policy gradient stochastic action selection

sampled from the learned probability distribution. Both modes are included because policy gradient learns probabilities, not Q-values.

The distinction between greedy and stochastic evaluation is important for interpreting the results. Greedy evaluation shows whether the policy can be used as a deterministic decision rule. Stochastic evaluation shows whether useful behavior exists in the learned probability distribution.

6.5 Actor-Critic Agent

The actor-critic agent represents the combined value-based and policy-based method in the implementation. It extends the idea of direct policy learning by adding a critic that estimates the value of the current state. The actor selects actions, while the critic evaluates how good the visited states are.

The implemented actor-critic agent uses a shared neural network structure with two output heads. The actor head outputs action logits for the policy distribution, while the critic head outputs a scalar state-value estimate. The following subsections describe the approach, network architecture, hyperparameters, core implementation, and training workflow of the actor-critic agent.

```
returns = self._compute_discounted_returns(normalize=False)
episode_return_mean = float(returns.mean().item())

if self.return_baseline is None:
    self.return_baseline = episode_return_mean
else:
    self.return_baseline = (
        self.baseline_decay * self.return_baseline
        + (1.0 - self.baseline_decay) * episode_return_mean
    )

advantages = returns - self.return_baseline

policy_losses = []

for log_probability, entropy, advantage in zip(
    self.saved_log_probs,
    self.saved_entropies,
    advantages
):
    policy_losses.append(
        -log_probability * advantage
        - self.entropy_coefficient * entropy
    )

loss = torch.cat(policy_losses).sum()
```

Listing 6.7: Policy gradient baseline and loss calculation

6.5.1 Approach

The actor-critic agent receives the coordinate-based observation from the GridWorld environment and converts it into a tensor. This tensor is passed to the actor-critic network. The network produces two outputs: action logits from the actor and a state-value estimate from the critic.

The actor output is converted into a categorical probability distribution over the four movement actions. During training, the agent samples an action from this distribution. The log probability and entropy of the selected action are stored because they are later used in the policy update.

The critic estimates the value $V(s)$ of the current state. After an episode has finished, the discounted returns are computed and compared with the critic's value estimates.

```
if use_reward_shaping:
    learning_reward = apply_potential_based_shaping(
        reward=original_reward,
        state=state,
        next_state=next_state,
        gamma=agent.gamma,
        shaping_scale=shaping_scale
    )
else:
    learning_reward = original_reward

agent.store_reward(learning_reward)
```

Listing 6.8: Optional potential-based reward shaping

The difference between the return and the value estimate is used as the advantage. This advantage tells the actor whether the selected actions performed better or worse than expected.

Compared with the basic policy gradient method, the actor-critic method uses the critic to provide a learned baseline. This can reduce the variance of the policy update because the actor is not updated only from raw episode returns, but from returns compared with the critic's state-value estimates.

6.5.2 Network Architecture and Hyperparameters

The concrete network structure and training settings of the actor-critic agent are summarized before describing the implementation details. Table 6.9 shows the architecture of the actor-critic network, while Table 6.10 lists the main hyperparameters used during training.

The actor-critic network uses two shared hidden layers because the input state is compact and coordinate-based. The shared representation is used by both the actor and the critic. The actor head learns the policy, while the critic head learns the state-value estimate. The critic coefficient controls how strongly the value loss contributes to the total loss. The entropy coefficient is gradually reduced during training so that the agent explores more at the beginning and becomes more stable later.

Component	Configuration
Input layer	Coordinate-based state vector. The input size depends on the environment configuration.
Shared hidden layer 1	Fully connected layer with 64 neurons and ReLU activation.
Shared hidden layer 2	Fully connected layer with 64 neurons and ReLU activation.
Actor head	Fully connected output layer with four action logits, one for each movement action.
Critic head	Fully connected output layer with one scalar state-value estimate $V(s)$.
Action distribution	Categorical distribution over the four movement actions.
Optimizer	Adam optimizer.

Table 6.9: Actor-critic network architecture

6.5.3 Training Workflow

The actor-critic training workflow starts by creating the selected GridWorld environment through the environment factory. The state size is derived from the Gymnasium observation space, and the action size is derived from the Gymnasium action space. An actor-critic agent is then created with the network architecture and hyperparameters shown in Tables 6.9 and 6.10.

The actor-critic training workflow is episode-based. During an episode, the agent stores log probabilities, received rewards, entropy values, and critic value estimates. After the episode ends, discounted returns are computed.

The advantage is calculated by comparing the discounted returns with the critic value estimates. The actor is updated using this advantage signal, while the critic is updated to predict the returns more accurately. The final loss combines actor loss, critic loss, and an entropy term.

This episode-based update was used because it produced more stable behavior in the implemented GridWorld experiments than a simple one-step update.

Hyperparameter	**Value**	**Purpose**
Discount factor γ	0.99	Weights future rewards strongly when computing discounted returns.
Hidden layer size	64 neurons	Defines the size of the shared hidden layers.
Learning rate	$1 \cdot 10^{-3}$, with $3 \cdot 10^{-4}$ for the standard configuration	Controls the update size of the Actor-Critic network.
Entropy coefficient	0.05 at the beginning of training	Encourages exploration during policy learning.
Minimum entropy coefficient	0.005	Keeps a small entropy contribution during later training.
Entropy decay	0.9995 per episode	Gradually reduces exploration pressure during training.
Critic coefficient	0.5	Weights the critic loss in the combined loss function to estimate returns more accurately.
Gradient clipping	1.0	Limits large gradient updates and improves training stability.

Table 6.10: Actor-critic hyperparameters

During development, the actor-critic agent was used as an intermediate approach between DQN and the basic policy gradient method. Compared with policy gradient, the critic provided a learned value baseline and helped stabilize the policy update. However, the method still depended on sampled actions during training and was therefore more sensitive to exploration settings than DQN.

The entropy coefficient starts at 0.05 and is gradually reduced to a minimum value of 0.005. This allows the agent to explore different actions at the beginning of training while reducing unnecessary randomness later. A critic coefficient of 0.5 was used to balance policy learning and value estimation. Gradient clipping was applied to reduce the effect of large updates during unstable episodes.

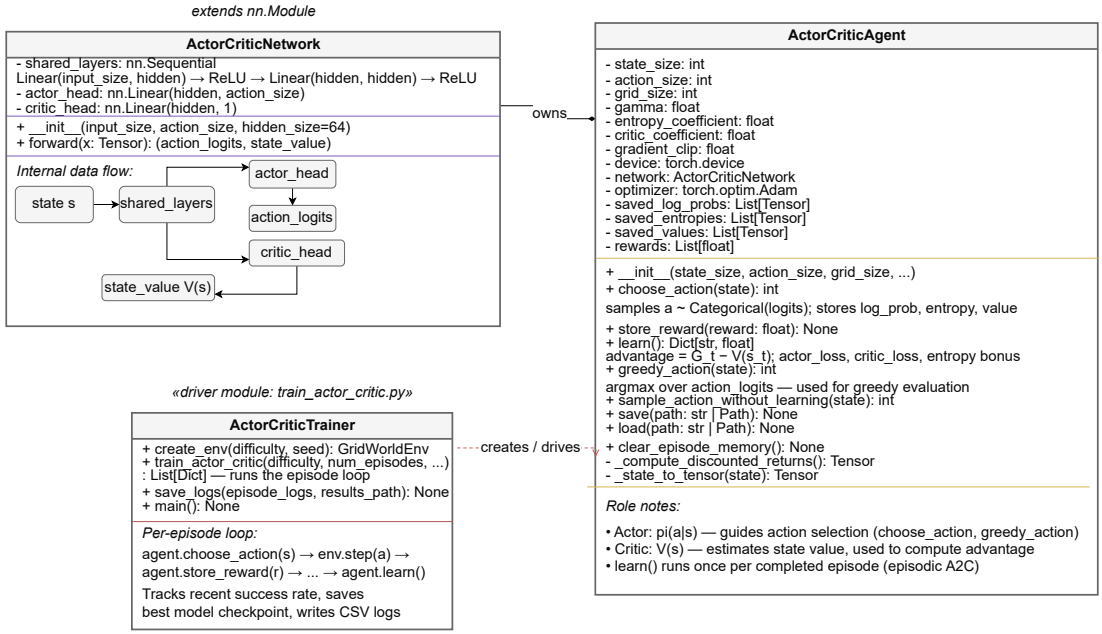


Figure 6.4: Architecture diagram of the implemented actor-critic network.

Figure 6.4 shows the actor-critic network as a UML-style architecture diagram, clearly illustrating the main components of the model, including the shared input layer, the actor network responsible for selecting actions, and the critic network responsible for evaluating state values. The diagram also describes how information flows between these components, highlighting the interaction between the actor and critic during the learning process.

6.5.4 Implementation

The following listings show the central implementation parts of the actor-critic agent. They focus on action selection, storage of actor and critic values, and the combined loss calculation.

Listing 6.9 shows how the actor-critic agent selects an action during training. The actor output is converted into a categorical distribution and an action is sampled from this distribution. At the same time, the critic output is stored as the value estimate of the current state. The log probability, entropy, and value estimate are stored until the episode-level update is performed.

Listing 6.10 shows the combined actor-critic loss calculation. The actor loss updates the policy using the advantage values. The critic loss trains the state-value estimate toward the discounted returns. The entropy term is subtracted from the loss because higher entropy encourages exploration during training.

```
def choose_action(self, state: Tuple[int, ...]) -> int:

        state_tensor = self._state_to_tensor(state)

        logits, state_value = self.network(state_tensor)
        distribution = Categorical(logits=logits)

        action = distribution.sample()
        log_probability = distribution.log_prob(action)
        entropy = distribution.entropy()

        self.saved_log_probs.append(log_probability)
        self.saved_entropies.append(entropy)
        self.saved_values.append(state_value.squeeze(-1))

        return int(action.item())
```

Listing 6.9: Actor-critic action selection

```
returns = self._compute_discounted_returns()

log_probs = torch.cat(self.saved_log_probs)
entropies = torch.cat(self.saved_entropies)
values = torch.cat(self.saved_values)

advantages = returns - values.detach()

actor_loss = -(log_probs * advantages).mean()
critic_loss = F.smooth_l1_loss(values, returns)
entropy_bonus = entropies.mean()

total_loss = (
    actor_loss
    + self.critic_coefficient * critic_loss
    - self.entropy_coefficient * entropy_bonus
)
```

Listing 6.10: Actor-critic advantage and loss calculation

```
self.optimizer.zero_grad()
total_loss.backward()

torch.nn.utils.clip_grad_norm_(
    self.network.parameters(),
    self.gradient_clip,
)

self.optimizer.step()
```

Listing 6.11: Actor-critic optimization step

Listing 6.11 shows the optimization step. Before the update, old gradients are cleared. The combined loss is then backpropagated through the shared network, gradients are clipped to avoid excessively large updates, and the Adam optimizer updates the network parameters.

Like the policy gradient agent, the actor-critic agent supports greedy and stochastic evaluation. In greedy mode, the action with the highest actor probability is selected. In stochastic mode, actions are sampled from the actor distribution.

7 Evaluation

This chapter evaluates the developed reinforcement learning framework and the implemented learning methods. The evaluation is based on the trained agents that were produced using the implementation, architectures, hyperparameters, and training workflows described in Chapter 6.

Before the final evaluation results are presented, the chapter first summarizes the agent development observations. This includes the main training behavior observed during development, the reasons for selected parameter choices, and the modifications that were necessary to obtain stable trained agents. Afterwards, the final trained models are evaluated on all designed GridWorld configurations.

The evaluated environments are the easy environment, the standard environment, the open random environment, the random obstacles environment, the short-term versus long-term reward environment, and the fixed multi-target environment.

The purpose of the evaluation is not only to determine whether an agent reaches the goal. Instead, the results are analyzed using several metrics which follow the comparison criteria defined in Chapter 4: success rate, trap rate, timeout rate, average reward, average number of steps, route selection, target selection, and the training behavior shown by the learning curves. This is necessary because reinforcement learning agents can solve the same task in different ways. A high success rate alone does not always show whether the learned strategy is efficient, stable, or optimal with respect to the reward structure.

The evaluated learning methods are deep Q-network, policy gradient, and actor-critic. DQN is evaluated using greedy action selection, where the action with the highest predicted action value is selected. Policy gradient and actor-critic are evaluated in both greedy and stochastic modes. Greedy evaluation shows the deterministic behavior of the learned policy, while stochastic evaluation shows the behavior when actions are sampled from the learned probability distribution.

7.1 Evaluation Methodology

Each trained agent was evaluated over 500 episodes per environment. At the beginning of each episode, the environment was reset. The agent then selected actions until the episode ended by termination or truncation.

For each episode, the evaluation scripts recorded the total reward, the number of steps, the final event, the success flag, the trap flag, and the timeout flag. In the fixed multi-target environment, the selected target was also recorded. This makes it possible to evaluate not only whether the agent reached a terminal state, but also which target it preferred when more than one valid target was available.

The main comparison uses greedy evaluation because it shows the final deterministic policy learned by each method. DQN naturally uses greedy evaluation by selecting the action with the highest estimated Q-value. For policy gradient and actor-critic, greedy evaluation selects the action with the highest policy output. Since policy gradient and actor-critic are policy-based methods, stochastic evaluation was also performed. In stochastic evaluation, actions are sampled from the learned policy distribution. This additional evaluation is useful because policy-based methods may still contain useful behavior in their probability distribution even when the greedy policy is less stable.

7.2 Agent Development Observations

Before the final evaluation was carried out, the implemented agents were developed and tested iteratively. This was necessary because the agents did not only differ in their theoretical learning approach, but also in their practical training behavior. The observations during development influenced the final training settings, such as the number of training episodes, exploration parameters, entropy regularization, reward shaping, and learning rate choices.

This section summarizes the most important development observations for the DQN, policy gradient, and actor-critic agents. The purpose is not to repeat the complete final evaluation, but to explain why certain implementation and training decisions were used for the final experiments.

7.2.1 DQN Development

The DQN agent was first tested on the easier fixed GridWorld configuration. This configuration was useful for verifying that the replay buffer, epsilon-greedy action selection, target-network synchronization, model saving, and evaluation procedure worked correctly. After the implementation was verified on the easier task, the same agent structure was applied to the standard environment and later to the more advanced configurations.

During development, the DQN agent showed stable learning behavior when replay-buffer learning and the target network were used together. The replay buffer reduced the dependency between consecutive transitions, while the target network made the target values more stable. Without sufficient exploration, however, the agent could exploit incomplete early Q-value estimates. Therefore, epsilon was started at 1.0 and reduced gradually during training until it reached the minimum value of 0.05.

The training duration was also adapted to the difficulty of the environment. The easy configuration required fewer episodes because the task did not contain traps and the optimal path could be learned quickly. The standard configuration required more training episodes because the agent had to learn not only how to reach the goal, but also how to avoid the trap. For this reason, the final experiments used a longer training duration for the standard and more difficult configurations.

A relevant observation was that DQN was particularly effective in configurations where long-term reward propagation was important. This was visible in the short-term versus long-term reward configuration, where the agent learned the route with the higher accumulated return instead of only selecting the shortest path. This behavior is consistent with the value-based nature of DQN, because the Q-values estimate expected future returns.

7.2.2 Policy Gradient Development

The policy gradient agent showed a different development behavior than DQN. Since the update is based on complete sampled episodes, the training process was more sensitive to the reward structure and showed higher variance. Early experiments showed that sparse rewards alone were not always sufficient for stable learning, especially in configurations with traps, randomized positions, or delayed reward effects.

To improve stability, several additions were used in the final implementation. A moving-average return baseline was added to reduce the variance of the policy update. Instead of updating the policy only with raw returns, the return was compared with the baseline. This helped the agent distinguish whether an episode performed better or worse than recent episodes.

An entropy term was also added to the loss function. This encouraged the policy to keep exploring during training and reduced the risk of converging too early to a poor deterministic behavior. The entropy coefficient was gradually reduced during training so that exploration was stronger at the beginning and weaker toward the end.

For selected difficult configurations, reward shaping was tested and used to improve the learning signal. This was especially relevant when the original reward was too sparse or when the agent had difficulty discovering successful trajectories. The original environment reward was still used for final evaluation, while the shaped reward was only used as a learning signal during training. This separation allowed the final evaluation to remain comparable with the other agents.

A further observation was that policy gradient sometimes behaved differently under greedy and stochastic evaluation. Since the agent learns a probability distribution over actions, the stochastic policy can still select alternative actions that are not the most likely action. This was relevant in randomized configurations, where stochastic evaluation sometimes produced stronger performance than greedy evaluation because the agent was not restricted to only the highest-probability action.

7.2.3 Actor-Critic Development

The actor-critic agent was developed as an intermediate approach between DQN and the basic policy gradient method. Like policy gradient, it learns a policy and samples actions from a probability distribution. However, it also includes a critic that estimates the value of the current state. This critic acts as a learned baseline and helps reduce the variance of the policy update.

During development, the actor-critic agent was generally more stable than the basic policy gradient agent. The critic provided additional feedback because the actor was updated using advantages rather than only raw discounted returns. This made the learning process less dependent on the absolute return of a complete episode.

Nevertheless, the actor-critic agent was still sensitive to exploration settings. Entropy regularization was therefore also used for this method. The entropy coefficient was reduced gradually during training, allowing the agent to explore more strongly at the beginning and behave more consistently later. Gradient clipping was also applied to reduce the effect of large updates during unstable episodes.

For some configurations, the learning rate had to be reduced to improve stability. This was especially relevant when the policy and value estimate changed too strongly between updates. A lower learning rate made the training process slower, but more stable. The critic coefficient was used to balance the actor loss and critic loss so that the value function supported the policy without dominating the update.

The actor-critic agent often achieved stable goal-reaching behavior, but it did not always select the strategy with the highest accumulated return. This was visible in the short-term versus long-term reward configuration, where it learned a successful route but not necessarily the route with the highest long-term reward. This shows that the critic improved stability compared with basic policy gradient, but the method was still influenced by the sampled trajectories and reward structure encountered during training.

Table 7.1 summarizes the main observations during agent development and the resulting implementation decisions.

7.3 Experimental Results and Analysis

This section presents and analyzes the experimental results of the implemented learning methods.

The analysis first gives an overview of the final greedy evaluation results across all environments. Afterwards, each environment is discussed separately. For each environment, the purpose of the configuration is briefly explained and the final evaluation results are interpreted. In addition to the final evaluation tables, the complete set of additional training plots is provided in Appendix A. These plots show important development aspects such as reward progression, success-rate development, epsilon decay for DQN, entropy decay for policy-based methods, and loss behavior.

Agent	Development observations	Resulting decisions
DQN	The agent required sufficient exploration at the beginning and stable target values during training. More difficult configurations also required more interaction with the environment.	Use epsilon-greedy exploration, replay buffer, target-network synchronization, and longer training durations for harder configurations.
Policy Gradient	The agent showed higher variance because updates were based on complete sampled episodes. It was also sensitive to sparse rewards and could converge too early to weak behavior.	Use a moving-average return baseline, entropy regularization, gradient clipping, and reward shaping in selected training runs.
Actor-Critic	The critic improved stability compared with basic Policy Gradient, but some configurations were still sensitive to large updates and exploration settings.	Use a shared actor-critic network, advantage-based policy updates, entropy regularization, gradient clipping, and lower learning rates where necessary.

Table 7.1: Summary of agent development observations and resulting decisions

7.3.1 Overview of Final Evaluation Results

Before comparing the learning agents, the random agent provides a non-learning reference. It selects actions uniformly at random and therefore does not improve through experience.

All final evaluations use the same evaluation metrics and the same number of evaluation episodes. Each trained model is evaluated for 500 episodes without further learning. Therefore, the reported evaluation values describe the behavior of fixed trained agents.

The neural network-based agents use the architectures described in Chapter 6. DQN, policy gradient, and actor-critic use compact fully connected networks with two hidden layers of 64 neurons. The action space is identical for all agents and consists of the four movement actions up, down, left, and right.

The environment configurations differ in layout, reward structure, randomization, and difficulty. Therefore, separate models were trained for the individual environment configurations. The hyperparameters were kept consistent where possible, but selected training settings, such as the number of training episodes, learning rate, entropy regularization, or reward shaping, were adapted when required by the difficulty of the configuration. These development decisions are summarized in Section 7.2.

Table 7.2 summarizes the reference evaluation results of the random agent across the six GridWorld environments. These results provide a non-learning baseline for comparing the performance of the implemented reinforcement learning methods.

Table 7.3 summarizes the final greedy evaluation results for DQN, policy gradient, and actor-critic across all environments. Greedy evaluation is used as the main comparison because it shows the deterministic policy learned by each method. For DQN, the selected action is the action with the highest predicted Q-value. For policy gradient and actor-critic, the selected action is the action with the highest probability under the learned policy.

The overview shows that all methods solved the easy, standard, short-term versus long-term reward, and fixed multi-target environments in greedy evaluation. It also shows that the largest performance differences between the methods appeared in the randomized environments.

Environment	**Success**	**Trap**	**Timeout**	**Avg. reward**	**Avg. steps**
Easy	28.50%	0.00%	71.50%	−23.08	67.68
Standard	2.20%	46.70%	51.10%	−47.96	31.46
Open Random	40.80%	0.00%	59.20%	−12.55	79.40
Random Obstacles	26.20%	40.80%	33.00%	−33.68	59.40
Short-Term vs. Long-Term	21.40%	0.00%	78.60%	−92.18	37.07
Fixed Multi-Target	26.80%	0.00%	73.20%	−45.37	45.53

Table 7.2: Random Agent reference evaluation

Environment	Method	Success	Trap	Timeout	Avg. reward	Avg. steps
Easy	DQN	100.00%	0.00%	0.00%	9.10	10.00
Easy	Policy Gradient	100.00%	0.00%	0.00%	9.10	10.00
Easy	Actor-Critic	100.00%	0.00%	0.00%	9.10	10.00
Standard	DQN	100.00%	0.00%	0.00%	1.00	10.00
Standard	Policy Gradient	100.00%	0.00%	0.00%	1.00	10.00
Standard	Actor-Critic	100.00%	0.00%	0.00%	1.00	10.00
Open Random	DQN	100.00%	0.00%	0.00%	9.45	6.48
Open Random	Policy Gradient	77.60%	0.00%	22.40%	2.21	27.48
Open Random	Actor-Critic	100.00%	0.00%	0.00%	9.45	6.46
Random Obstacles	DQN	90.60%	0.80%	8.60%	6.15	15.30
Random Obstacles	Policy Gradient	35.40%	2.00%	62.60%	−40.52	64.66
Random Obstacles	Actor-Critic	53.00%	5.60%	41.40%	−36.12	44.87
Short-Term vs. Long-Term	DQN	100.00%	0.00%	0.00%	33.60	10.00
Short-Term vs. Long-Term	Policy Gradient	100.00%	0.00%	0.00%	22.50	6.00
Short-Term vs. Long-Term	Actor-Critic	100.00%	0.00%	0.00%	22.50	6.00
Fixed Multi-Target	DQN	100.00%	0.00%	0.00%	9.30	8.00
Fixed Multi-Target	Policy Gradient	100.00%	0.00%	0.00%	9.10	10.00
Fixed Multi-Target	Actor-Critic	100.00%	0.00%	0.00%	9.30	8.00

Table 7.3: Greedy evaluation results across all environments

7.3.2 Environments Evaluation

Easy Environment

The easy environment was used to verify that the agents can learn basic goal-directed navigation. The environment contains no trap, so the main challenge is reaching the goal efficiently before the maximum number of steps is reached. Figure 7.1 shows the layout of the easy environment.

All three methods achieved a success rate of 100% in greedy evaluation. DQN, policy gradient, and actor-critic all reached an average reward of 9.10 and an average episode length of 10 steps. This shows that all methods learned the same efficient route in the basic fixed environment. The stochastic evaluations of policy gradient and actor-critic also showed strong performance. Actor-critic maintained a success rate of 100%, while policy gradient achieved 99.20%. The small difference between greedy and stochastic evaluation indicates that the learned policies were stable in this simple environment.

Because all three methods reached identical greedy performance in this environment, the easy environment mainly serves as a verification case. It confirms that the environ-

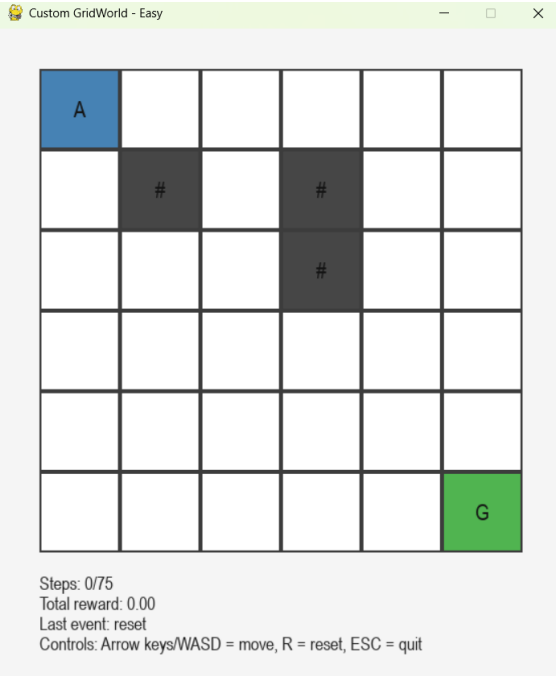


Figure 7.1: GridWorld easy layout

ment interface, training loops, and evaluation scripts work correctly before more difficult configurations are considered.

Standard Environment

The standard environment extends the fixed layout by adding a trap. This tests whether the agents can learn to reach the goal while avoiding an unsafe terminal state. Figure 7.2 shows the layout of the standard environment.

In greedy evaluation, all three methods achieved a success rate of 100%, a trap rate of 0%, and a timeout rate of 0%. The average reward was 1.00 and the average episode length was 10 steps for all methods. This indicates that all three agents learned a stable path to the goal and avoided the trap completely.

The stochastic results show a difference between the policy-based methods. Actor-critic remained stable, with a success rate of 98.80% and no trap episodes. Policy gradient, however, achieved only 59.00% success in stochastic mode and reached the trap in 38.40% of the episodes. This shows that the greedy policy gradient policy solved the task, but the

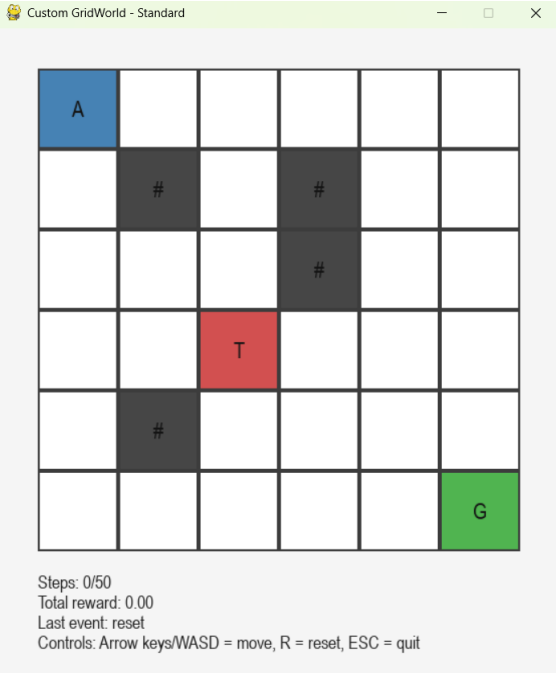


Figure 7.2: GridWorld standard layout

stochastic policy still assigned enough probability to unsafe actions that trap-reaching behavior remained possible.

The standard environment therefore demonstrates the difference between deterministic success and probabilistic safety. A policy can produce perfect greedy behavior while still containing unsafe action probabilities when sampled stochastically. This is especially relevant for policy-based methods, where the output of the network is a probability distribution rather than a single deterministic action.

Open Random Environment

The open random environment evaluates generalization to changing start and goal positions. Since the environment does not contain walls or traps, the main challenge is learning a general navigation strategy instead of memorizing one fixed path. Figure 7.3 shows the layout of the open random environment.

DQN and actor-critic both achieved a success rate of 100% in greedy evaluation. Their average rewards were almost identical, with both methods reaching 9.45. Their average

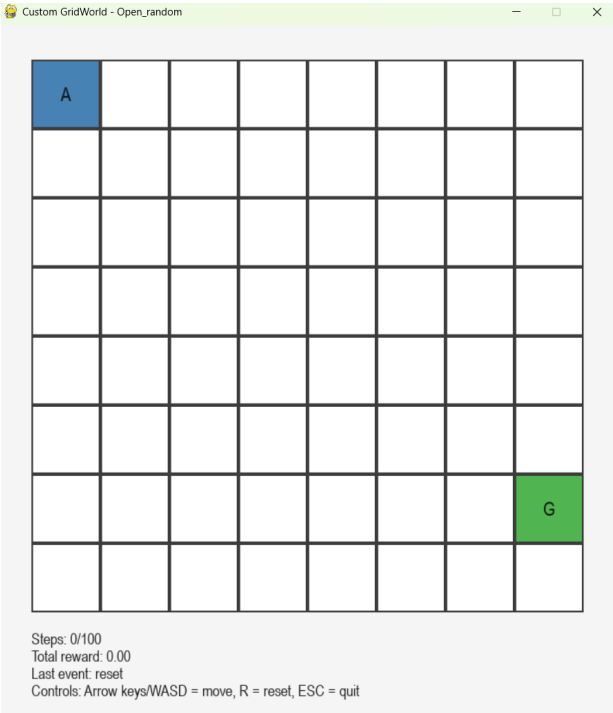


Figure 7.3: GridWorld open random layout

number of steps was also very similar, with 6.48 steps for DQN and 6.46 steps for actor-critic. This shows that both methods learned a strong general navigation policy based on the relative positions of the agent and the goal.

Policy gradient performed weaker in greedy evaluation. It achieved a success rate of 77.60%, a timeout rate of 22.40%, an average reward of 2.21, and an average episode length of 27.48 steps. However, in stochastic evaluation, policy gradient achieved 99.40% success. This difference indicates that the learned policy gradient distribution still contained useful action probabilities, but the deterministic greedy extraction of the policy was less reliable in randomized start-goal settings.

The open random environment therefore shows that DQN and actor-critic generalized well under greedy evaluation, while policy gradient required stochastic action sampling to show its strongest behavior. This is an important result because it shows that evaluating policy-based methods only with greedy action selection can hide useful behavior contained in the learned action distribution.

Random Obstacles Environment

The random obstacles environment is the most difficult environment in the main comparison. It combines randomized start and goal positions with walls and a trap. The agent must therefore generalize navigation behavior while avoiding obstacles and unsafe terminal states. Figure 7.4 shows different layouts of the random obstacles environment.

DQN achieved the strongest greedy performance in this environment. It reached a success rate of 90.60%, a trap rate of 0.80%, and a timeout rate of 8.60%. The average reward was 6.15 and the average number of steps was 15.30. This shows that DQN learned a comparatively robust policy for the randomized obstacle layout.

Actor-critic achieved 53.00% success in greedy evaluation, with a trap rate of 5.60% and a timeout rate of 41.40%. Policy gradient achieved 35.40% success, with a trap rate of 2.00% and a timeout rate of 62.60%. These greedy results show that the policy-based methods had more difficulty producing stable deterministic behavior in this environment.

Figure A.1 supports the final evaluation results by showing the learning behavior in the random obstacles environment. The environment is more unstable than the fixed layouts because the agent cannot rely on one memorized path. It must react to changing start-goal positions and obstacle arrangements.

The stochastic evaluation provides an important additional insight. In stochastic mode, actor-critic achieved 90.20% success and policy gradient achieved 92.00% success. This is much higher than their greedy performance. Therefore, the policy distributions learned by these methods still contained useful behavior, but converting them into a deterministic greedy policy reduced performance.

Short-Term versus Long-Term Reward Environment

The short-term versus long-term reward environment was designed to test route preference and delayed reward behavior. In this environment, all methods reached the goal successfully, but they did not learn the same route. Figure 7.5 shows the layout of the short-term vs. long-term reward environment.

DQN achieved a success rate of 100%, an average reward of 33.60, and an average episode length of 10 steps. This indicates that DQN learned the longer path with the higher

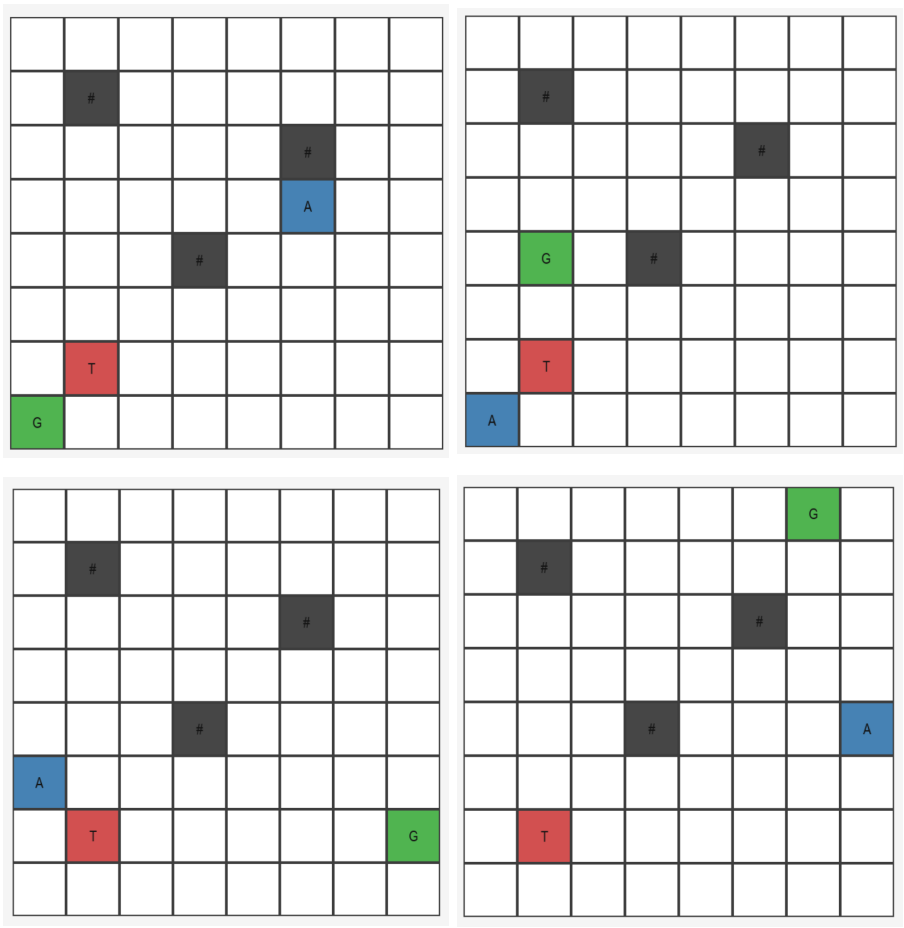


Figure 7.4: Gridworld random obstacles layouts

accumulated return. This result is consistent with the value-based nature of DQN, where the agent estimates the expected future return of actions.

Policy gradient and actor-critic also achieved 100% success in greedy evaluation, but both obtained an average reward of 22.50 and reached the goal in 6 steps. This means that both methods learned the shorter path. The shorter path solves the task, but it produces a lower accumulated reward because of the delayed penalties along the route.

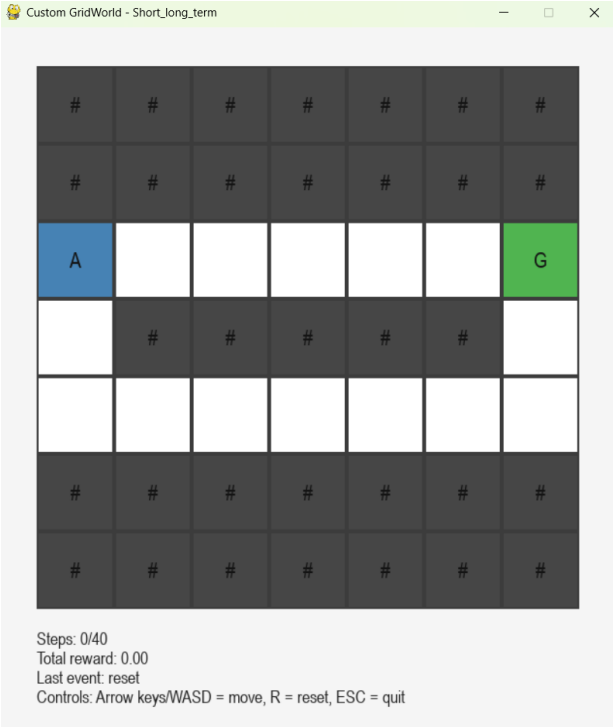


Figure 7.5: GridWorld short-term vs. long-term reward layout

Fixed Multi-Target Environment

The fixed multi-target environment was designed to evaluate target selection behavior. The environment contains two valid terminal targets as shown in Figure 7.6. Reaching either target ends the episode successfully and gives the same terminal reward.

All methods achieved a success rate of 100% in greedy evaluation. DQN reached an average reward of 9.30 and an average episode length of 8 steps. Actor-critic achieved the same average reward and average number of steps. Policy gradient also solved the task, but required 10 steps on average and achieved a slightly lower average reward of 9.10.

The target selection behavior differed between the methods. DQN selected Target 2 in all 500 evaluation episodes. Actor-critic and policy gradient selected Target 1 in all 500 evaluation episodes. Since both targets produce the same reward, this does not mean that one method selected an incorrect target. Instead, it shows that different methods can converge to different valid policies when the environment does not specify a preferred target depending on the algorithm, exploration behavior and initialization.

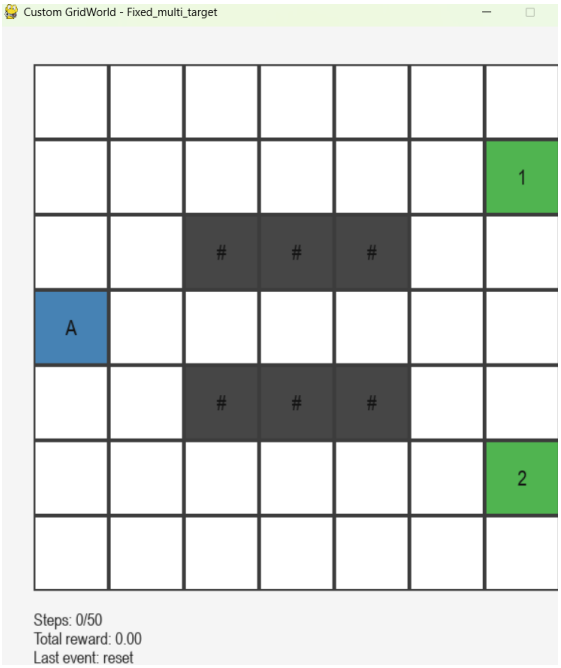


Figure 7.6: GridWorld fixed multi-target layout

7.3.3 Stochastic Policy Evaluation

Policy gradient and actor-critic were also evaluated in stochastic mode. Table 7.4 presents the stochastic evaluation results for the policy-based methods, namely policy gradient and actor-critic, across all GridWorld environments. It shows that stochastic evaluation can differ strongly from greedy evaluation for policy-based methods. This is especially visible in the open random and random obstacles environments.

These differences show that the learned policy distributions still contained useful behavior, even when the greedy action selection did not produce the best deterministic policy. However, the stochastic results also show that exploration can remain risky. In the standard environment, stochastic policy gradient reached the trap in 38.40% of the episodes, even though the greedy policy reached the goal in 100% of the episodes.

7.4 Evaluation with respect to objectives and requirements

This section evaluates whether the developed system satisfies the objectives and requirements defined in the requirements analysis. The focus is not yet on the numerical performance differences between the learning methods. Instead, this section verifies whether the developed framework provides the necessary environment, agent structure, training workflow, evaluation workflow, visualization, and result logging required for a meaningful comparison.

The main objective of the thesis was to develop a custom reinforcement learning environment and use it as the basis for comparing selected deep reinforcement learning methods. This objective was fulfilled. A custom GridWorld environment was developed without relying on a predefined reinforcement learning benchmark. The environment defines its own state representation, action space, reward system, termination conditions, truncation condition, and configuration variants. It therefore satisfies the requirement that the thesis should not only use an existing benchmark, but develop the environment as part of the work itself.

The objective of creating an environment suitable for reinforcement learning was also fulfilled. The environment follows the standard agent-environment interaction structure. At the beginning of each episode, the environment is reset. During an episode, the agent selects an action, the environment processes the action, updates the state, calculates the reward, and returns the next observation together with termination and truncation

Environment	**Method**	**Success**	**Trap**	**Timeout**	**Avg. reward**	**Avg. steps**
Easy	Policy Gradient	99.20%	0.00%	0.80%	8.13	10.83
Easy	Actor-Critic	100.00%	0.00%	0.00%	7.58	11.52
Standard	Policy Gradient	59.00%	38.40%	2.60%	−10.31	10.65
Standard	Actor-Critic	98.80%	0.00%	1.20%	−1.91	11.41
Open Random	Policy Gradient	99.40%	0.00%	0.60%	6.57	23.64
Open Random	Actor-Critic	100.00%	0.00%	0.00%	9.41	6.60
Random Obstacles	Policy Gradient	92.00%	7.80%	0.20%	1.84	14.61
Random Obstacles	Actor-Critic	90.20%	8.80%	1.00%	0.18	11.77
Short-Term vs. Long-Term	Policy Gradient	56.60%	0.00%	43.40%	−28.16	20.96
Short-Term vs. Long-Term	Actor-Critic	100.00%	0.00%	0.00%	22.20	6.15
Fixed Multi-Target	Policy Gradient	100.00%	0.00%	0.00%	6.12	11.25
Fixed Multi-Target	Actor-Critic	100.00%	0.00%	0.00%	7.59	8.88

Table 7.4: Stochastic evaluation results for policy-based methods

information. This makes the environment suitable for episodic reinforcement learning and allows all implemented agents to interact with it through the same interface.

The environment also satisfies the requirement of having a clearly defined observation and action space. The basic state representation contains the agent position and the goal position. In the fixed multi-target environment, the observation was extended to include the positions of both targets. The action space remains consistent across all configurations and consists of the four discrete movement actions up, down, left, and right. This consistency is important because the learning agents can be tested on different environment configurations without changing the action interface.

The reward-system requirement was fulfilled through the implementation of step penalties, invalid movement penalties, terminal goal rewards, trap penalties, timeout handling, and additional reward structures for advanced configurations. The short-term versus long-term reward environment further extends this by adding route-dependent rewards and penalties. The fixed multi-target environment extends the terminal condition by allowing more than one valid target. These extensions show that the environment is flexible enough to test more than simple goal-reaching behavior.

The objective of comparing different reinforcement learning methods under identical conditions was also fulfilled. DQN, policy gradient, and actor-critic were implemented as separate learning agents, but all of them interact with the same environment interface. The same environment configurations, episode structure, and evaluation metrics are used for all methods. This allows the results to be interpreted as differences in learning behavior rather than differences caused by different task definitions.

The evaluation and logging requirements were also satisfied. Training and evaluation results are stored in CSV files. The recorded metrics include success rate, trap rate, timeout rate, average reward, average number of steps, training loss, recent success rate, recent average reward, and method-specific values such as entropy or value estimates. For the fixed multi-target environment, the selected target is also recorded. These logs make the experiments easier to analyze and allow the results to be visualized using plots.

The visualization and manual testing requirements were fulfilled through the text-based rendering and the Pygame renderer. These tools were useful during development because they made it possible to inspect environment layouts, verify walls and traps, check randomized start-goal sampling, and confirm that the multi-target environment displayed

both targets correctly. This supports the reliability of the environment before using it for training.

The requirement of extensibility was also fulfilled. The environment started with simple fixed configurations and was later extended with randomized start-goal positions, random obstacles, short-term versus long-term reward paths, and fixed multi-target behavior. These additions were implemented without changing the general agent-environment interface. This shows that the framework is modular and can support additional future experiments.

Table 7.5 summarizes how each objective or requirement was addressed during the implementation and evaluation phases.

Overall, the system satisfies the main objectives and requirements of the thesis. A custom environment was developed, several environment configurations were added, the selected learning methods were implemented, and a common training and evaluation framework was created. The following sections therefore focus on the numerical results and compare how the implemented learning methods performed in the developed environments.

Objective / Requirement	**Evaluation**	**Status**
Development of a custom GridWorld environment	A self-developed GridWorld environment was implemented with states, actions, rewards, termination, truncation, walls, traps, and several configuration variants.	Fulfilled
Suitability for reinforcement learning	The environment provides reset and step functions and follows an episodic agent-environment interaction structure.	Fulfilled
Defined observation space	The environment uses coordinate-based observations and extends them for the multi-target environment.	Fulfilled
Defined action space	All environments use the same four discrete actions: up, down, left, and right.	Fulfilled
Defined reward system	The environment includes step penalties, invalid movement penalties, goal rewards, trap penalties, timeout handling, and route-dependent reward variants.	Fulfilled
Multiple difficulty configurations	Easy, Standard, Open Random, Random Obstacles, Short-Term vs. Long-Term Reward, and Fixed Multi-Target configurations were created.	Fulfilled
Manual visualization and testing	The environment can be inspected using text rendering and the Pygame renderer.	Fulfilled
Random baseline	A Random Agent was implemented and used as a non-learning reference.	Fulfilled
DQN implementation	DQN was implemented with neural-network Q-value approximation, replay memory, and target-network updates.	Fulfilled
Policy Gradient implementation	Policy Gradient was implemented using sampled actions, discounted returns, a baseline, entropy regularization, and greedy/stochastic evaluation.	Fulfilled
Actor-Critic implementation	Actor-Critic was implemented using an actor for policy learning and a critic for value estimation.	Fulfilled
Common evaluation metrics	Success rate, trap rate, timeout rate, average reward, average steps, route behavior, and target selection were recorded.	Fulfilled
Result logging	Training and evaluation results were saved to CSV files and used for tables and plots.	Fulfilled
Comparability of methods	All methods were trained and evaluated on the same environment interface and under the same evaluation metrics.	Fulfilled
Extensibility	New configurations were added without changing the main environment-agent interface.	Fulfilled

Table 7.5: Evaluation of objectives and requirements

8 Conclusion and Outlook

8.1 Conclusion

This thesis investigated the comparative behavior of selected deep reinforcement learning methods in a self-developed GridWorld environment. The main goal was not only to apply existing algorithms, but to design a controlled experimental framework in which different learning approaches can be implemented, trained, evaluated, and compared under the same task conditions.

The work achieved this goal by developing a custom GridWorld environment with clearly defined observations, actions, rewards, termination conditions, and multiple environment configurations. The environment was extended from simple fixed layouts to more advanced configurations. This made it possible to evaluate not only whether an agent reaches the goal, but also how the agent behaves under different task structures.

The implemented framework also provided the required components for a structured reinforcement learning experiment. It includes separate learning agents, training scripts, evaluation scripts, result logging, visualization support, and plotting utilities. This modular structure made it possible to compare the learning methods using the same environment interface and the same evaluation criteria.

The evaluation showed that all selected learning methods were able to solve the simpler fixed environments. However, the more advanced configurations revealed clear differences between the methods. DQN showed the most stable greedy behavior overall and was especially effective in situations where long-term reward propagation was important. Policy gradient was able to learn successful behavior, but it was more sensitive to the reward structure and to the difference between greedy and stochastic evaluation. Actor-critic showed stronger stability than the basic policy gradient method in several cases, but it did not always select the highest-return strategy.

One central conclusion of the thesis is that success rate alone is not sufficient for evaluating reinforcement learning agents. Several agents reached the goal successfully, but learned different strategies. Therefore, metrics such as average reward, average steps, timeout rate, selected route, and selected target are necessary to understand the quality and behavior of the learned policy. The custom GridWorld environment proved useful for making these differences visible in a controlled and understandable way.

Overall, the thesis demonstrates that a small custom environment can be a meaningful basis for analyzing deep reinforcement learning methods. Although the environment is simple compared with large benchmark tasks, it provides enough flexibility to study important reinforcement learning aspects such as delayed rewards, stochastic policies, trap avoidance, generalization, and multiple valid solutions. The developed framework therefore provides a solid foundation for further experiments and future extensions.

8.2 Outlook

The developed framework provides several possibilities for future work. A first direction is the implementation of more advanced reinforcement learning methods. For the value-based approach, double DQN, dueling DQN, prioritized experience replay, or rainbow-style combinations could be added. These methods could be compared with the implemented DQN agent to analyze whether they improve stability and performance in the randomized obstacle environment. For policy-based learning, methods such as proximal policy optimization (PPO) could be added because they are designed to improve the stability of direct policy optimization. For actor-critic learning, advantage actor-critic or asynchronous actor-critic variants could be investigated.

A second extension concerns the environment itself. The GridWorld could be enlarged, dynamic obstacles could be added, or the transition system could be made stochastic. This would increase the difficulty of the navigation task and make generalization more challenging.

Another possible extension is image-based learning. Instead of using coordinate vectors as input, the agents could receive rendered grid images. This would require convolutional neural networks and would shift the task closer to visual deep reinforcement learning. Such an extension would allow a comparison between coordinate-based and image-based state representations.

The fixed multi-target environment could also be extended further. Future work could introduce target-specific rewards, hidden target preferences, changing target positions, or sequential target collection. This would make it possible to analyze whether agents can learn preferences, priorities, or long-term plans involving multiple objectives.

Finally, the framework could be extended toward multi-agent reinforcement learning. A second agent could be added to the environment, either as a cooperative partner or as an opponent. This would create new research questions involving coordination, competition, communication, and interaction between agents. The current single-agent GridWorld framework provides a suitable foundation for such an extension because the environment logic, evaluation workflow, and agent structure are already modular.

In summary, this thesis provides a complete starting point for controlled comparisons of deep reinforcement learning methods in a custom environment. The implemented framework can be used as a basis for further experiments, more advanced algorithms, more complex environments, and future multi-agent extensions.

Bibliography

[1] Arulkumaran, Kai ; Deisenroth, Marc P. ; Brundage, Miles ; Bharath, Anil A.: Deep Reinforcement Learning: A Brief Survey. In: *IEEE Signal Processing Magazine* 34 (2017), Nr. 6, S. 26–38

[2] Auer, Peter ; Cesa-Bianchi, Nicolò ; Fischer, Paul: Finite-time Analysis of the Multiarmed Bandit Problem. In: *Machine Learning* 47 (2002), Nr. 2–3, S. 235–256

[3] Bertsekas, Dimitri P. ; Tsitsiklis, John N.: *Neuro-Dynamic Programming*. Athena Scientific, 1996. – ISBN 978-1886529106

[4] Brownlee, Jason: *Neural Networks are Function Approximation Algorithms*. https://machinelearningmastery.com/neural-networks-are-function-approximators/. 2020. – Accessed: 27.05.2026

[5] Dai, Yanyan ; Yang, Seokho ; Lee, Kidong: Sensing and Navigation for Multiple Mobile Robots Based on Deep Q-Network. In: *Remote Sensing* 15 (2023), Nr. 19, S. 4757

[6] François-Lavet, Vincent ; Henderson, Peter ; Islam, Riashat ; Bellemare, Marc G. ; Pineau, Joelle: An Introduction to Deep Reinforcement Learning. In: *Foundations and Trends in Machine Learning* 11 (2018), Nr. 3–4, S. 219–354

[7] GeeksforGeeks: *Agent-Environment Interface in AI*. https://www.geeksforgeeks.org/artificial-intelligence/agent-environment-interface-in-ai/. 2025. – Accessed: 31.05.2026

[8] Goodfellow, Ian ; Bengio, Yoshua ; Courville, Aaron: *Deep Learning*. The MIT Press, 2016. – ISBN 9780262035613

[9] Greensmith, Evan ; Bartlett, Peter L. ; Baxter, Jonathan: Variance Reduction Techniques for Gradient Estimates in Reinforcement Learning. In: *Journal of Machine Learning Research* 5 (2004), S. 1471–1530

[10] Grondman, Ivo ; Busoniu, Lucian ; Lopes, Gabriel A. D. ; Babuska, Robert: A Survey of Actor-Critic Reinforcement Learning: Standard and Natural Policy Gradients. In: *IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews* 42 (2012), Nr. 6, S. 1291–1307

[11] Haarnoja, Tuomas ; Zhou, Aurick ; Abbeel, Pieter ; Levine, Sergey: Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In: *Proceedings of the 35th International Conference on Machine Learning* Bd. 80, 2018, S. 1861–1870

[12] Huber, Peter J.: Robust Estimation of a Location Parameter. In: *The Annals of Mathematical Statistics* 35 (1964), Nr. 1, S. 73–101

[13] Kaelbling, Leslie P. ; Littman, Michael L. ; Moore, Andrew W.: Reinforcement Learning: A Survey. In: *Journal of Artificial Intelligence Research* 4 (1996), S. 237–285

[14] Konda, Vijay R. ; Tsitsiklis, John N.: Actor-Critic Algorithms. In: *Advances in Neural Information Processing Systems 12*, 1999, S. 1008–1014

[15] Lin, Long-Ji: Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching. In: *Machine Learning* 8 (1992), Nr. 3–4, S. 293–321

[16] Mnih, Volodymyr ; Badia, Adrià P. ; Mirza, Mehdi ; Graves, Alex ; Harley, Tim ; Lillicrap, Timothy P. ; Silver, David ; Kavukcuoglu, Koray: Asynchronous Methods for Deep Reinforcement Learning. In: *Proceedings of the 33rd International Conference on Machine Learning* Bd. 48, 2016, S. 1928–1937

[17] Mnih, Volodymyr ; Kavukcuoglu, Koray ; Silver, David ; Graves, Alex ; Antonoglou, Ioannis ; Wierstra, Daan ; Riedmiller, Martin: Playing Atari with Deep Reinforcement Learning. In: *arXiv preprint arXiv:1312.5602* (2013). – URL https://arxiv.org/abs/1312.5602

[18] Mnih, Volodymyr ; Kavukcuoglu, Koray ; Silver, David ; Rusu, Andrei A. ; Veness, Joel ; Bellemare, Marc G. ; Graves, Alex ; Riedmiller, Martin ; Fidjeland, Andreas K. ; Ostrovski, Georg ; Petersen, Stig ; Beattie, Charles ; Sadik, Amir ; Antonoglou, Ioannis ; King, Helen ; Kumaran, Dharshan ; Wierstra, Daan ; Legg, Shane ; Hassabis, Demis: Human-level control through deep reinforcement learning. In: *Nature* 518 (2015), Nr. 7540, S. 529–533

[19] Mon, Bisni F. ; Wasfi, Asma ; Hayajneh, Mohammad ; Slim, Ahmad ; Abu Ali, Najah: Reinforcement Learning in Education: A Literature Review. In: *Informatics* 10 (2023), Nr. 3, S. 74

[20] Nair, Vinod ; Hinton, Geoffrey E.: Rectified Linear Units Improve Restricted Boltzmann Machines. In: *Proceedings of the 27th International Conference on Machine Learning*, 2010, S. 807–814

[21] Puterman, Martin L.: *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994. – ISBN 0-47 1-72782-2

[22] Schulman, John ; Moritz, Philipp ; Levine, Sergey ; Jordan, Michael ; Abbeel, Pieter: High-Dimensional Continuous Control Using Generalized Advantage Estimation. In: *International Conference on Learning Representations*, 2016

[23] Sutton, Richard S.: *Temporal Credit Assignment in Reinforcement Learning*, University of Massachusetts Amherst, Dissertation, 1984

[24] Sutton, Richard S. ; Barto, Andrew G.: *Reinforcement Learning: An Introduction*. 2. The MIT Press, 2018

[25] Sutton, Richard S. ; McAllester, David ; Singh, Satinder ; Mansour, Yishay: Policy Gradient Methods for Reinforcement Learning with Function Approximation. In: *Advances in Neural Information Processing Systems 12*, 1999, S. 1057–1063

[26] Sutton, Richard S. ; McAllester, David ; Singh, Satinder ; Mansour, Yishay: Policy Gradient Methods for Reinforcement Learning with Function Approximation. In: *Advances in Neural Information Processing Systems 12*, 2000, S. 1057–1063

[27] Szepesvári, Csaba: *Algorithms for Reinforcement Learning*. Morgan & Claypool Publishers, 2010. – ISBN 978-1-60845-492-1

[28] Thrun, Sebastian B.: Efficient Exploration in Reinforcement Learning / Carnegie Mellon University. 1992 (CMU-CS-92-102). – Forschungsbericht

[29] Tokic, Michel: Adaptive ϵ-Greedy Exploration in Reinforcement Learning Based on Value Differences. In: *KI 2010: Advances in Artificial Intelligence* Bd. 6359, Springer, 2010, S. 203–210

[30] Watkins, Christopher J. C. H. ; Dayan, Peter: Q-learning. In: *Machine Learning* 8 (1992), Nr. 3, S. 279–292

[31] Williams, Ronald J.: Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. In: *Machine Learning* 8 (1992), S. 229–256

[32] Xiao, Gwen: *Feed Forward Neural Network Basics 1: Overview*. https://medium.com/@justygwen/feed-forward-neural-network-basics-1-overview-bccf7a217d49. 2023. – Accessed: 27.05.2026

[33] Zai, Alexander ; Brown, Brandon: *Deep Reinforcement Learning in Action*. Manning Publications Co., 2020. – ISBN 9781617295430

A Training Plots

This appendix contains selected training plots that support the interpretation of the results in Chapter 7. The main evaluation chapter focuses on final evaluation results obtained from fixed trained models. The plots in this appendix show learning behavior during training. Therefore, the values shown in the training curves are not expected to match the final evaluation tables exactly. During training, the agents may still explore, the policy parameters are continuously updated, and policy-based methods sample actions from a probability distribution.

The plots are included as supporting material because they help illustrate learning stability and instability.

Figure A.1 compares the recent success rate during training in the random obstacles environment. This configuration is difficult because the obstacle layout and navigation problem vary between episodes. The plots show how consistently each method learned successful behavior under changing task conditions. DQN and actor-critic provide more stable learning mechanisms through value estimation, while policy gradient is more sensitive to stochastic trajectories and therefore can show stronger fluctuations during training.

Figure A.2 shows the entropy decay of the two policy-based agents in the random obstacles environment. Entropy is relevant because policy gradient and actor-critic learn stochastic policies. A higher entropy indicates stronger exploration, while a lower entropy indicates that the policy becomes more deterministic. The decay therefore illustrates how exploration is gradually reduced during training. This is especially important in the random obstacles environment, where too little exploration can lead to weak policies and too much exploration can prevent stable goal-reaching behavior.

Figure A.3 compares the recent average reward during training in the short-term versus long-term reward environment. This metric is important because reaching the goal alone is not sufficient in this configuration. The agents should ideally learn the route with

the higher accumulated return. The reward plots therefore support the interpretation of whether an agent only learns a successful path or whether it also learns the more beneficial long-term strategy.

Figure A.4 compares the episode length during training in the short-term versus long-term reward environment. The number of steps is useful here because the shorter route and the higher-return route are not identical. A lower number of steps does not automatically mean a better strategy. Therefore, this plot must be interpreted together with the reward plot in Figure A.3. This distinction helps explain why some agents can reach the goal efficiently but still obtain a lower accumulated return.

Figure A.6 compares the recent success rate during training in the standard environment. The plot shows whether the agents gradually learn reliable goal-reaching behavior despite the presence of the trap. This figure supports the final evaluation because it shows not only the final trained performance, but also how the agents developed toward that performance during training.

Figure A.7 shows the recent average reward during training in the standard environment. This metric combines several aspects of behavior, including goal reaching, trap avoidance, invalid moves, and episode length. Therefore, it provides a more complete view of training development than success rate alone. Stable reward improvement indicates that the agent is not only reaching the goal more often, but also doing so with fewer penalties.

Figure A.8 compares the recent average reward during training in the open random environment. In this configuration, start and goal positions change between episodes, while no traps or obstacles are present. The plot is therefore useful for observing whether the agents learn a general navigation strategy instead of memorizing one fixed route. More stable reward development indicates better adaptation to the randomized task structure.

Figure A.9 shows the entropy decay of policy gradient and actor-critic in the open random environment. Since this environment requires general navigation across changing start and goal positions, exploration remains important during training. The entropy curves show how the policy-based agents gradually reduce randomness while still learning from varied trajectories. This supports the interpretation of the differences between greedy and stochastic evaluation for these methods.

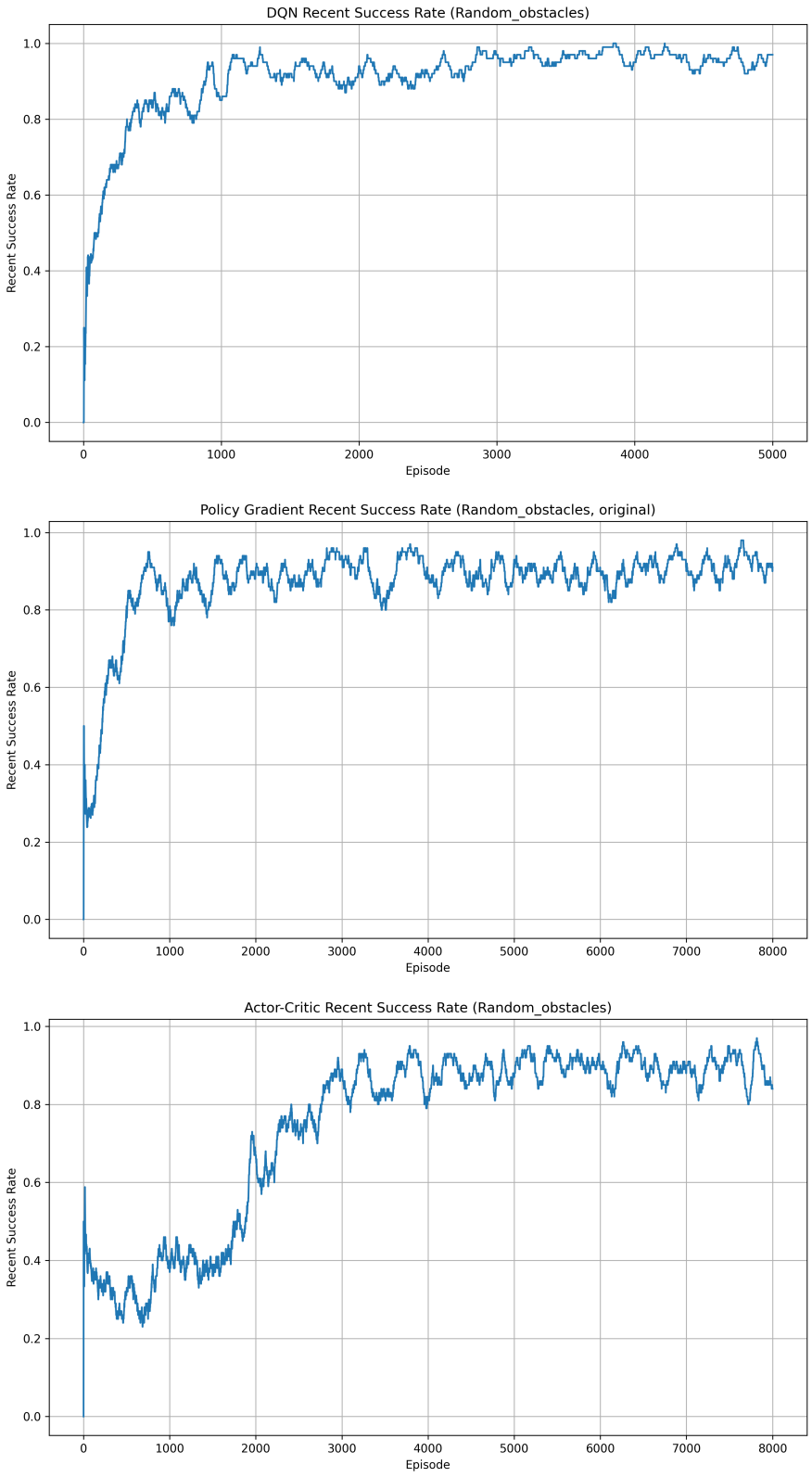


Figure A.1: Recent success rate during training in the random obstacles environment for DQN, policy gradient, and actor-critic

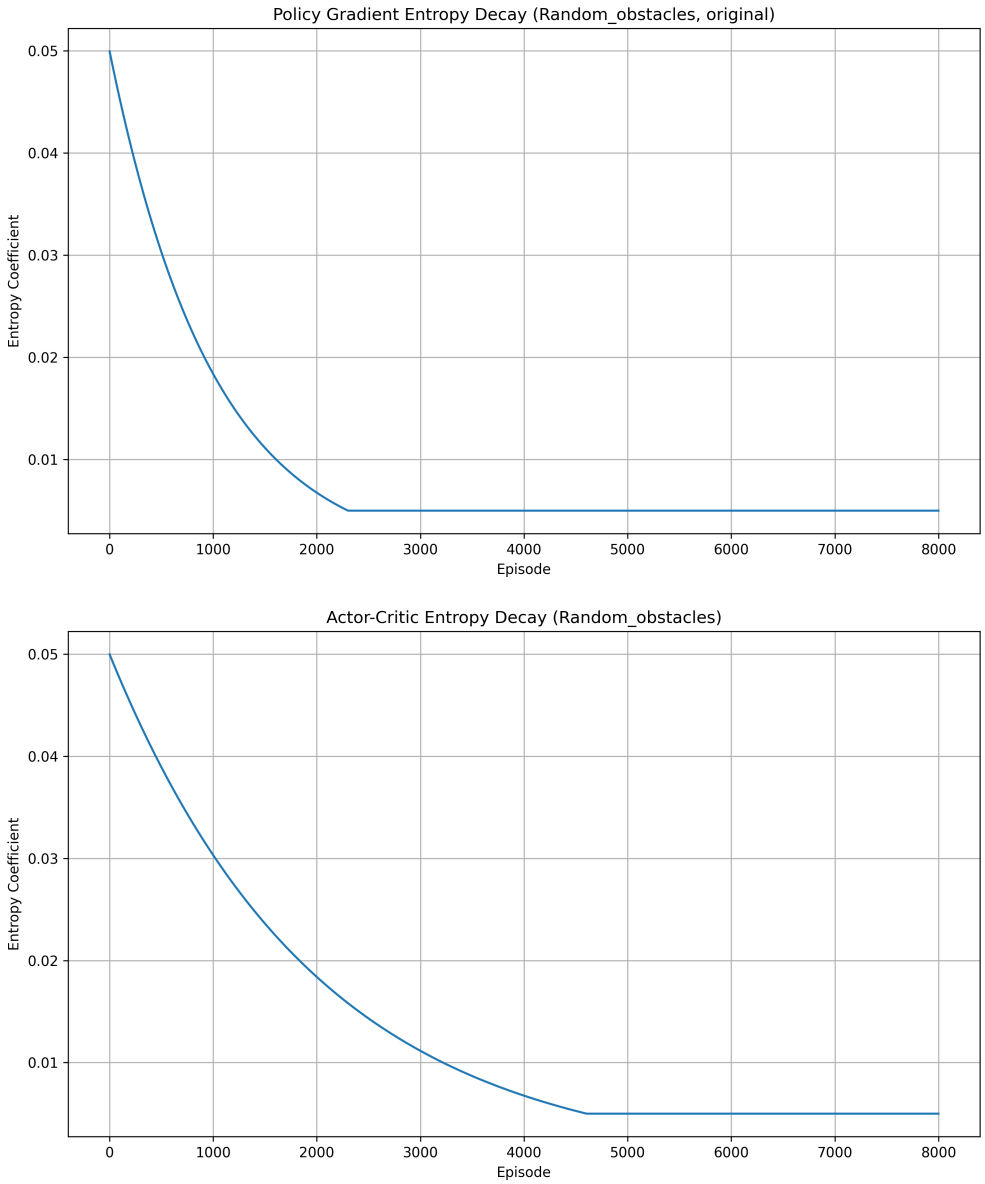


Figure A.2: Entropy decay during training in the random obstacles environment for policy gradient and actor-critic

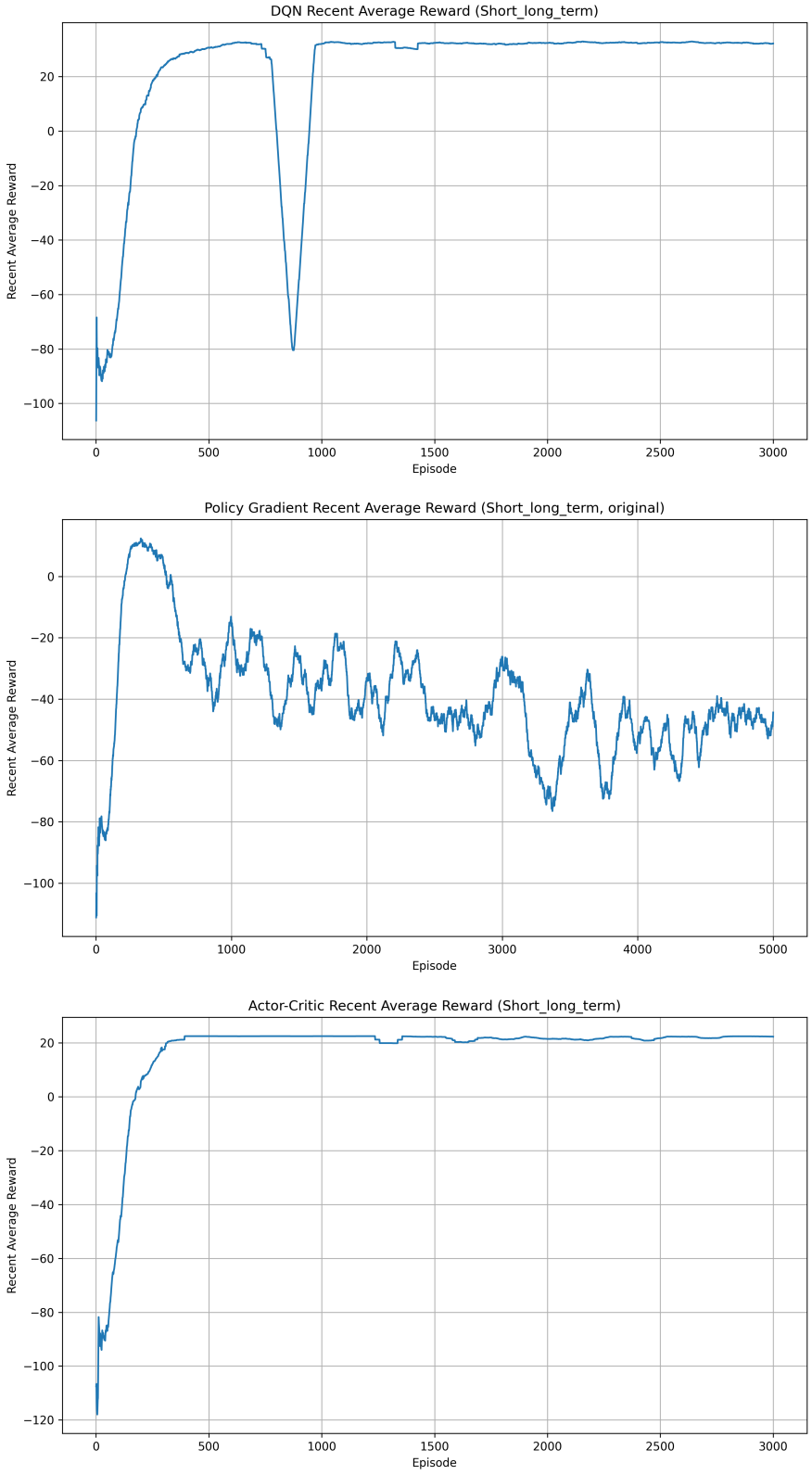


Figure A.3: Recent average reward during training in the short-term vs. long-term reward environment for DQN, policy gradient, and actor-critic

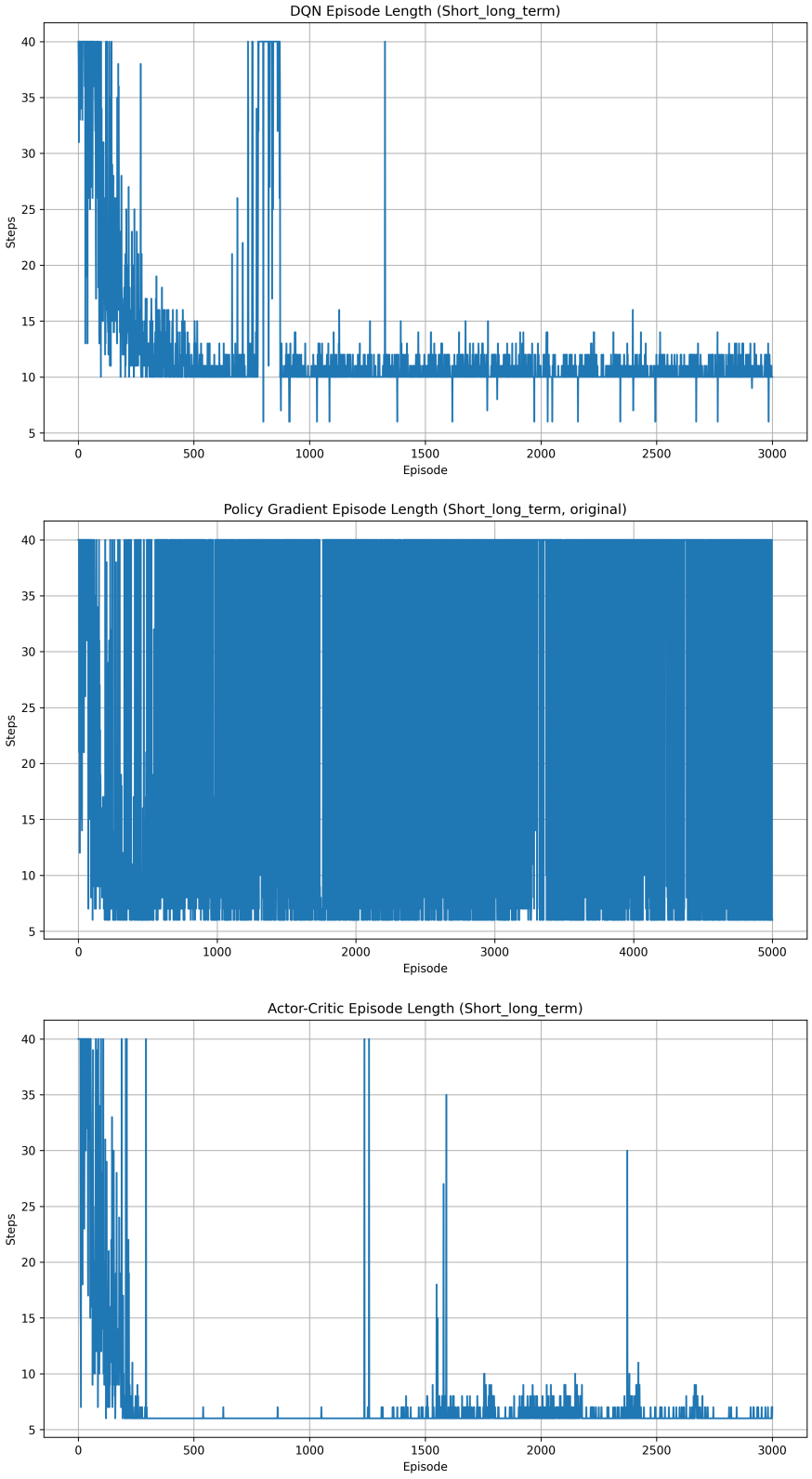


Figure A.4: Episode steps during training in the short-term vs. long-term reward environment for DQN, policy gradient, and actor-critic

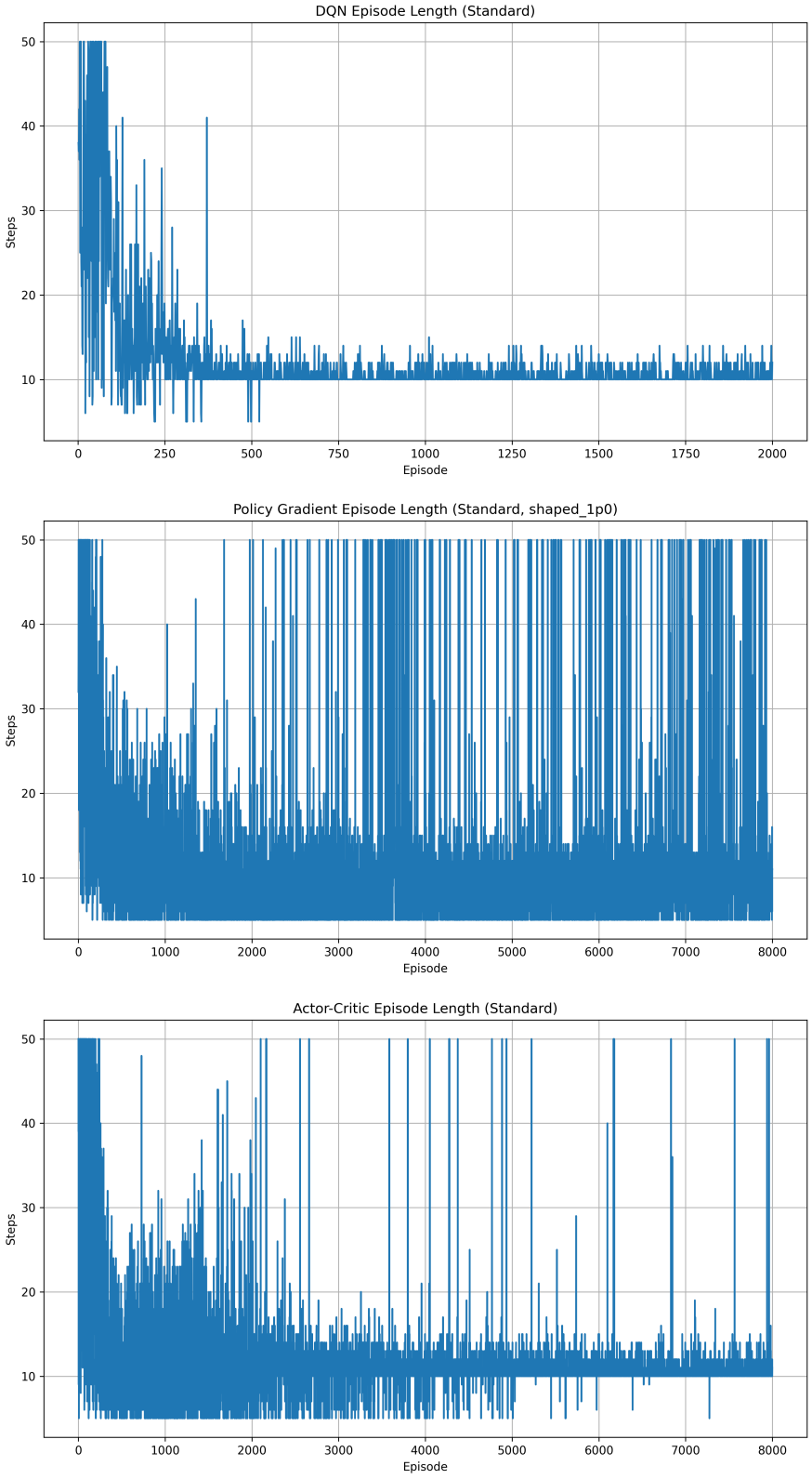


Figure A.5: Episode steps during training in the standard environment for DQN, policy gradient, and actor-critic

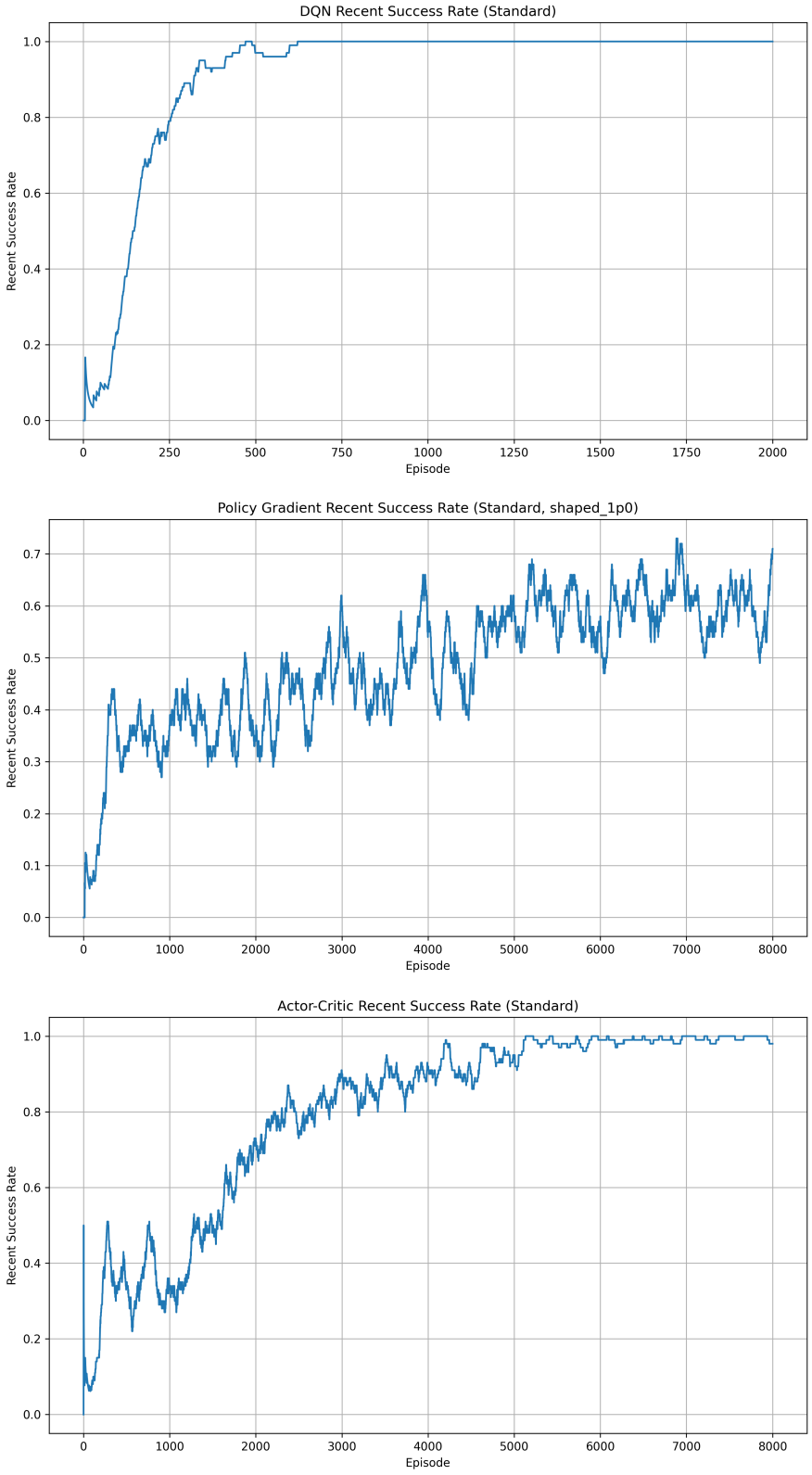


Figure A.6: Recent success rate during training in the standard environment for DQN, policy gradient, and actor-critic

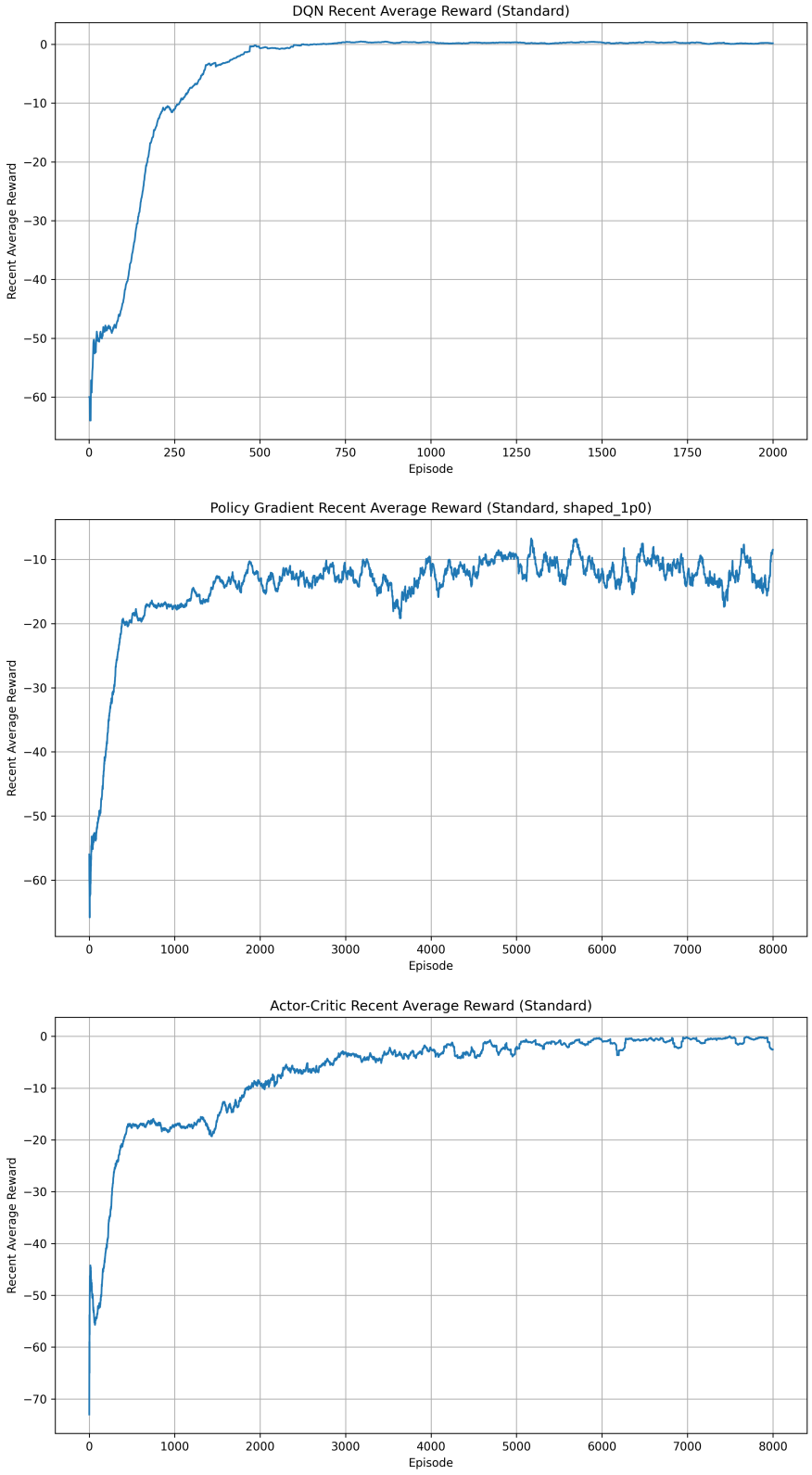


Figure A.7: Recent average reward during training in the standard environment for DQN, policy gradient, and actor-critic

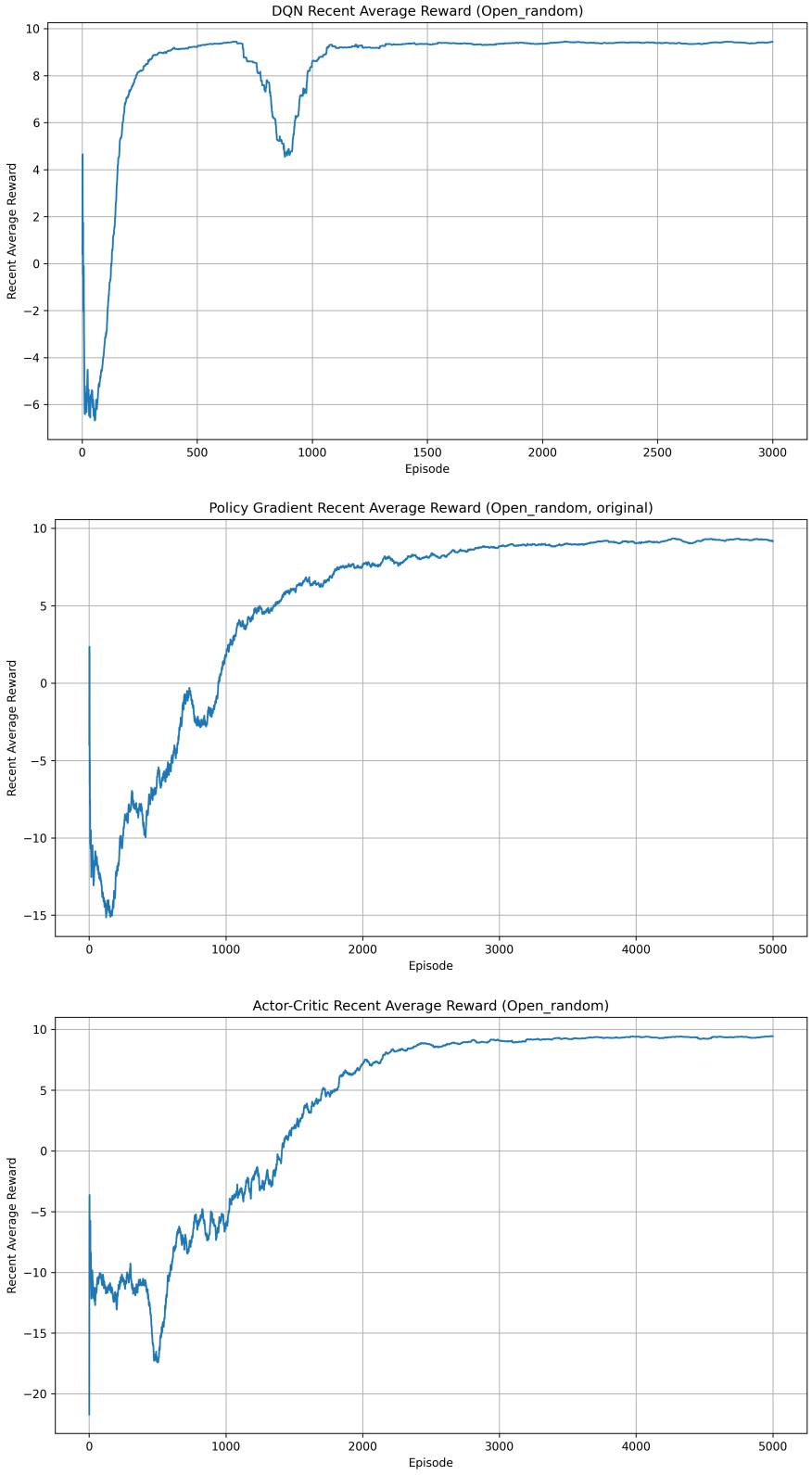


Figure A.8: Recent average reward during training in the open random environment for DQN, policy gradient, and actor-critic

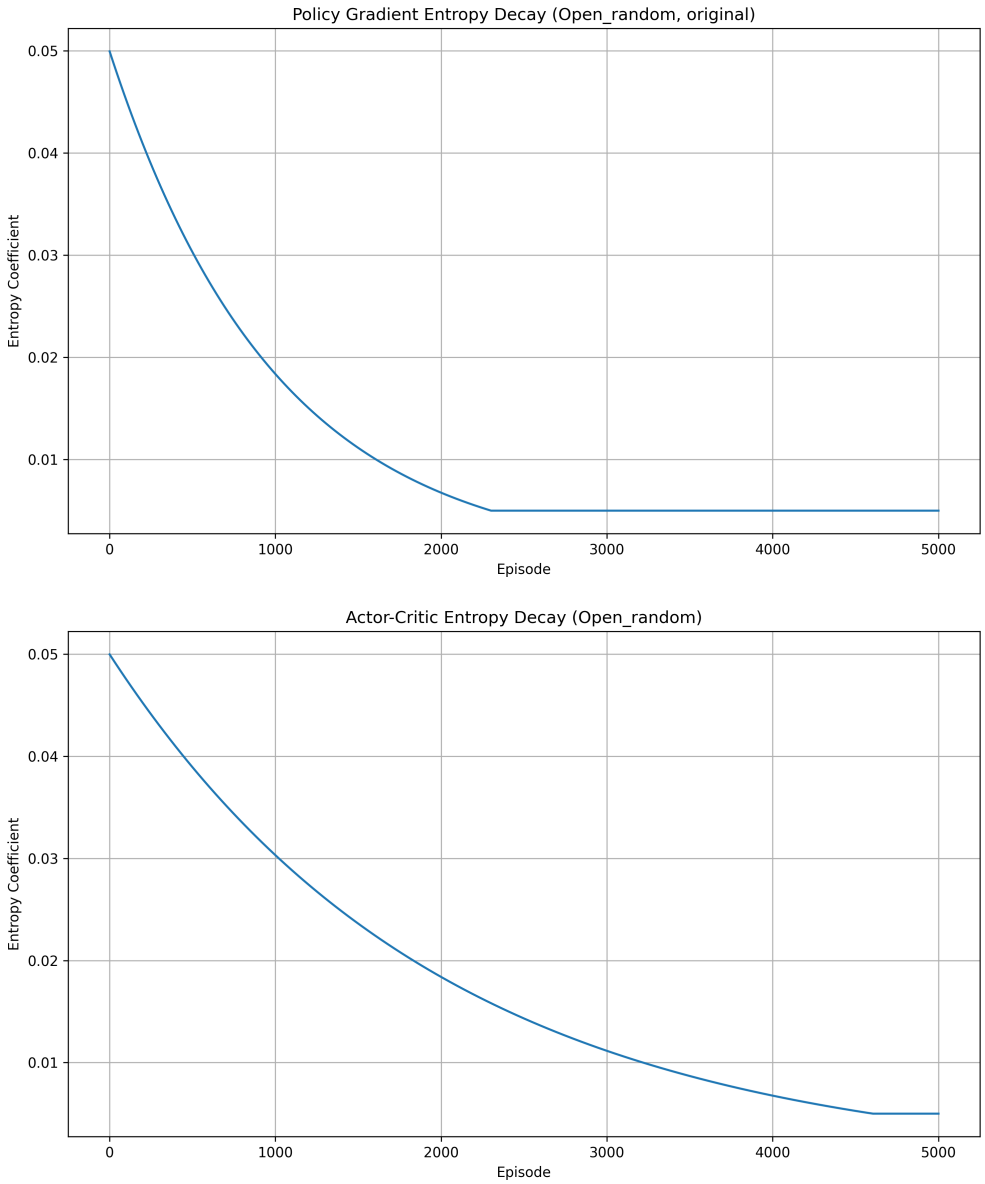


Figure A.9: Entropy decay during training in the open random environment for policy gradient and actor-critic

Declaration

I declare that this Bachelor Thesis has been completed by myself independently without outside help and only the defined sources and study aids were used.

City	Date	Signature